

Introduction to Salam

The Official Tutorial

Programming in Your Own Language —
from your first program to professional development

A bilingual language for everyone:
students, hobbyists, and professional developers

The Salam Language Team

First Edition

This is the official tutorial for the Salam programming language. Every code listing in this book was tested with the official **salam** compiler and is meant to compile and run as shown. Salam is free software; the language, its compiler, and its standard library are developed in the open.

The Salam logo and name belong to the Salam Language Team. This book may be shared and adapted for educational purposes.

First Edition. Typeset with X_YL^AT_EX.

Preface

Why Salam exists

Almost every programming language in wide use today was designed around English. Its keywords are English words — `if`, `while`, `return` — and its tutorials, error messages, and documentation assume that the reader is comfortable reading English. For hundreds of millions of people whose first language is Persian or Arabic, that assumption is a wall to climb before the real learning can even begin.

Salam (from the word سلام, meaning *peace*) was created to lower that wall. It is, at its heart, an ordinary modern programming language: statically typed, compiled, fast, with structs, generics, interfaces, closures, a real standard library, and the ability to call C. What makes it unusual is that it is *bilingual*. The very same language can be written with English keywords or with Persian keywords, and both produce identical programs. A loop is spelled `while` in one and `تا وقتی` in the other; a function is `func` or `تابع`. The compiler, `salam`, understands both.

This means a student in Tehran can write their first program in their own language, reading their own script, with error messages in their own words — and then, when they are ready, switch the very same concepts into English to work with the wider software world. The ideas transfer perfectly, because they are *the same ideas*.

Who this book is for

This book assumes *no prior programming experience*. If you have never written a line of code in your life, start at Chapter 1 and walk forward one step at a time — every term is explained the first time it appears, and every concept is shown with a small, complete program you can run yourself.

If you are already a programmer coming from C, Python, Go, or Rust, you

will find Salam comfortably familiar. You can move quickly through the early chapters, treat the *In Depth* boxes as your main course, and use the appendices as a quick reference. Salam will feel like a language you already half-know.

The book is therefore written on two levels at once:

- The **main text** is friendly and gradual. It uses plain language, small examples, and analogies, and it never assumes you have seen a concept before.
- The **In Depth** boxes dig into how things actually work — the memory model, monomorphization, the C that Salam generates, performance trade-offs. They are clearly marked as optional. Skip them on a first reading; come back when you are curious.

How to use this book

Read with your hands, not just your eyes. Programming is a craft, and crafts are learned by doing. Every chapter contains complete programs — type them in, run them, and then *change* them. Break them on purpose and see what error the compiler gives. The single most effective way to learn a language is to argue with its compiler until you both agree.

Each chapter ends with exercises. Attempt them before peeking at Appendix D, where solutions to selected exercises are collected. The boxes sprinkled through the text mean:

- **Note** — a clarification or a fact worth remembering.
- **Tip** — a practical shortcut or good habit.
- **Watch out** — a common mistake; read these twice.
- **In Depth** — optional, deeper material for the curious.

A note on scope

Salam also ships with a built-in *layout* dialect for describing web pages (it compiles small `layout`: blocks straight to HTML and CSS). That side of the language is a topic of its own and is *not* covered here. This book is about Salam as a general-purpose programming language: variables, types,

control flow, functions, data structures, the standard library, concurrency, and the foreign function interface. Everything you learn here is pure Salam programming.

Note

All the example programs in this book are drawn from, or directly modelled on, the real, tested examples that ship with the Salam compiler (under `examples/en/` and `tests/en/`). They are not pseudo-code; they compile and run.

Conventions

Salam source code is shown in framed, syntax-highlighted boxes with line numbers:

```
1 func main:
2     println "Hello, Salam!"
3 end
```

Terminal commands and program output are shown in plain grey boxes:

```
$ salam exec hello.salam
Hello, Salam!
```

Inline code such as `func` or a filename like `hello.salam` appears in a monospaced font. When a Persian keyword is shown beside its English form, it appears like this: `func` / تابع.

Welcome to Salam. Let us begin.

— The Salam Language Team

Contents

Preface	i
1 Getting Started	1
1.1 What is a program?	1
1.2 What is a compiler, and why do we need one?	2
1.3 Getting the compiler	2
1.4 Your first program: Hello, Salam!	3
1.4.1 Running the program	3
1.5 Anatomy of the first program	4
1.6 The same program in Persian	5
1.7 Comments: notes to yourself	6
1.8 Printing with and without a new line	7
1.9 Starting a new project	8
1.10 Chapter summary	8
1.11 Exercises	9
2 Variables and Data Types	11
2.1 Variables: named boxes	11
2.2 Mutable and immutable variables	12
2.3 Type inference with <code>auto</code>	13
2.4 The primitive types	14
2.4.1 Integers	14
2.4.2 Floating-point numbers	14
2.4.3 Booleans	15

2.4.4	Characters	15
2.4.5	Strings	16
2.4.6	Void	16
2.5	Literals: writing values directly	16
2.6	Converting between types with <code>as</code>	17
2.7	Type aliases	18
2.8	Naming variables	18
2.9	Persian type names	18
2.10	In Depth: how values are stored	19
2.11	Chapter summary	20
2.12	Exercises	20
3	Operators and Expressions	23
3.1	Arithmetic operators	23
3.1.1	Integer versus floating-point division	24
3.1.2	The power operator	24
3.2	Compound assignment	25
3.3	Comparison operators	25
3.4	Logical operators	26
3.5	A note on bitwise operators	27
3.6	String concatenation	27
3.7	Precedence and grouping	28
3.8	Putting it together	28
3.9	Chapter summary	29
3.10	Exercises	29
4	Control Flow	31
4.1	The <code>if</code> statement	31
4.2	<code>else</code> : the other path	32
4.3	<code>else if</code> : choosing among many	32
4.4	The inline form	33
4.5	Conditions are just boolean expressions	33
4.6	Nesting	34

4.7	Decisions inside functions	35
4.8	In Persian	35
4.9	Chapter summary	36
4.10	Exercises	36
5	Loops	37
5.1	The <code>while</code> loop	37
5.2	The <code>for</code> loop	38
5.3	The <code>repeat</code> loop	39
5.3.1	Repeat a fixed number of times	39
5.3.2	Repeat over a range	40
5.3.3	Repeat with a step	40
5.4	Breaking out and skipping ahead	41
5.5	Nested loops	42
5.6	In Persian	42
5.7	Which loop should I use?	43
5.8	Chapter summary	43
5.9	Exercises	43
6	Functions	45
6.1	Defining and calling a function	45
6.2	Returning a value	46
6.3	Functions that return nothing	46
6.4	Multiple parameters	47
6.5	Arguments are passed by value	48
6.6	Recursion	49
6.7	Definition order does not matter	50
6.8	Function overloading	50
6.9	Use every parameter	51
6.10	Designing good functions	51
6.11	Chapter summary	52
6.12	Exercises	52

7	Arrays and Vectors	55
7.1	Fixed-size arrays	55
7.1.1	Changing elements	56
7.1.2	Walking an array with a loop	56
7.2	Multi-dimensional arrays	57
7.3	The limit of fixed arrays	57
7.4	Vectors: arrays that grow	57
7.4.1	Reading and writing elements	58
7.4.2	Walking a vector	58
7.4.3	Removing from the end	59
7.4.4	The common vector operations	59
7.5	Arrays or vectors?	60
7.6	A worked example: collecting even numbers	61
7.7	Chapter summary	61
7.8	Exercises	62
8	Strings and Characters	63
8.1	String literals	63
8.2	Strings are immutable	64
8.3	Joining strings with +	64
8.4	Escape sequences	64
8.5	Triple-quoted strings	65
8.6	Characters	65
8.7	Strings are not indexed with brackets	66
8.8	The <code>str</code> module	66
8.8.1	Case and trimming	66
8.8.2	Searching and testing	67
8.8.3	Substrings and replacement	67
8.8.4	Converting between strings and numbers	68
8.8.5	Splitting and joining	68
8.9	A worked example: counting words	69
8.10	Chapter summary	70

8.11 Exercises	70
9 Structs and Methods	73
9.1 Defining a struct	73
9.2 Creating instances	73
9.3 Reading and changing fields	74
9.4 Structs are copied by value	75
9.5 Methods: behaviour on a struct	75
9.6 Methods that compute and methods that act	76
9.7 A fuller example: a bank account	77
9.8 Structs within structs	78
9.9 Chapter summary	79
9.10 Exercises	80
10 Enums and Pattern Matching	81
10.1 Defining an enum	81
10.2 Using enum values	81
10.3 Enums are numbers underneath	82
10.3.1 Choosing the numbers	82
10.4 Pattern matching with <code>if/else if</code>	83
10.5 Enums as state machines	84
10.6 Chapter summary	85
10.7 Exercises	86
11 Interfaces and Polymorphism	87
11.1 Defining an interface	87
11.2 Implementing an interface	87
11.3 Static polymorphism: interface bounds	88
11.4 Dynamic polymorphism: <code>dyn</code>	89
11.5 Designing with interfaces	90
11.6 Chapter summary	90
11.7 Exercises	91
12 Generics	93

12.1 The problem generics solve	93
12.2 Generic functions	93
12.3 Generic structs	94
12.4 A generic container: a simple stack	95
12.5 Bounded generics revisited	96
12.6 How generics work: monomorphization	97
12.7 When to make something generic	97
12.8 Chapter summary	97
12.9 Exercises	98
13 Closures and Lambdas	99
13.1 Functions as values	99
13.1.1 The type of a function value	100
13.1.2 Multi-statement bodies	100
13.2 Higher-order functions	100
13.3 Closures: capturing the surrounding state	101
13.4 Closures that remember and change state	101
13.5 Why closures matter	102
13.6 Chapter summary	103
13.7 Exercises	103
14 Error Handling and Deferred Execution	105
14.1 Salam’s philosophy: errors are values	105
14.2 Pattern 1: a boolean success flag	105
14.3 Pattern 2: a sentinel return value	106
14.4 Pattern 3: <code>Option</code> — a value that may be absent	107
14.5 Deferred execution with <code>defer</code>	108
14.5.1 Defers run in reverse order	109
14.5.2 <code>Defer</code> runs even on an early return	109
14.6 Putting it together: <code>acquire</code> , <code>defer</code> , <code>use</code>	110
14.7 Chapter summary	111
14.8 Exercises	111

15 Modules and Packages	113
15.1 Importing the standard library	113
15.2 Importing several modules	114
15.3 Aliasing an import	114
15.4 How imports are resolved	115
15.5 Writing your own modules	115
15.5.1 Declaring a package	115
15.5.2 Exposing names with <code>pub</code>	115
15.6 A tour of the standard library's layout	116
15.7 Chapter summary	117
15.8 Exercises	118
16 The Standard Library, Part 1	119
16.1 Input and output: <code>io</code>	119
16.2 Formatting values: <code>fmt</code>	120
16.3 Mathematics: <code>math</code>	121
16.3.1 Rounding and sign	121
16.3.2 Roots, powers, and logarithms	121
16.3.3 A reference of common <code>math</code> functions	122
16.3.4 Integer helpers	122
16.4 Randomness: <code>rand</code>	123
16.5 Time: <code>time</code>	124
16.6 Strings: a reminder	124
16.7 A worked example: a guessing game	125
16.8 Chapter summary	125
16.9 Exercises	126
17 The Standard Library, Part 2	127
17.1 Collections beyond the vector	127
17.1.1 HashMap: key-value lookup	127
17.1.2 Stack and Queue	128
17.2 Reading and writing files	129
17.3 The operating system: <code>os</code>	129

17.4 Working with JSON	130
17.5 Pattern matching with regular expressions	131
17.6 Other data-format modules	132
17.7 A worked example: word frequencies	132
17.8 Chapter summary	133
17.9 Exercises	133
18 Memory Management	135
18.1 Two kinds of memory	135
18.2 Values that manage themselves	135
18.3 Containers you must free	136
18.3.1 Pairing creation with free using defer	137
18.4 No hidden ownership rules	138
18.5 Manual memory with mem	138
18.6 Chapter summary	139
18.7 Exercises	139
19 The Foreign Function Interface	141
19.1 Declaring a C function	141
19.2 How Salam types map to C	142
19.3 Calling the C math library	142
19.4 Linking external libraries	143
19.5 Working with pointers across the boundary	144
19.6 When to use the FFI	145
19.7 Chapter summary	145
19.8 Exercises	146
20 Concurrency	147
20.1 Threads: running code in parallel	147
20.2 The danger of shared data	148
20.3 Mutexes: one at a time	148
20.4 Waiting for many threads	149
20.5 When concurrency pays — and when it costs	150

20.6 Chapter summary	151
20.7 Exercises	151
21 Testing and Debugging	153
21.1 Why test?	153
21.2 Assertions with the <code>testing</code> module	153
21.2.1 The assertion functions	154
21.2.2 Reading a failure	154
21.3 Organising tests	155
21.4 Debugging: finding out why	156
21.4.1 Print debugging	156
21.4.2 Let the compiler help	157
21.4.3 Heavier tools	157
21.5 Chapter summary	157
21.6 Exercises	158
22 Bilingual Programming in Salam	159
22.1 One language, two vocabularies	159
22.2 The keyword equivalents	160
22.3 Compiling a Persian program	161
22.4 What you can mix	162
22.5 Why bilingualism matters	162
22.6 Chapter summary	163
22.7 Exercises	164
23 Advanced Topics and Best Practices	165
23.1 Performance: fast by default, faster with care	165
23.2 Structuring a program	166
23.3 Coding style	167
23.4 Common pitfalls, revisited	167
23.5 Where to go next	168
23.6 A closing word	168
23.7 Chapter summary	169

23.8 Exercises	169
A Keyword Reference (English / Persian)	171
B Grammar (Simplified)	175
C Cheat Sheet	181
D Solutions to Selected Exercises	187

Chapter 1

Getting Started

In this chapter we take the very first steps: we get the Salam compiler ready, write our first program, and run it. Do not worry if you have never written a line of code before — this chapter takes you by the hand and walks you forward one step at a time.

1.1 What is a program?

A computer is a machine that is extraordinarily fast and extraordinarily literal. It does not know what you want; you must *tell* it, in exact detail. A list of instructions, written in order so that the computer can carry out a task, is called a **program**. The language we write those instructions in is called a **programming language**.

Salam is one such language, with one special quality: you can write your instructions in *Persian* as comfortably as in English. The instruction “print something on the screen” is written `println "Salam"` in English, or چاپ "سلام" in Persian. Both mean exactly the same thing to the computer.

Note

A computer does *precisely* what you tell it — no more, no less. So when a program does not behave the way you expected, the mistake is almost always in the instructions you gave, not in the machine. This is completely normal; even professional programmers run into it every single day. Learning to read what the computer actually did, rather than what you *meant*, is half the craft.

1.2 What is a compiler, and why do we need one?

The program you write is meant to be readable by a *human*. The computer, however, does not understand it directly — deep down it only understands tiny numeric instructions. We need a program that translates what we wrote into something the machine can run. That translator is called a **compiler**. The compiler for the Salam language is called **salam**.

salam does a great deal of work for us: it reads the text of your program, checks that it makes sense, reports any mistakes, and finally produces something you can run.

In Depth: the journey of a program (optional, for advanced readers)

When **salam** processes your program, it passes through several stages: *lexing* (breaking the text into tokens), *parsing* (building a tree that represents the structure), *semantic analysis* (checking types and names), and finally *code generation*. Salam’s main backend translates your program into C source code and then invokes a C compiler (**tcc** by default, but **gcc** or **clang** work too) to produce a native executable. There is also a built-in *interpreter* that can run your program directly without any C toolchain, and an experimental LLVM backend. We will refer to these stages occasionally, but you never need to know their details to write programs.

1.3 Getting the compiler

To work with Salam you need the **salam** compiler. It is built from source. In the root folder of the Salam project, run:

```
sh tools/build-compiler.sh
```

When it finishes, an executable called **salam** (or **salam.exe** on Windows) appears in the project folder. Alternatively you can build with CMake, which also builds the runtime library and enables the test suite:

```
cmake -B build && cmake --build build
```

Tip

To confirm the compiler is working, ask it for its version:

```
$ salam version
```

```
salam 0.2.0
```

Note

Throughout this book we assume `salam` is reachable from your terminal: either it sits in your current folder (call it as `./salam`) or it is installed on your system `PATH` (call it as `salam`). For brevity we will simply write `salam` everywhere.

1.4 Your first program: Hello, Salam!

There is a long-standing tradition among programmers: the very first program you write in a new language is one that displays the words “Hello, World!”. Let us honour that tradition.

Create a file named `hello.salam` and type these few lines into it:

```
1 func main:
2     name str = "Salam"
3     println "Hello,", name
4     println "2 + 3 =", 2 + 3
5 end
```

Note

Every Salam program file ends in `.salam`. The extension tells the tools (and you) that the file contains Salam source code.

1.4.1 Running the program

The simplest way to see the result is to use Salam’s built-in interpreter, which runs your program directly — no C compiler required. In your terminal, type:

```
salam exec hello.salam
```

and the output will be:

```
Hello, Salam
2 + 3 = 5
```

If you would rather build a standalone executable (which runs faster and is what you ship to other people), use the `build` command:

```
$ salam build hello.salam --output=hello.exe
$ ./hello.exe
Hello, Salam
2 + 3 = 5
```

There is also a one-shot `run` command that builds and immediately runs without leaving any files behind:

```
salam run hello.salam
```

Tip

What is the difference between `exec`, `run`, and `build`? `exec` runs the program *immediately* with the interpreter and needs no C toolchain — it is perfect for learning and quick experiments. `build` produces a native executable you can run again and again, and is what you use to deliver a finished program. `run` is the convenient middle ground: compile and run once, keep nothing. Throughout most of this book we will use `exec` for simplicity.

1.5 Anatomy of the first program

Let us now go through the program line by line. Do not skim any part of this — these few lines are the doorway to every concept that follows.

Line 1 — the start of the program.

```
1 func main:
```

This line says: “define a **function** named **main**.” A function is a named bundle of instructions. The function named `main` is the *starting point* of every Salam program: when you run the program, the computer begins here. The colon (`:`) at the end of the line says that the body of the function starts on the next line.

Line 2 — a variable.

```
1     name str = "Salam"
```

Here we create a **variable** named `name`. Picture a variable as a labelled box that holds a value. In this box we have placed the text `"Salam"`. The word `str` states that the box holds *text* (more on types in the next chapter). Any piece of text is written between double quotes (`"`). Notice that this line

begins with a few spaces of **indentation**; that whitespace shows the line lives *inside* the `main` function.

Line 3 — printing output.

```
1 println "Hello,", name
```

The word `println` (“print line”) tells the computer to display something on the screen and then move to the next line. Here we display two things: the fixed text `"Hello,"` and then the contents of the variable `name` (which is `"Salam"`). The two are separated by a comma (`,`), and `Salam` puts a single space between them. That is why the output reads `Hello, Salam`.

Line 4 — a little arithmetic.

```
1 println "2 + 3 =", 2 + 3
```

`Salam` does not only print text; it can calculate too. The expression `2 + 3` is worked out *before* printing, and the result (5) is displayed. Notice the difference: `"2 + 3 ="` is inside quotes, so it is a piece of *text* printed exactly as written; but `2 + 3` is outside quotes, so it is a *calculation*.

Watch out

Text in quotes is printed verbatim; code outside quotes is evaluated. Writing `println "2 + 3"` would print the three characters `2 + 3`, not the number 5. The quotes make all the difference.

Last line — the end of the function.

```
1 end
```

The word `end` tells `Salam` that the body of `main` is finished. Any block that opens with a colon (`:`) must be closed with `end`. This is a rule you will see throughout the language: *blocks open with a colon and close with end*.

1.6 The same program in Persian

One of the joys of `Salam` is that any program can be written in Persian too. This is the exact Persian equivalent of the program above:

```

1 تابع اصلی:
2 نام رشته = "سلام"
3 چاپ "سلام", نام
4 چاپ "3 + 2", "= 3 + 2"
5 پایان

```

The structure is identical; only the words changed: `func` ↔ `تابع`, `main` ↔ `اصلی`, `str` ↔ `رشته`, `println` ↔ `چاپ`, and `end` ↔ `پایان`. A full table of these equivalents is in Appendix A. To compile a Persian file, tell the compiler the language:

```
salam exec hello_fa.salam --lang=fa
```

Note

The entry function's name is itself part of the language: in English it is `main`, in Persian `اصلی`. We will return to bilingual programming properly in Chapter 22. For now, just notice that the *ideas* are the same in both languages — only the spelling of the keywords differs.

1.7 Comments: notes to yourself

A **comment** is text that the compiler ignores completely. Comments are written for *humans* — to explain what a piece of code does, or to leave a reminder. A line comment starts with `//` and runs to the end of the line. A block comment is wrapped in `/* ... */` and may span several lines.

```

1 func main:
2     // This is a line comment: the compiler skips it.
3     println "working"           // comments can also follow code
4
5     /* This is a block comment.
6        It can stretch across
7        as many lines as you like. */
8     println "done"
9 end

```

```

working
done

```

Tip

Good comments explain *why*, not *what*. The code already says what it does; a comment is most valuable when it records the reasoning that the code cannot show by itself.

1.8 Printing with and without a new line

Salam has two main ways to display output:

- `println` — displays its arguments and *then moves to the next line*.
- `print` — displays its arguments but *stays on the same line*; whatever is printed next continues right after it.

Look at this example:

```
1 func main:
2   print "Salam "
3   print "world"
4   println ""
5   println "a fresh line"
6 end
```

```
Salam world
a fresh line
```

The two `print` calls both wrote on the same line, and then `println ""` (which prints an empty piece of text) moved us down to the next line. This simple trick — writing with `print` and ending the line with `println ""` — will be used again and again in the chapter on loops to draw shapes.

Note

Salam gives these two so often that it provides short aliases: `_` is the same as `print`, and `__` is the same as `println`. So `_ "hi"` and `print "hi"` are identical, and `__ "hi"` equals `println "hi"`. You will meet the aliases in some of the bundled examples; this book prefers the spelled-out names for clarity.

In Depth: other ways to write the same thing (optional, for advanced readers)

Salam is flexible about a few surface details. Blocks may be written with a colon and `end`, as in this book, *or* with curly braces:

```
1 func main { println("Hello, World!") }
```

Likewise, a call's arguments may be wrapped in parentheses (`println("hi")`) or not (`println "hi"`). The two styles are equivalent; the colon/`end` form with bare arguments is the style used by most of the examples that ship with Salam, so it is the style we adopt here. Pick one and be consistent — and remember the `salam fmt` command, which reformats your code into the canonical style automatically.

1.9 Starting a new project

When you begin something larger than a single file, the compiler can scaffold a project folder for you:

```
salam new myproject --lang=en
```

This creates a `myproject` directory with a starter program already in place, so you can build and run it immediately. For the small examples in the coming chapters, a single `.salam` file is all you need.

1.10 Chapter summary

In this chapter we learned that:

- A program is a list of instructions, and the compiler `salam` translates it for the computer.
- Every Salam program starts at the `main` function (اصلي in Persian).
- Blocks open with a colon (`:`) and close with `end`.
- `println` prints and moves to a new line; `print` prints and stays on the line.
- Text goes in double quotes; code outside quotes is evaluated.
- Comments (`//` and `/* ... */`) are ignored by the compiler.

- `salam exec` runs a program instantly; `salam build` makes a standalone executable.
- Any program can be written in Persian or English with identical meaning.

1.11 Exercises

Exercise 1.1. W

rite a program that prints your name and your age on two separate lines.

Exercise 1.2. W

rite a program that prints the result of `12 * 8` together with a short label, for example: `12 * 8 = 96`.

Exercise 1.3. W

rite “Hello, World!” once in English and once in Persian, and run both. Is the output the same?

Exercise 1.4. U

sing `print` (not `println`), display your first and last name on a *single* line with one space between them, then move to a new line.

Exercise 1.5. A

dd a block comment at the top of one of your programs describing what it does, then run it and confirm the comment changes nothing.

Chapter 2

Variables and Data Types

In Chapter 1 we used a variable without dwelling on it. Now we slow down and look properly at the two ideas that underpin every program: *variables* (named places to keep values) and *types* (what kind of value a place may hold). Get these right and everything else in the language falls into place.

2.1 Variables: named boxes

A **variable** is a name attached to a value. Picture a labelled box: the label is the name, and the contents are the value. You create a variable by writing its name, its type, and (usually) an initial value:

```
1 func main:
2     age i32 = 30
3     name str = "Sara"
4     pi f64 = 3.14159
5     println name, "is", age, "years old"
6 end
```

Sara is 30 years old

Read each declaration left to right: a *name*, then a *type*, then = and an *initial value*. So `age i32 = 30` means “make a variable called `age`, of type `i32` (a 32-bit integer), holding 30.”

Note

Unlike some languages, Salam does *not* use a keyword such as `let` or `var` to introduce a variable. The shape “*name type = value*” is enough. This keeps declarations short and lines up the names neatly down the

left edge of your code.

2.2 Mutable and immutable variables

By default, a variable in Salam is **immutable**: once you give it a value, that value cannot change. This is a deliberate and very useful safety net — it means a value you thought was fixed really is fixed. Try to reassign an immutable variable and the compiler stops you:

```
1 func main:
2     score i32 = 10
3     score = 20      // error: 'score' is immutable
4 end
```

When you genuinely need a value to change over time — a counter, a running total, a loop variable — mark it **mutable** with the `mut` keyword:

```
1 func main:
2     mut score i32 = 10
3     score = 20
4     score = score + 5
5     println score
6 end
```

25

There is also `const`, for a value that is fixed *and* known when the program is compiled — a true constant, such as a maximum size or a mathematical constant:

```
1 const MAX_USERS = 100
2
3 func main:
4     println "capacity:", MAX_USERS
5 end
```

capacity: 100

So there are three flavours of binding, from most to least restrictive:

- `const` — fixed, and computed at compile time. Use for true constants. Conventionally written in `UPPER_CASE`.

- `plain` (*bare*) — fixed once set at run time, but computed while the program runs. This is the default; prefer it.
- `mut` — may be reassigned. Use only when a value really must change.

Tip

Reach for `mut` only when you must. “Immutable by default” is one of Salam’s quiet strengths: the fewer things that can change, the fewer things can go wrong, and the easier your program is to reason about. If the compiler complains that something is immutable, pause and ask whether it truly needs to change before adding `mut`.

2.3 Type inference with `auto`

Often the type is obvious from the value, and writing it feels redundant. The `auto` keyword asks the compiler to *infer* the type from the initializer:

```
1 func main:
2   count auto = 42           // inferred as i32
3   ratio auto = 3.14         // inferred as f64
4   label auto = "items"      // inferred as str
5   ready auto = true         // inferred as bool
6   println count, ratio, label, ready
7 end
```

```
42 3.14 items true
```

`auto` is about convenience, not weakness: the variable still has one fixed type, decided once, at compile time. `count` above is exactly an `i32` — you simply did not have to spell it out. You can combine `auto` with `mut`: `mut total auto = 0` makes a mutable `i32`.

Watch out

`auto` needs an initializer to infer from. `x auto` on its own is meaningless — there is nothing to deduce the type from — and the compiler will reject it. When in doubt, or when you want to document intent, write the type out explicitly.

2.4 The primitive types

Salam’s built-in scalar types are the atoms from which everything larger is built. Here are the ones you will use constantly.

2.4.1 Integers

Integers are whole numbers. Salam offers them in several sizes, both *signed* (able to hold negatives) and *unsigned* (zero and up only). The number in the name is how many bits the value occupies, which fixes its range:

Type	Signed?	Bits	Range
<code>i8</code>	signed	8	−128 ... 127
<code>i16</code>	signed	16	−32768 ... 32767
<code>i32</code>	signed	32	about ±2.1 billion
<code>i64</code>	signed	64	about ±9.2 quintillion
<code>u8</code>	unsigned	8	0 ... 255
<code>u16</code>	unsigned	16	0 ... 65535
<code>u32</code>	unsigned	32	0 ... about 4.3 billion
<code>u64</code>	unsigned	64	0 ... about 18 quintillion

You do not need to memorise these. For everyday whole numbers, reach for `i32` — it is the default and is almost always the right choice. Step up to `i64` when you might exceed two billion (timestamps, file sizes, big counters), and use the small or unsigned types only when you have a specific reason, such as raw bytes (`u8`) or interfacing with C.

Note

Three of these have friendlier alias names: `int` is another spelling of `i32`, `uint` of `u32`, and `float` of `f32`. They are exactly the same types — the compiler treats `int` and `i32` as identical — so use whichever reads better to you. This book mostly uses the explicit `i32`.

2.4.2 Floating-point numbers

For numbers with a fractional part — measurements, averages, scientific values — use a floating-point type:

- `f64` — a 64-bit “double precision” number. This is the default for decimal literals and the one to use unless you have a reason not to.

- `f32` (alias `float`) — a 32-bit number; less precise, but smaller. Useful for graphics or large arrays where space matters.

```
1 func main:
2     temperature f64 = 36.6
3     half auto = 7.0 / 2.0      // 3.5, a floating-point division
4     println temperature, half
5 end
```

```
36.6 3.5
```

2.4.3 Booleans

A `bool` holds one of just two values: `true` or `false`. Booleans are the language of decisions — every `if` and every `while` in the coming chapters tests a `bool`.

```
1 func main:
2     raining bool = false
3     weekend auto = true
4     println "raining?", raining
5     println "weekend?", weekend
6 end
```

```
raining? false
weekend? true
```

2.4.4 Characters

A `char` is a single byte — one character in the basic Latin range, or any value from 0 to 255. Character literals are written between *single* quotes:

```
1 func main:
2     grade char = 'A'
3     println grade
4     println grade as i32      // the numeric code of 'A'
5     println 66 as char       // the character with code 66
6 end
```

```
A
65
B
```

A `char` is really a small number in disguise: `'A'` is 65. You can convert between the two with `as`, which is how the example turns `'A'` into 65 and 66 back into `'B'`.

Note

A single quote makes a `char`; a double quote makes a `str`. So `'A'` is one character, while `"A"` is a string that happens to be one character long. They are different types. We meet strings properly in Chapter 8.

2.4.5 Strings

A `str` is a piece of text — a sequence of characters — written between double quotes. Strings are *immutable*: operations on them produce new strings rather than changing the original. You can join two strings with `+`:

```
1 func main:
2     first str = "Salam"
3     greeting auto = first + ", world!"
4     println greeting
5 end
```

Salam, world!

2.4.6 Void

Finally, `void` is the “no value” type. You never store a `void`; it appears only as the result type of a function that returns nothing (a function that *does* something rather than *computes* something). We return to it in Chapter 6.

2.5 Literals: writing values directly

A **literal** is a value written straight into your program. Each kind of literal has a natural type:

Literal	Type	Examples
whole number	<code>i32</code>	0, 42, -7
decimal number	<code>f64</code>	3.14, 2.0, 1.5e3
character	<code>char</code>	<code>'a'</code> , <code>'9'</code>
text	<code>str</code>	<code>"hi"</code> , <code>""</code>
truth value	<code>bool</code>	<code>true</code> , <code>false</code>

This leads to one rule that surprises newcomers and is worth learning now.

Watch out

A plain whole-number literal is an `i32`. Salam does not silently shrink it to fit a smaller or unsigned type. So this is an *error*:

```
1 mut b u8 = 250           // error: cannot assign 'i32' to 'u8'
```

The literal `250` is an `i32`, and Salam will not narrow it for you without being asked. The fix is an explicit cast with `as`:

```
1 mut b u8 = 250 as u8     // fine
```

This strictness is a feature: it makes every change of type visible in the source, so a narrowing can never happen by accident.

2.6 Converting between types with `as`

Salam never converts a value from one type to another behind your back. When you *do* want a conversion, you ask for it explicitly with the `as` operator:

```
1 func main:
2     total i32 = 7
3     count i32 = 2
4     average f64 = (total as f64) / (count as f64)
5     println "average =", average
6 end
```

```
average = 3.5
```

Why the casts? Because `total` and `count` are integers, `total / count` would be *integer* division — it would compute 3, throwing away the remainder. By converting each to `f64` first, we ask for real division and get 3.5.

Watch out

Integer division truncates toward zero: `7 / 2` is 3, not 3.5, and `7 % 2` (the remainder) is 1. If you want a fractional result, at least one operand must be a floating-point value. This is one of the most common beginner surprises — keep it in mind.

2.7 Type aliases

When a type’s meaning matters more than its mechanics, you can give it a clearer name with `type`:

```
1 type Age = u32
2
3 func main:
4     a Age = 30 as u32
5     println a
6 end
```

```
30
```

An alias is *transparent*: `Age` and `u32` are completely interchangeable — the alias is just a more readable spelling. Aliases come into their own with the bigger types we build later; for now, simply know the keyword exists.

2.8 Naming variables

A good name is documentation that never goes out of date. Salam identifiers may contain letters, digits, and underscores, and may not start with a digit. Crucially, they may also contain *Unicode* letters — so Persian names are first-class:

```
1 func main:
2     user_count i32 = 5
3     maxScore auto = 100
4     println user_count, maxScore
5 end
```

Tip

Choose names that say what the value *means*, not what type it is. `user_count` tells the reader far more than `n` or `i32val`. The extra keystrokes pay for themselves the first time you re-read your own code a week later.

2.9 Persian type names

Just as the keywords have Persian forms, so do the types. A Persian program writes the very same declarations with Persian type names:

English	Persian
i32	صحیح ۳۲
i64	صحیح ۶۴
f64	اعشاری ۶۴
bool	منطقی
char	کاراکتر
str	رشته
auto	خودکار

Here is the chapter's first program written in Persian:

```

1 تابع اصلی:
2 سن صحیح ۳۲ = 30
3 نام رشته = "سارا"
4 چاپ نام, "is", سن, "years" old
5 پایان

```

Notice that the digits inside صحیح ۳۲ are Persian digits (۳۲), while the value 30 is written with ordinary digits, because arithmetic uses the standard digit characters. We will untangle this fully in Chapter 22; for now, just admire that the structure is identical to the English version.

2.10 In Depth: how values are stored

In Depth: representation and the generated C (optional, for advanced readers)

Each Salam scalar maps directly onto a C type when the program is compiled: `i32` becomes `int32_t`, `f64` becomes `double`, `bool` becomes a one-byte value, `char` a single byte, and `str` a pointer to immutable, NUL-terminated bytes. This 1-to-1 mapping is why Salam programs run at C speed and why the foreign function interface (Chapter 19) is so direct. Because the sizes are fixed and known, the compiler can check at *compile time* that 250 does not fit in an `i8` and that a `u8` and an `i32` are not accidentally mixed — the strictness you met above is the visible edge of this static type system.

2.11 Chapter summary

- A variable is declared as *name type = value*; no `let/var` keyword is needed.
- Variables are *immutable by default*; add `mut` to allow reassignment, or `const` for a compile-time constant.
- `auto` infers the type from the initializer.
- Integers come in signed (`i8–i64`) and unsigned (`u8–u64`) sizes; `i32` is the everyday default. `int/uint/float` are aliases for `i32/u32/f32`.
- `f64` is the default for decimals; `bool` is `true` or `false`; `char` is one byte; `str` is immutable text.
- A whole-number literal is an `i32`; conversions are never implicit — use `as`. Integer division truncates.
- Types have Persian names too (رشته, صحیح, ...).

2.12 Exercises

Exercise 2.1. D

Declare three variables describing a book: a title (`str`), a page count (`i32`), and a price (`f64`). Print them in one sentence.

Exercise 2.2. S

Start with `mut total i32 = 0`, add 17 and then 25 to it in two steps, and print the result after each step.

Exercise 2.3. C

Compute the average of 10, 15, and 20 as an `f64` and print it. (Remember what plain integer division would do.)

Exercise 2.4. S

Store the character 'Z' in a `char`, then print both the character and its numeric code.

Exercise 2.5. T

ry to assign 300 to a u8 variable and read the compiler's error. Then fix it with a cast and explain in a comment what the cast does.

Exercise 2.6. R

ewrite the book-description program from the first exercise using Persian type names.

Chapter 3

Operators and Expressions

So far our programs have mostly stored and printed values. Now they start to *compute*. An **operator** is a symbol that combines values to produce a new one — + adds, < compares, && reasons about truth. A combination of values and operators that produces a result is an **expression**. This chapter is a tour of Salam’s operators and the rules that decide what an expression means.

3.1 Arithmetic operators

The five everyday arithmetic operators work as you would expect:

Operator	Meaning	Example
+	addition	$3 + 4 \rightarrow 7$
-	subtraction	$10 - 6 \rightarrow 4$
*	multiplication	$6 * 7 \rightarrow 42$
/	division	$17 / 5 \rightarrow 3$
%	remainder (modulo)	$17 \% 5 \rightarrow 2$

```
1 func main:
2   println 3 + 4 * 2      // 11
3   println 17 / 5, 17 % 5  // 3 2
4   println 10 - 6, 6 * 7   // 4 42
5 end
```

```
11
3 2
4 42
```

3.1.1 Integer versus floating-point division

We met this in Chapter 2, but it is worth repeating because it trips up almost everyone once. When *both* operands of `/` are integers, you get *integer* division: the result is a whole number, with any fractional part thrown away.

```
1 func main:
2   println 7 / 2           // 3 (integer division)
3   println 7.0 / 2.0       // 3.5 (floating-point division)
4   println 7 as f64 / 2    // 3.5 (one float operand is enough)
5 end
```

```
3
3.5
3.5
```

The `%` operator gives the *remainder* of an integer division: `7 % 2` is 1, because 7 is 2 threes with 1 left over. Modulo is surprisingly useful: `n % 2 == 0` tests whether `n` is even, and `n % 10` extracts the last decimal digit of `n`.

3.1.2 The power operator

Salmon has a dedicated exponentiation operator, `**`. It raises the left operand to the power of the right and produces a *floating-point* result:

```
1 func main:
2   println 2 ** 10         // 1024
3   x f64 = 9 ** 0.5        // 3 -- a square root, since x^(1/2) =
4   println x               sqrt(x)
5 end
```

```
1024
3
```

Note

Because `**` always yields a floating-point number, `2 ** 10` is the floating value `1024.0`; it just prints as `1024` because there is no fractional part to show. If you need an integer power and the values are small, plain multiplication (or a loop, as in Chapter 5) keeps the result an integer.

3.2 Compound assignment

When you update a variable using its own current value — a very common pattern — a **compound assignment** operator says it more briefly. Each one combines an arithmetic operation with assignment:

Shorthand	Means
<code>x += y</code>	<code>x = x + y</code>
<code>x -= y</code>	<code>x = x - y</code>
<code>x *= y</code>	<code>x = x * y</code>
<code>x /= y</code>	<code>x = x / y</code>
<code>x %= y</code>	<code>x = x % y</code>

```

1 func main:
2   mut n i32 = 10
3   n += 5      // n is now 15
4   n -= 2      // 13
5   n *= 2      // 26
6   println n
7 end

```

26

Watch out

Compound assignment only works on a `mut` variable — it *changes* the variable, after all. Used on an immutable binding, it is the same immutability error you saw in Chapter 2.

3.3 Comparison operators

A comparison asks a yes/no question about two values and answers with a `bool`. There are six:

Operator	Question	Example
<code>==</code>	equal?	<code>5 == 5 → true</code>
<code>!=</code>	not equal?	<code>5 != 3 → true</code>
<code><</code>	less than?	<code>3 < 5 → true</code>
<code>></code>	greater than?	<code>3 > 5 → false</code>
<code><=</code>	less than or equal?	<code>5 <= 5 → true</code>
<code>>=</code>	greater than or equal?	<code>4 >= 5 → false</code>

```

1 func main:
2   println 5 > 3, 5 == 5, 5 != 3, 5 <= 5
3 end

```

```
true true true true
```

Watch out

Do not confuse `=` with `==`. A single `=` *assigns* a value to a variable; a double `==` *compares* two values and yields a `bool`. Writing `=` where you meant `==` is a classic slip — fortunately Salam’s static checks catch most such mistakes for you.

Comparisons also work on strings: `==` and `!=` test whether two strings have the same contents (not merely whether they are the same object), and `<`, `>`, and friends compare them in dictionary order.

```

1 func main:
2   println "abc" == "abc"    // true
3   println "abc" == "abd"    // false
4 end

```

```
true
false
```

3.4 Logical operators

Logical operators combine `bool` values — the raw material of decisions. There are three:

Operator	Meaning	True when...
<code>&&</code>	and	both sides are true
<code> </code>	or	at least one side is true
<code>!</code>	not	the operand is false

```

1 func main:
2   age i32 = 20
3   has_ticket bool = true
4   can_enter auto = age >= 18 && has_ticket
5   println can_enter

```

```
6 println true || false, !true
7 end
```

```
true
true false
```

In Depth: short-circuit evaluation (optional, for advanced readers)

`&&` and `||` are *short-circuiting*: they evaluate the right-hand side only if they must. In `a && b`, if `a` is already `false`, the whole expression is `false` regardless of `b`, so `b` is never evaluated. Likewise `a || b` skips `b` when `a` is `true`. This is not just an optimization — it lets you write a guard and a use of it in one line, e.g. testing that a count is positive *before* dividing by it, knowing the risky part runs only when it is safe.

Note

Salam spells the logical operators with symbols (`&&`, `||`, `!`), not words. There is no `and`, `or`, or `not` keyword.

3.5 A note on bitwise operators

Some languages provide *bitwise* operators (`&`, `|`, `<<`, and so on) that manipulate the individual bits of an integer. The current version of Salam does *not* include them — writing `6 | 1`, for instance, is a syntax error (the compiler even reminds you that `||` is the logical “or”). If you need bit manipulation today, you can fall back to arithmetic (multiplying or dividing by powers of two shifts bits) or call a C function through the foreign function interface of Chapter 19.

3.6 String concatenation

You have already seen that `+` joins two strings end to end. This is the same `+` symbol as numeric addition, but Salam picks the right meaning from the types of the operands: two numbers add, two strings concatenate.

```
1 func main:
2     first str = "Salam"
3     full auto = first + ", " + "world!"
4     println full
5 end
```

```
Salam, world!
```

We devote all of Chapter 8 to strings; for now, just remember that `+` between two `str` values builds a new, larger string.

3.7 Precedence and grouping

When several operators appear in one expression, **precedence** decides who acts first. The rules follow ordinary mathematics: `**` binds tightest, then `*`, `/`, and `%`, then `+` and `-`, then the comparisons, and finally the logical operators. So

```
1 func main:
2     println 3 + 4 * 2           // 11, not 14: * happens before +
3     println 2 + 3 > 4           // true: 2+3 first, then compare
4 end
```

```
11
true
```

When in doubt — or simply to make your intent unmistakable — add parentheses. They cost nothing and override any precedence rule:

```
1 func main:
2     println (3 + 4) * 2         // 14: parentheses force + first
3 end
```

```
14
```

Tip

You do not need to memorise the full precedence table. Two habits cover almost everything: remember that multiplication and division come before addition and subtraction, and reach for parentheses whenever an expression is even slightly hard to read at a glance. Clear beats clever.

3.8 Putting it together

Here is a small program that uses several operator families at once: it decides whether a year is a leap year, a genuinely useful rule built entirely from `%`, comparison, and logical operators.

```
1 func main:
2     year i32 = 2024
3     leap auto = (year % 4 == 0 && year % 100 != 0) || (year % 400 ==
4         0)
5     println year, "leap?", leap
end
```

```
2024 leap? true
```

Read the condition in English: a year is a leap year if it is divisible by 4 but *not* by 100 — unless it is also divisible by 400. The operators let us say that precisely. In Chapter 4 we will use such conditions to make the program actually *do* different things.

3.9 Chapter summary

- Arithmetic: `+` `-` `*` `/` `%`, plus `**` for powers (which yields a floating-point result).
- Integer `/` truncates; use a float operand for fractional results. `%` gives the remainder.
- Compound assignment (`+=`, `-=`, ...) updates a `mut` variable in place.
- Comparisons (`==` `!=` `<` `>` `<=` `>=`) yield a `bool` and also work on strings.
- Logical operators are `&&`, `||`, `!`, and they short-circuit. There are no word forms and no bitwise operators.
- `+` concatenates strings.
- Precedence follows mathematics; parentheses override it and aid clarity.

3.10 Exercises

Exercise 3.1. P

rint the results of `20 / 3`, `20 % 3`, and `20.0 / 3.0`. Explain in a comment why the first two differ from the third.

Exercise 3.2. G

iven `mut x i32 = 100`, use compound assignment to halve it, then add 7, then print it.

Exercise 3.3. W

rite a boolean expression that is `true` exactly when an integer `n` is between 1 and 12 inclusive, and test it with a couple of values.

Exercise 3.4. W

ithout running it first, predict the value of `2 + 3 * 4 > 10 && 1 < 2`. Then run it and check.

Exercise 3.5. B

uild a person's full name from `first` and `last` string variables with a space between them, using `+`, and print it.

Chapter 4

Control Flow

Until now our programs have run straight through, top to bottom, doing the same thing every time. Real programs make *decisions*: do this when the user is old enough, do that otherwise. The tool for choosing between paths is the `if` statement, and it turns the boolean expressions of Chapter 3 into action.

4.1 The `if` statement

An `if` runs a block of code only when a condition is true. The condition is any expression of type `bool`; the block opens with a colon and closes with `end`:

```
1 func main:
2   age i32 = 20
3   if age >= 18:
4     println "You may enter."
5   end
6   println "Done."
7 end
```

```
You may enter.
Done.
```

If `age` were 15, the condition `age >= 18` would be `false`, the indented block would be skipped entirely, and only `Done.` would print. The structure is always the same: `if`, a condition, a colon, the body, and `end`.

4.2 else: the other path

Often you want one thing to happen when the condition holds and a *different* thing otherwise. That is what `else` is for:

```
1 func main:
2   age i32 = 15
3   if age >= 18:
4     println "You may enter."
5   else:
6     println "Sorry, too young."
7   end
8 end
```

Sorry, too young.

Exactly one of the two blocks runs — never both, never neither. Think of it as a fork in the road: the condition decides which way to go.

4.3 else if: choosing among many

When there are more than two possibilities, chain them with `else if`. Salam checks each condition in order and runs the block belonging to the *first* one that is true; if none match, the final `else` (if present) runs.

```
1 func main:
2   score i32 = 83
3   if score >= 90:
4     println "Grade: A"
5   else if score >= 80:
6     println "Grade: B"
7   else if score >= 70:
8     println "Grade: C"
9   else:
10    println "Grade: F"
11  end
12 end
```

Grade: B

The order matters. 83 satisfies `>= 80`, so we print `B` and stop — the remaining branches are not even tested. Because the checks run top to bottom, arrange them from the most specific or highest threshold downward, as here.

4.4 The inline form

When a branch's body is a single short statement, you may write it on the same line as the colon. This keeps tidy little decisions compact:

```
1 func main:
2     n i32 = 7
3     if n % 2 == 0: println "even"
4     else:          println "odd"
5     end
6 end
```

```
odd
```

The `end` is still required; only the line breaks change. Many of the example programs that ship with Salam use this form to line up short branches neatly, like a small table. Use it when it improves readability, and the multi-line form when a body grows beyond a single statement.

4.5 Conditions are just boolean expressions

There is nothing special about the condition in an `if` — it is any expression that produces a `bool`, exactly the kind you built in Chapter 3. That means you can combine tests with `&&`, `||`, and `!`:

```
1 func main:
2     age i32 = 20
3     has_ticket bool = true
4     if age >= 18 && has_ticket:
5         println "Welcome to the show."
6     else:
7         println "Entry denied."
8     end
9 end
```

```
Welcome to the show.
```

You can also store a condition in a `bool` variable first and test that — sometimes a well-named boolean makes the intent far clearer than a long condition crammed into the `if`:

```
1 func main:
2     age i32 = 20
```

```
3   has_ticket bool = true
4   let_in auto = age >= 18 && has_ticket
5   if let_in:
6       println "Welcome to the show."
7   end
8 end
```

Watch out

Salam’s conditions must be genuine `bool` values. Unlike some languages, you cannot use a number as a shortcut for “truth” — there is no rule that `0` means false and everything else means true. Write the comparison out: `count != 0` rather than just `count`. This keeps conditions explicit and removes a whole category of subtle bugs.

4.6 Nesting

The body of an `if` is an ordinary block, so it may contain another `if`. This is called *nesting*, and it lets you ask a follow-up question only after a first one has been answered:

```
1 func main:
2     logged_in bool = true
3     is_admin bool = false
4     if logged_in:
5         if is_admin:
6             println "Welcome, administrator."
7         else:
8             println "Welcome, user."
9         end
10    else:
11        println "Please log in."
12    end
13 end
```

Welcome, user.

Tip

Deep nesting quickly becomes hard to follow. Two habits keep it shallow: combine related tests with `&&` instead of nesting two `ifs`, and — once you reach Chapter 6 — use an early `ret` to handle a special case and leave the main path un-indented. If you find yourself four levels deep,

that is usually a sign to restructure.

4.7 Decisions inside functions

Conditions become especially powerful inside functions (the subject of Chapter 6), where a branch can *return* a result. Here is a function that classifies a score, using `else if` with the inline `ret` form:

```

1 func grade(score i32) str:
2     if score >= 90: ret "A"
3     else if score >= 80: ret "B"
4     else if score >= 70: ret "C"
5     else: ret "F"
6     end
7 end
8
9 func main:
10     println grade(95), grade(83), grade(71), grade(50)
11 end

```

A B C F

Each branch hands back a different string, and the first matching one wins. We will return to functions properly soon; for now, notice how naturally `if/else if` expresses “pick the right answer.”

4.8 In Persian

Conditionals read just as naturally in Persian. The keywords are `اگر` for `if`, `وگرنه` for `else`, and `پایان` for `end`:

```

1 تابع اصلی:
2 سن صحیح ۳۲ = 15
3 اگر سن <= 18:
4 چاپ "بزرگسال"
5 وگرنه:
6 چاپ "کم سن"
7 پایان
8 پایان

```

An `else if` chain is written by combining `وگرنه` and `اگر`, exactly as in English.

4.9 Chapter summary

- `if cond`: runs a block only when `cond` (a `bool`) is true; close it with `end`.
- `else` provides the alternative path; exactly one branch runs.
- `else if` chains several conditions; the first true one wins.
- A single-statement branch may be written inline after the colon.
- Conditions are ordinary boolean expressions and must be genuine `bools` — numbers are not `truthy`.
- `ifs` may be nested, but prefer `&&` and early returns to keep nesting shallow.

4.10 Exercises

Exercise 4.1. W

Write a program that reads a fixed integer variable `temp` and prints "freezing" if it is below 0, "cold" if below 15, "warm" if below 30, and "hot" otherwise.

Exercise 4.2. U

Using a single `if` with `&&`, print "valid" only when a variable `hour` is between 0 and 23 inclusive.

Exercise 4.3. W

Write a function `sign(n i32) str` that returns "negative", "zero", or "positive", and test it with three values.

Exercise 4.4. R

Rewrite the grade example so the inline branches become multi-line blocks that each also print a short message. Does the output change?

Exercise 4.5. T

Translate the freezing/cold/warm/hot program into Persian.

Chapter 5

Loops

Computers are tireless. Where a person would balk at adding the numbers from 1 to a million, a computer does it without complaint — *if* we can express the work as a repetition. A **loop** runs a block of code over and over, and it is what turns a few lines of source into millions of actions. Salam gives you three looping tools: **while**, the C-style **for**, and the friendly **repeat**. We will meet all three.

5.1 The while loop

A while loop repeats its body *as long as* a condition stays true. It checks the condition, runs the body if true, then checks again — round and round until the condition becomes false.

```
1 func main:
2   mut i i32 = 1
3   while i <= 3:
4     print i
5     print " "
6     i += 1
7   end
8   println ""
9 end
```

```
1 2 3
```

Trace it: *i* starts at 1; *1 <= 3* is true, so we print 1 and bump *i* to 2; and so on until *i* becomes 4, at which point *4 <= 3* is false and the loop stops. The variable that drives the loop — here *i* — must be **mut**, because the loop

changes it.

Watch out

Every `while` loop needs a way to *end*. If the condition never becomes false — for example if you forget the `i += 1` above — the loop runs forever and your program hangs. Whenever you write a `while`, find the line that makes progress toward the exit and check that it is really there.

`while` shines when you do not know in advance how many times you will loop. A classic example is Euclid’s algorithm for the greatest common divisor, which loops until a remainder reaches zero:

```

1 func gcd(a i32, b i32) i32:
2     mut x i32 = a
3     mut y i32 = b
4     while y != 0:
5         mut t i32 = y
6         y = x % y
7         x = t
8     end
9     ret x
10 end
11
12 func main:
13     println gcd(48, 36)
14 end

```

12

5.2 The for loop

When you *do* know the pattern of counting — start here, continue while a condition holds, step like this each time — the `for` loop gathers all three parts onto one line. It has the shape `for init; condition; step:`.

```

1 func main:
2     for mut i = 0; i < 5; i = i + 1:
3         print i
4     end
5     println ""
6 end

```

01234

The three parts, separated by semicolons, are:

- **init** — `mut i = 0` runs once, before the loop starts. It creates and initializes the loop variable.
- **condition** — `i < 5` is checked before each pass; the loop continues while it is true.
- **step** — `i = i + 1` runs at the end of each pass, moving the loop forward.

Everything a `for` does could be written with a `while`; the `for` simply keeps the three pieces together where you can see them at a glance. It is the natural choice for walking over a range of numbers or, as we will see in Chapter 7, the indices of an array.

```
1 func main:
2     // print the 7 times table
3     for mut n = 1; n <= 10; n = n + 1:
4         println "7 x", n, "=", 7 * n
5     end
6 end
```

```
7 x 1 = 7
7 x 2 = 14
...
7 x 10 = 70
```

5.3 The repeat loop

For the very common case of “just do this *N* times” or “count from *A* to *B*,” Salam offers `repeat`, which reads almost like plain English. It comes in three forms.

5.3.1 Repeat a fixed number of times

`repeat N:` runs its body *N* times. There is no loop variable to manage — when you only need the *repetition*, this is the cleanest tool:

```
1 func main:
2     repeat 3:
3         println "Salam!"
4     end
5 end
```

```
Salam!  
Salam!  
Salam!
```

If the count is zero or negative, the body simply does not run.

5.3.2 Repeat over a range

`repeat A to B`: runs the body once for every value from A up to B *inclusive* — so `repeat 1 to 5` runs five times. The range endpoints are part of the loop, not an index you can read; when you need the running value, keep your own counter:

```
1 func main:  
2   mut sum i32 = 0  
3   mut k i32 = 0  
4   repeat 1 to 5:  
5     k += 1  
6     sum += k  
7   end  
8   println "1+2+3+4+5 =", sum  
9 end
```

```
1+2+3+4+5 = 15
```

Note

The upper bound is *inclusive*: `repeat 1 to 5` covers 1, 2, 3, 4, 5 — five iterations, not four. This matches how we count in everyday speech (“count from 1 to 5”) and differs from the half-open `mut i = 1; i < 5` pattern, which would stop at 4. Keep the distinction in mind when you choose between the two.

5.3.3 Repeat with a step

Add every 5 to stride by 5 instead of 1. `repeat 0 to 20 every 2` visits 0, 2, 4, ..., 20:

```
1 func main:  
2   mut count i32 = 0  
3   repeat 0 to 20 every 2:  
4     count += 1  
5   end
```



```
6 println "even numbers in 0..20:", count
7 end
```

```
even numbers in 0..20: 11
```

There are eleven of them (0, 2, ..., 20), and the loop counts each — another reminder that the upper bound is included.

5.4 Breaking out and skipping ahead

Two keywords give you finer control *inside* any loop:

- `break` stops the loop immediately and continues with whatever comes after it.
- `continue` abandons the current pass and jumps straight to the next one.

```
1 func main:
2   for mut n = 0; n < 6; n = n + 1:
3     if n == 3: continue end      // skip 3
4     if n == 5: break end        // stop before 5
5     print n
6   end
7   println ""
8 end
```

```
0124
```

The loop prints 0, 1, and 2 normally; at `n == 3` the `continue` skips the `print`, so 3 never appears; at `n == 5` the `break` ends the loop before printing. Hence 0124.

Tip

`break` and `continue` are handy, but a loop peppered with them can be hard to follow. Often a clearer condition in the `while` or `for` header expresses the same intent without jumping. Reserve `break` for cases like “search until found,” where stopping early is the whole point.

5.5 Nested loops

A loop body may itself contain a loop. Nested loops are how you work with grids, tables, and shapes. Here a loop over rows contains a loop over columns to draw a triangle of stars — using the “print then end the line” trick from Chapter 1:

```
1 func main:
2   mut row i32 = 0
3   repeat 1 to 5:
4     row += 1
5     repeat 1 to row:
6       print "*"
7     end
8     println ""
9   end
10 end
```

```
*
**
***
****
*****
```

The outer loop runs once per row; for each row, the inner loop prints as many stars as the row number, and then `println ""` moves to the next line. Nested loops multiply: an outer loop of 5 around an inner loop of up to 5 does up to 25 iterations in total.

Watch out

`break` and `continue` affect only the *innermost* loop that encloses them. A `break` inside the inner star loop would stop that row’s stars, not the whole triangle.

5.6 In Persian

The loop keywords have Persian forms too: `تا وقتی` for `while`, `برای` for `for`, `تکرار` for `repeat`, `تا` for `to`, and `هر` for `every`. Here is the star loop in Persian:

	تابع اصلی:	1
	تکرار 1 تا 5:	2
	بنویس	3
	پایان	4
	چاپ	5
	پایان	6

5.7 Which loop should I use?

- Reach for `repeat N` when you just need to do something a set number of times, and `repeat A to B` (optionally with `every`) when you are counting over a clear range.
- Use `for` when you need an index you can read and use inside the body — especially with arrays.
- Use `while` when the number of iterations is not known up front and the loop ends when some condition changes.

5.8 Chapter summary

- `while cond`: repeats while a boolean condition holds; make sure it can end.
- `for init; cond; step`: packages counting into one header.
- `repeat N`, `repeat A to B`, and `repeat A to B every S` cover fixed counts and ranges; the upper bound is *inclusive*.
- `break` ends the innermost loop; `continue` skips to its next pass.
- Loops nest, and that is how you handle grids and shapes.

5.9 Exercises

Exercise 5.1. U

se a `while` loop to print the numbers from 10 down to 1, then print "Lift-off!".

Exercise 5.2. U

se a `for` loop to compute and print the sum of all numbers from 1 to 100.

Exercise 5.3. U

sing `repeat ... every`, print every multiple of 5 from 5 to 50 on one line, separated by spaces.

Exercise 5.4. U

se a loop with `continue` to print the numbers from 1 to 20, skipping every multiple of 3.

Exercise 5.5. D

raw an upside-down triangle of stars (five stars on the first row, one on the last) using nested loops.

Exercise 5.6. R

ewrite the times-table example so it prints the table for a number stored in a variable, in Persian.

Chapter 6

Functions

We have used `main` since the first page, and a few helper functions along the way. Now we study functions properly, because they are the single most important tool for organising a program. A **function** is a named, reusable piece of code that takes some inputs, does some work, and (often) hands back a result. Functions let you write a piece of logic once, give it a name, and call it as many times as you like.

6.1 Defining and calling a function

You have already seen the shape. A function definition is the keyword `func`, a name, a parenthesised list of parameters, an optional return type, a colon, the body, and `end`:

```
1 func square(n i32) i32:
2     ret n * n
3 end
4
5 func main:
6     println square(5)
7     println square(9)
8 end
```

```
25
81
```

`square` takes one **parameter**, `n` of type `i32`, and its **return type** (written after the parameter list) is also `i32`. The body computes `n * n` and hands it back with `ret`. In `main` we **call** the function by writing its name and

an **argument** in parentheses: `square(5)`. The argument 5 becomes the parameter `n` for that call.

Note

Parameter and *argument* are two ends of the same connection. The parameter is the name in the definition (`n`); the argument is the value you pass at the call site (5). The argument is copied into the parameter when the call begins.

6.2 Returning a value

The `ret` keyword does two things at once: it ends the function immediately and sends a value back to the caller. The type of that value must match the function’s declared return type.

A function can have several `ret` statements — useful when different cases produce different answers. The first one reached wins:

```
1 func abs(n i32) i32:
2     if n < 0:
3         ret -n
4     end
5     ret n
6 end
7
8 func main:
9     println abs(-7), abs(7)
10 end
```

```
7 7
```

If `n` is negative, the first `ret` fires and the function is done; the final `ret n` is reached only for non-negative inputs. Using an early `ret` to dispatch a special case, then letting the main path continue un-indented, is a clean and common style.

6.3 Functions that return nothing

Some functions *do* something rather than *compute* something — they print, they update, they save a file. Such a function has no return type: you simply omit it after the parameter list. Its (implicit) result type is `void`, the “no value” type from Chapter 2.

```
1 func greet(name str):  
2     println "Hello,", name  
3     println "Welcome to Salam."  
4 end  
5  
6 func main:  
7     greet("Sara")  
8     greet("Ali")  
9 end
```

```
Hello, Sara  
Welcome to Salam.  
Hello, Ali  
Welcome to Salam.
```

Inside a void function you may still use a bare `ret` (with no value) to return early — for instance to bail out when an input is invalid — but you never return a value from one.

6.4 Multiple parameters

A function may take any number of parameters, separated by commas. Each needs its own type:

```
1 func add(a i32, b i32) i32:  
2     ret a + b  
3 end  
4  
5 func max3(a i32, b i32, c i32) i32:  
6     mut m i32 = a  
7     if b > m: m = b end  
8     if c > m: m = c end  
9     ret m  
10 end  
11  
12 func main:  
13     println add(3, 4)  
14     println max3(7, 2, 9)  
15 end
```

```
7  
9
```

Arguments are matched to parameters *by position*: in `add(3, 4)`, 3 becomes `a` and 4 becomes `b`.

Watch out

The current version of Salam does not support *default* parameter values, so every parameter must be supplied at every call. Calling `add(3)` is an error. (To offer a “default,” write a second overload — see below.)

6.5 Arguments are passed by value

When you pass an argument, the function receives a *copy* of it. Changing a parameter inside the function therefore does *not* change the caller’s variable:

```
1 func tryToChange(x i32):
2     mut local i32 = x
3     local = 999           // only the copy changes
4 end
5
6 func main:
7     n i32 = 5
8     tryToChange(n)
9     println n             // still 5
10 end
```

5

This “pass by value” rule makes functions easy to reason about: a function cannot secretly reach back and alter the variables you passed in. When you want a function to produce a new value, the clean approach is to *return* it and let the caller decide what to do:

```
1 func incremented(x i32) i32:
2     ret x + 1
3 end
4
5 func main:
6     mut n i32 = 5
7     n = incremented(n)    // the caller chooses to update n
8     println n
9 end
```

6

Note

Salam deliberately has no “pass by reference” for ordinary variables — you cannot take the address of a local with `&` and hand it to a function. This keeps the flow of data visible: values go *in* as arguments and come *out* as return values. (In Chapter 9 you will see that a struct’s own methods can update the struct they belong to; that is the sanctioned way to modify data in place.)

6.6 Recursion

A function is allowed to call *itself*. This is called **recursion**, and it is the natural way to express anything defined in terms of a smaller version of itself. The classic example is the factorial, where $n! = n \times (n - 1)!$:

```
1 func fact(n i32) i32:
2     if n <= 1:
3         ret 1
4     end
5     ret n * fact(n - 1)
6 end
7
8 func main:
9     println fact(5)
10 end
```

120

Every recursive function needs two things to be correct: a **base case** that stops the recursion (here, `n <= 1` returns 1 without calling again), and a **recursive step** that moves *toward* the base case (here, calling `fact(n - 1)` with a smaller argument). Trace `fact(5)`: it becomes `5 * fact(4)`, which becomes `5 * 4 * fact(3)`, and so on down to `fact(1)`, which is 1 — then the multiplications unwind: $5 \times 4 \times 3 \times 2 \times 1 = 120$.

Watch out

Forget the base case — or fail to shrink the argument — and a recursive function calls itself forever, until the program runs out of stack space and crashes. Always ask: “what stops this?” and “does each call get closer to stopping?”

In Depth: recursion versus iteration (optional, for advanced readers)

Anything you can compute with recursion you can also compute with a loop, and vice versa. Recursion often expresses a problem more clearly (tree walks, divide and conquer), while a loop avoids the small cost of repeated function calls. For factorials a loop is perfectly natural:

```

1 func fact(n i32) i32:
2     mut result i32 = 1
3     for mut i = 2; i <= n; i = i + 1:
4         result = result * i
5     end
6     ret result
7 end

```

Choose whichever makes the intent clearest; both are idiomatic Salam.

6.7 Definition order does not matter

You may call a function that is defined *later* in the file. Salam reads all the top-level definitions before it runs anything, so `main` can freely call helpers written below it:

```

1 func main:
2     println later()
3 end
4
5 func later() str:
6     ret "defined after main"
7 end

```

```
defined after main
```

This means you never need “forward declarations,” and you are free to put `main` at the top of the file where a reader expects to find it.

6.8 Function overloading

Two functions may share a name as long as their parameters differ — in number or in type. This is called **overloading**, and Salam picks the right one based on the arguments at each call:

```

1 func add(a i32, b i32) i32:

```

```

2      ret a + b
3  end
4
5  func add(a f64, b f64) f64:
6      ret a + b
7  end
8
9  func main:
10     println add(2, 3)           // calls the i32 version
11     println add(1.5, 2.5)      // calls the f64 version
12 end

```

```

5
4

```

The two `adds` are genuinely different functions that happen to share a name. Overloading lets a family of related operations present one consistent name instead of `addInt`, `addFloat`, and so on. (The second result prints as `4` because it is the floating value `4.0`.)

6.9 Use every parameter

Salam is tidy about parameters: if you declare one and never use it, the compiler reports an error rather than letting the dead weight slip by. If you genuinely need a parameter you do not use — perhaps to match an expected shape — prefix its name with an underscore to say “unused on purpose”:

```

1  func ignore_second(a i32, _b i32) i32:
2      ret a
3  end

```

Tip

This rule nudges you toward honest signatures: a function’s parameter list should list exactly what it needs. When the compiler flags an unused parameter, it is often pointing at a leftover from an earlier version of the code that you can simply delete.

6.10 Designing good functions

Functions are how you tame complexity. A few guidelines that pay off:

- **One job each.** A function should do a single, nameable thing. If you struggle to name it, it is probably doing too much.
- **Short is good.** If a function grows past a screenful, look for a piece you can lift out into its own well-named helper.
- **Name for the result.** `is_leap(year)` and `average(scores)` tell the reader what comes back. A good name turns a call site into a readable sentence.
- **Prefer returning over reaching out.** Compute a value and return it, rather than relying on shared mutable state.

6.11 Chapter summary

- A function is `func name(params) RetType: ... end; ret` returns a value and ends the function.
- Omit the return type for a function that returns nothing (`void`); a bare `ret` returns early from one.
- Parameters are matched by position and passed *by value* (the function gets a copy). There are no default parameters and no reference parameters.
- A function may call itself (recursion); give it a base case and make each call smaller.
- Definition order is irrelevant — you can call functions defined later.
- Functions may be *overloaded* by differing parameters.
- Every declared parameter must be used, or prefixed with `_`.

6.12 Exercises

Exercise 6.1. W

rite a function `cube(n i32) i32` that returns n^3 using multiplication, and print the cubes of 1 through 5 with a loop.

Exercise 6.2. W

rite a void function `banner(text str)` that prints the text surrounded by a line of dashes above and below it.

Exercise 6.3. W

rite `min2(a i32, b i32) i32` and use it to build `min3(a, b, c)` by calling `min2` twice.

Exercise 6.4. W

rite a recursive function `sum_to(n i32) i32` that returns $1 + 2 + \dots + n$. Identify its base case and recursive step in comments.

Exercise 6.5. O

verload a function `describe` so that `describe(42)` prints “an integer” and `describe(3.14)` prints “a float”. Test both.

Exercise 6.6. R

ewrite the recursive `sum_to` from a previous exercise as a loop, and confirm both give the same answers.

Chapter 7

Arrays and Vectors

A single variable holds one value. But programs constantly deal with *collections* — a row of scores, a list of names, a grid of cells. Salam gives you two tools for holding many values under one name: the **array**, whose size is fixed when you write it, and the **vector**, which grows and shrinks as your program runs. This chapter covers both.

7.1 Fixed-size arrays

An **array** is a numbered row of values, all of the same type. You declare one by writing the element type followed by the length in square brackets, and you can give the initial contents in a bracketed list:

```
1 func main:
2   scores i32[5] = [90, 85, 70, 95, 60]
3   println scores[0]      // the first element
4   println scores[4]      // the last element
5 end
```

```
90
60
```

The type `i32[5]` reads “an array of five i32s.” The values are stored in *slots* numbered from 0, so a length-5 array has slots 0, 1, 2, 3, 4. You reach a slot with square brackets: `scores[0]` is the first, `scores[4]` the last.

Watch out

Indices start at *zero*, not one. The first element is `scores[0]` and the *last* of a length-*n* array is `scores[n-1]` — here `scores[4]`, not `scores[5]`. Off-by-one slips around this rule are among the most common bugs in all of programming; the zero-based habit becomes second nature with practice.

7.1.1 Changing elements

To modify an element, assign to its slot. As with any modification, the array must be `mut`:

```
1 func main:
2   mut a i32[3] = [1, 2, 3]
3   a[1] = 20
4   println a[0], a[1], a[2]
5 end
```

```
1 20 3
```

7.1.2 Walking an array with a loop

The `for` loop and arrays are made for each other: the loop variable steps through the valid indices, and the body does something with each element. Here we sum an array and find its largest value in a single pass:

```
1 func main:
2   a i32[7] = [5, 9, 1, 7, 3, 8, 2]
3   mut sum i32 = 0
4   mut hi i32 = a[0]
5   for mut i = 0; i < 7; i = i + 1:
6     sum += a[i]
7     if a[i] > hi: hi = a[i] end
8   end
9   println "sum =", sum, "max =", hi
10 end
```

```
sum = 35 max = 9
```

Notice the loop condition `i < 7`: it stops *before* index 7, visiting exactly slots 0 through 6. This half-open pattern — start at 0, continue while `i <`

`length` — is the canonical way to walk an array, and it lines up perfectly with zero-based indexing.

7.2 Multi-dimensional arrays

An array's elements can themselves be arrays, giving you a grid. `i32[2][3]` is “2 rows of 3 columns.” You index with two pairs of brackets, row first:

```
1 func main:
2     mut grid i32[2][3] = [[1, 2, 3], [4, 5, 6]]
3     grid[1][2] = 60
4     println grid[0][0], grid[1][2]
5 end
```

```
1 60
```

Nested loops walk a grid naturally: an outer loop over rows, an inner loop over columns. This is the foundation for anything tabular — game boards, matrices, spreadsheets.

7.3 The limit of fixed arrays

A fixed array's length is part of its type and is decided when you write the program. That is perfect when you know the count up front (a week has 7 days, a board has 64 squares). But often you do *not* know how many items there will be — you are reading lines from a file, or collecting whatever the user types. For that you need a collection that can grow. Enter the vector.

7.4 Vectors: arrays that grow

A **vector**, written `Vector<T>`, is a sequence of `T` values whose length can change at run time. You start with an empty one and add to it as you go. Create an empty vector with `Vector {}`:

```
1 func main:
2     mut v Vector<i32> = Vector {}
3     v.push(10)
4     v.push(20)
5     v.push(30)
6     println "length:", v.len()
7 end
```

```
length: 3
```

The angle brackets in `Vector<i32>` say what the vector holds — here, `i32`s. (This is your first glimpse of *generics*, the subject of Chapter 12: `Vector` is one type that works for any element type.) The `.push(x)` method appends a value to the end, and `.len()` reports how many values are currently stored.

7.4.1 Reading and writing elements

A vector's `.get(i)` method returns a *pointer* to the element at index `i`; to read the value itself, follow the pointer with `[0]`. To overwrite an element, use `.set(i, value)`:

```
1 func main:
2     mut v Vector<i32> = Vector {}
3     v.push(10)
4     v.push(20)
5     v.push(30)
6     println "first:", v.get(0)[0]
7     v.set(0, 100)
8     println "first now:", v.get(0)[0]
9 end
```

```
first: 10
first now: 100
```

Note

Why `v.get(i)[0]` rather than just `v[i]`? `.get` returns a *pointer* into the vector's storage, and `[0]` reads through that pointer to the value. The pointer form is what makes vectors efficient — no copying — and you will understand it fully after the chapter on memory (Chapter 18). For now, treat `v.get(i)[0]` as the idiom for “the value at index `i`.”

7.4.2 Walking a vector

Because `.len()` can change, you typically walk a vector with a `while` loop bounded by its current length:

```
1 func main:
2     mut v Vector<i32> = Vector {}
3     v.push(5)
4     v.push(8)
```

```
5   v.push(2)
6   mut total i32 = 0
7   mut i i32 = 0
8   while i < v.len():
9       total += v.get(i)[0]
10      i += 1
11  end
12  println "total:", total
13  end
```

```
total: 15
```

7.4.3 Removing from the end

`.pop()` removes the last element and returns it — turning a vector into a *stack*, where the last thing in is the first thing out:

```
1  func main:
2      mut v Vector<i32> = Vector {}
3      v.push(1)
4      v.push(2)
5      v.push(3)
6      println "pop:", v.pop()      // 3
7      println "pop:", v.pop()      // 2
8      println "left:", v.len()     // 1
9  end
```

```
pop: 3
pop: 2
left: 1
```

7.4.4 The common vector operations

The methods you will reach for most often:

Method	What it does
<code>v.push(x)</code>	append <code>x</code> to the end
<code>v.pop()</code>	remove and return the last element
<code>v.len()</code>	the number of elements
<code>v.get(i)[0]</code>	read the element at index <code>i</code>
<code>v.set(i, x)</code>	overwrite the element at index <code>i</code>
<code>v.is_empty()</code>	<code>true</code> if the vector has no elements
<code>v.cap()</code>	current capacity (see the In Depth box)
<code>v.free()</code>	release the vector's memory

Watch out

When you are finished with a vector, call `v.free()` to release the memory it allocated. Unlike a fixed array (whose storage is reclaimed automatically when it goes out of scope), a vector manages a growable buffer that you are responsible for freeing. We discuss this fully in Chapter 18; for now, make `v.free()` the last thing you do with a vector.

In Depth: capacity versus length, and why push is cheap (optional, for advanced readers)

A vector keeps two numbers: its *length* (how many elements you have stored) and its *capacity* (how many it could store before needing a bigger buffer). When you push past the capacity, the vector quietly allocates a larger buffer — typically doubling — and copies the elements across. Because the capacity grows geometrically, the *average* cost of a `push` is constant, even though an occasional one triggers a copy. That is why building a vector element by element is efficient, and why `v.cap()` is usually larger than `v.len()`: pushing two items into a fresh vector may already reserve room for four.

7.5 Arrays or vectors?

- Use a **fixed array** when you know the number of elements when you write the code, and it will not change — coordinates, a fixed lookup table, the cells of a board.
- Use a **vector** when the count is decided at run time or changes as the program works — reading input, accumulating results, anything “however many there turn out to be.”

7.6 A worked example: collecting even numbers

This program walks the numbers 1 to 10, collects the even ones into a vector, and reports what it found — a pattern (filter into a growing collection) you will use constantly:

```

1 func main:
2     mut evens Vector<i32> = Vector {}
3     for mut n = 1; n <= 10; n = n + 1:
4         if n % 2 == 0:
5             evens.push(n)
6         end
7     end
8     print "found "
9     print evens.len()
10    println " even numbers"
11    mut i i32 = 0
12    while i < evens.len():
13        print evens.get(i)[0]
14        print " "
15        i += 1
16    end
17    println ""
18    evens.free()
19 end

```

```

found 5 even numbers
2 4 6 8 10

```

7.7 Chapter summary

- A fixed array `T[N]` holds `N` values of type `T`; elements are indexed from `0` to `N-1`.
- Walk an array with `for mut i = 0; i < N; i = i + 1.`
- Arrays can nest (`T[R][C]`) to form grids, indexed row-first.
- A `Vector<T>` grows at run time; create it with `Vector {}`.
- Key vector methods: `push`, `pop`, `len`, `get(i)[0]` to read, `set(i, x)` to write, `is_empty`, `free`.
- Call `v.free()` when done with a vector.

- Choose arrays for fixed counts, vectors for counts that vary.

7.8 Exercises

Exercise 7.1. C

reate an array of the five vowels (char values) and print each on its own line with a loop.

Exercise 7.2. G

iven a `i32[6] = [4, 8, 15, 16, 23, 42]`, compute and print the average as an `f64`.

Exercise 7.3. B

uild a vector of the squares $1, 4, 9, \dots, 100$ using a loop, print its length, then print every element.

Exercise 7.4. W

rite a program that reads a fixed array of integers and prints the minimum and maximum, using a single loop.

Exercise 7.5. F

ill a `i32[3][3]` grid so that `grid[r][c] = r * 3 + c`, then print it as three rows of three numbers.

Exercise 7.6. U

se a vector as a stack: push the numbers 1 to 5, then pop and print them all, observing that they come out in reverse order. Remember to `free` the vector.

Chapter 8

Strings and Characters

Text is everywhere in programming: names, messages, file contents, user input. In Salam a piece of text is a `str`, and a single character is a `char`. We have used both since Chapter 1; this chapter gathers everything about them in one place — how to write them, join them, search them, and transform them with the standard library.

8.1 String literals

A **string literal** is text between double quotes:

```
1 func main:
2     greeting str = "Hello, Salam!"
3     empty str = ""
4     println greeting
5     println "length:", len(greeting)
6 end
```

```
Hello, Salam!
length: 13
```

The built-in `len` function reports the number of bytes in a string — 13 for "Hello, Salam!". A string with nothing between the quotes, "", is the *empty string*; it is perfectly valid and has length 0.

8.2 Strings are immutable

A `str` cannot be changed in place. Every operation that seems to “modify” a string actually builds a *new* one and leaves the original untouched. This is why you can share strings freely without fear that someone will alter one behind your back, and it is why joining strings always produces a fresh result rather than growing an existing one.

8.3 Joining strings with +

The `+` operator concatenates — glues together — two strings, producing a new string:

```
1 func main:
2     first str = "Salam"
3     last str = "World"
4     full auto = first + ", " + last + "!"
5     println full
6 end
```

Salam, World!

Each `+` builds a larger string from its two operands. Because the result is itself a string, you can chain as many as you like, as the example does.

Watch out

`+` joins a string to a string. It does *not* automatically turn a number into text: `"count: " + 5` is a type error, because `5` is an `i32`, not a `str`. To attach a number, convert it first — the standard library’s `str.FromInt` does exactly that, as we will see shortly. (When you simply want to *print* mixed values, `println` already accepts a comma-separated mix of any types — no conversion needed.)

8.4 Escape sequences

Some characters are awkward to type directly inside a string — a newline, a tab, or a double quote (which would otherwise end the string). For these you use an **escape sequence**: a backslash followed by a code.

Escape	Means
<code>\n</code>	newline (move to the next line)
<code>\t</code>	tab
<code>\\</code>	a literal backslash
<code>\"</code>	a literal double quote

```
1 func main:
2     println "line one\nline two"
3     println "name:\tSalam"
4     println "she said \"hi\""
5     println "a backslash: \\"
6 end
```

```
line one
line two
name:  Salam
she said "hi"
a backslash: \
```

The `\n` split the first string across two lines; the `\t` inserted a tab; `\"` put real quotes *inside* the text; and `\\` produced a single backslash.

8.5 Triple-quoted strings

When a string itself contains many double quotes, escaping each one is tedious. A **triple-quoted** string, wrapped in `"""`, lets you write double quotes directly without escaping:

```
1 func main:
2     line str = """She said "hi" to me"""
3     println line
4 end
```

```
She said "hi" to me
```

Triple-quoted strings are handy for text with embedded quotes — snippets of other code, JSON, or dialogue.

8.6 Characters

A `char` is a single character, written between *single* quotes. As we saw in Chapter 2, a `char` is really a small number (its byte code), and you can

convert between the two with as:

```
1 func main:
2     letter char = 'S'
3     println letter           // S
4     println letter as i32    // 83, the code for 'S'
5     println 33 as char       // '!', the character with code 33
6 end
```

```
S
83
!
```

Note

'S' (single quotes) is a `char` — one character. "S" (double quotes) is a `str` — text that happens to be one character long. They are different types and are not interchangeable; keep an eye on which quotes you are typing.

8.7 Strings are not indexed with brackets

You might expect `s[0]` to give the first character of a string, as it would for an array. In Salam it does *not* — a `str` is not an array, and indexing one is an error. To work with parts of a string, you use the functions of the `str` standard-library module, which we turn to now.

8.8 The `str` module

The built-in `+` and `len` cover the basics, but most real string work goes through the `str` module. Import it with `import "str"` and call its functions with the `str.` prefix. Here are the ones you will use most.

8.8.1 Case and trimming

```
1 import "str"
2
3 func main:
4     println str.Upper("salam")    // SALAM
5     println str.Lower("SALAM")    // salam
```

```
6   println str.Trim("  hi  ")    // "hi" (surrounding spaces
   removed)
7   println str.Reverse("Salam")  // malaS
8   println str.Repeat("ab", 3)   // ababab
9   end
```

```
SALAM
salam
hi
malaS
ababab
```

8.8.2 Searching and testing

```
1   import "str"
2
3   func main:
4       println str.Find("hello world", "world")    // 6 (the index)
5       println str.Contains("hello", "ell")        // true
6       println str.StartsWith("hello.salam", "hello") // true
7       println str.EndsWith("hello.salam", ".salam") // true
8   end
```

```
6
true
true
true
```

`str.Find` returns the index at which the substring begins (here 6, since "world" starts at position 6 in "hello world"), or a negative value if it is not present.

8.8.3 Substrings and replacement

```
1   import "str"
2
3   func main:
4       println str.Substr("programming", 0, 7)    // "program"
5       println str.Replace("a-b-c", "-", "+")     // "a+b+c"
6   end
```

```
program
a+b+c
```

`str.Substr(s, start, n)` extracts `n` characters beginning at index `start` — so `Substr("programming", 0, 7)` takes seven characters from the start. This is also how you read a single character: `str.Substr(s, i, 1)` gives the one-character string at index `i`.

8.8.4 Converting between strings and numbers

Turning numbers into text and back is constant work, and the `str` module makes it direct:

```
1 import "str"
2
3 func main:
4     n i32 = 2026
5     text auto = str.FromInt(n as i64)      // "2026"
6     println "Year: " + text                // now '+' works: both are
7                                           strings
8
9     back auto = str.ToInt("42")            // 42, an i32
10    println back + 8                       // 50
end
```

```
Year: 2026
50
```

`str.FromInt` converts an integer to its text form (it takes an `i64`, so cast smaller integers with `as i64`); `str.ToInt` parses a string back into an `i32`. With `FromInt` you can finally use `+` to attach a number to a message, because both sides are now strings.

8.8.5 Splitting and joining

Two of the most useful string operations break a string apart and put one back together. `str.Split` cuts a string at every occurrence of a separator, returning a `Vector<str>`; `str.Join` does the reverse:

```
1 import "str"
2
3 func main:
4     mut parts Vector<str> = str.Split("ada,grace,linus", ",")
```

```

5   println "count:", parts.len()
6   println str.Join(parts, " & ")
7   parts.free()
8   end

```

```

count: 3
ada & grace & linus

```

Splitting "ada,grace,linus" on "," yields a vector of three strings; joining them with " & " stitches them back into one. Because `Split` returns a vector, remember to `free` it when you are done (Chapter 7).

8.9 A worked example: counting words

Putting several of these together, here is a small program that counts the words in a sentence and prints each on its own line:

```

1   import "str"
2
3   func main:
4       sentence str = "the quick brown fox"
5       mut words Vector<str> = str.Split(sentence, " ")
6       println "word count:", words.len()
7       mut i i32 = 0
8       while i < words.len():
9           println str.Upper(words.get(i)[0])
10          i += 1
11       end
12       words.free()
13   end

```

```

word count: 4
THE
QUICK
BROWN
FOX

```

In Depth: bytes, characters, and Unicode (optional, for advanced readers)

A Salam `str` is stored as a sequence of bytes in UTF-8 encoding, and `len` reports the number of *bytes*. For plain English text, one byte is one character, so the distinction never bites. But a character outside

the basic Latin range — an accented letter, or a letter of another script — may occupy several bytes, so its byte length and its visible length differ. The `str` module's higher-level functions (`Split`, `Find`, `Replace`, and friends) are the safe way to manipulate text; reach for raw byte arithmetic only when you know your data is plain ASCII.

8.10 Chapter summary

- A `str` is immutable text in double quotes; `len(s)` gives its byte length; `+` concatenates.
- Escape sequences (`\n`, `\t`, `\"`, `\\`) embed special characters; `"""..."""` avoids escaping inner quotes.
- A `char` is a single character in single quotes and converts to/from its numeric code with `as`.
- Strings are *not* indexed with `[]`; use the `str` module.
- `str` provides `Upper`, `Lower`, `Trim`, `Reverse`, `Repeat`, `Find`, `Contains`, `StartsWith`, `EndsWith`, `Substr`, `Replace`, `FromInt`, `ToInt`, `Split`, and `Join`.

8.11 Exercises

Exercise 8.1. A

ask for a `str` variable holding your full name and print its length, its uppercase form, and its reverse.

Exercise 8.2. B

Build the string "Item No.7 costs \$42" by converting the numbers 7 and 42 with `str.FromInt` and joining with `+`.

Exercise 8.3. G

Given "one,two,three,four", split on the comma, print how many parts there are, and print them re-joined with " - ".

Exercise 8.4. W

Write a function `is_palindrome(s str) bool` that returns whether a string reads the same forwards and backwards (hint: compare `s` with `str.Reverse(s)`).

Exercise 8.5. U

se `str.Substr` to print each character of "Salam" on its own line.

Exercise 8.6. W

rite a program that, given a filename like "report.txt", prints just the extension ("txt") by finding the last ".".

Chapter 9

Structs and Methods

So far our data has been loose: an `i32` here, a `str` there. But the things we model are rarely single values. A bank account has an owner *and* a balance; a point has an *x* *and* a *y*. A **struct** lets you bundle several values into one named type, and **methods** let you attach behaviour to that type. Together they are how you build your own data types in Salam.

9.1 Defining a struct

A struct is declared with the `struct` keyword, a name, and a list of **fields** — each a name, a type, and a default value — closed with `end`:

```
1 struct Point:
2     x i32 = 0
3     y i32 = 0
4 end
```

This defines a new type, `Point`, with two fields. Each field has a default value (here `0`), which is used when you do not supply one. `Point` is now a type you can use anywhere a type is expected, just like `i32` or `str`.

9.2 Creating instances

To make a value of a struct type — an **instance** — write the type name followed by braces listing the fields you want to set:

```
1 struct Point:
2     x i32 = 0
3     y i32 = 0
```

```

4 end
5
6 func main:
7     origin Point = Point {}           // both fields default to
        0
8     p Point = Point { x = 3, y = 4 }   // set both fields
9     println origin.x, origin.y
10    println p.x, p.y
11 end

```

```

0 0
3 4

```

Inside the braces you assign fields by name with `field = value`. Any field you leave out keeps its default — `Point {}` relies on the defaults for both, giving `(0, 0)`. You reach a field of an instance with a dot: `p.x` is `p`'s `x` field.

Note

Because every field has a default, `Point {}` is always a valid, fully formed value — there is no such thing as a half-built struct with missing fields. This is why field defaults are required in the struct definition: they guarantee that an instance is never left with an uninitialised field.

9.3 Reading and changing fields

Field access with the dot works both ways: you can read a field, and — if the instance is `mut` — you can assign to one:

```

1 struct Point:
2     x i32 = 0
3     y i32 = 0
4 end
5
6 func main:
7     mut p Point = Point { x = 1, y = 1 }
8     p.x = 10
9     p.y = p.y + 5
10    println p.x, p.y
11 end

```

```

10 6

```

9.4 Structs are copied by value

A struct behaves like the simple values of Chapter 2: assigning it, passing it to a function, or returning it makes a *copy*. Changing the copy does not affect the original:

```
1 struct Point:
2     x i32 = 0
3     y i32 = 0
4 end
5
6 func main:
7     mut p Point = Point { x = 1, y = 2 }
8     mut q Point = p          // q is a copy of p
9     q.x = 99
10    println p.x, q.x        // p is untouched
11 end
```

```
1 99
```

This “value semantics” makes structs predictable: a `Point` you hand to a function cannot be quietly modified by that function. It is the same guarantee you saw for ordinary arguments in Chapter 6, now extended to your own types.

9.5 Methods: behaviour on a struct

A struct can carry not just data but **methods** — functions that belong to the type and act on a particular instance. You write a method inside the struct block, exactly like a function, and refer to the current instance as `this`:

```
1 struct Counter:
2     value i32 = 0
3     func increment():
4         this.value = this.value + 1
5     end
6     func get() i32:
7         ret this.value
8     end
9 end
10
11 func main:
```

```

12  mut c Counter = Counter {}
13  c.increment()
14  c.increment()
15  c.increment()
16  println c.get()
17  end

```

3

You *call* a method through an instance with the dot: `c.increment()`. Inside the method, `this` refers to the instance the method was called on — so `this.value` is *that* counter's value. Each `Counter` keeps its own value, and a method always works on the one it was called through.

Note

Unlike an ordinary argument, a method *can* change the instance it is called on: `increment` really does update `c`'s value. This is the sanctioned way to modify data in place that we hinted at in Chapter 6 — a struct's own methods are trusted to change it, while outside functions get copies.

9.6 Methods that compute and methods that act

Just like free functions, a method may return a value or return nothing. A method that returns a value is written with a return type after its parameter list:

```

1  struct Rectangle:
2    width i32 = 0
3    height i32 = 0
4    func area() i32:
5      ret this.width * this.height
6    end
7    func is_square() bool:
8      ret this.width == this.height
9    end
10 end
11
12 func main:
13   r Rectangle = Rectangle { width = 4, height = 4 }
14   println "area:", r.area()
15   println "square?", r.is_square()
16 end

```

```
area: 16
square? true
```

Here `area` and `is_square` only *read* the rectangle (through `this`) and compute an answer; they do not change anything. A method may also take parameters of its own, in addition to the implicit `this`:

```
1 struct Rectangle:
2     width i32 = 0
3     height i32 = 0
4     func scale(factor i32):
5         this.width = this.width * factor
6         this.height = this.height * factor
7     end
8 end
9
10 func main:
11     mut r Rectangle = Rectangle { width = 2, height = 3 }
12     r.scale(5)
13     println r.width, r.height
14 end
```

```
10 15
```

9.7 A fuller example: a bank account

Let us combine fields and methods into something realistic — a bank account that guards its own balance. The `withdraw` method returns a `bool` reporting whether the withdrawal succeeded:

```
1 import "str"
2
3 struct Account:
4     owner str = ""
5     balance i32 = 0
6     func deposit(amount i32):
7         this.balance = this.balance + amount
8     end
9     func withdraw(amount i32) bool:
10         if amount > this.balance:
11             ret false
12         end
13         this.balance = this.balance - amount
```

```

14         ret true
15     end
16     func describe() str:
17         ret this.owner + " has " + str.FromInt(this.balance as i64)
18     end
19 end
20
21 func main:
22     mut acc Account = Account { owner = "Sara", balance = 100 }
23     acc.deposit(50)
24     println acc.describe()
25     println "withdraw 30?", acc.withdraw(30)
26     println "withdraw 1000?", acc.withdraw(1000)
27     println acc.describe()
28 end

```

```

Sara has 150
withdraw 30? true
withdraw 1000? false
Sara has 120

```

This is the heart of good design: the `Account` *owns* its balance, and the only way to change it is through `deposit` and `withdraw`, which enforce the rules (you cannot withdraw more than you have). The data and the operations that protect it live together in one place.

9.8 Structs within structs

A field may itself be a struct, letting you build larger structures from smaller ones. You reach a nested field by chaining dots:

```

1 struct Point:
2     x i32 = 0
3     y i32 = 0
4 end
5
6 struct Line:
7     start Point = Point {}
8     finish Point = Point {}
9 end
10
11 func main:
12     ln Line = Line {
13         start = Point { x = 0, y = 0 },

```

```
14     finish = Point { x = 5, y = 5 }  
15     }  
16     println ln.start.x, ln.finish.y  
17 end
```

```
0 5
```

`ln.finish.y` reads the `y` field of the `finish` field of `ln`. Composing structs this way is how you model anything with structure — a line from two points, a person with an address, a game piece with a position and a colour.

In Depth: structs become C structs (optional, for advanced readers)

Each Salam struct compiles to a plain C struct with the same fields in the same order, and a method compiles to an ordinary C function that takes a pointer to the struct as a hidden first argument — which is exactly what `this` is. Because the layout is a direct, flat C struct, accessing a field is a single memory read with no indirection, and passing a struct by value is a simple memory copy. This is why structs are both convenient *and* fast, and why they map so cleanly onto C libraries through the foreign function interface (Chapter 19).

9.9 Chapter summary

- A struct bundles named fields, each with a default value, into a new type.
- Create an instance with `Type { field = value, ... }`; omitted fields use their defaults.
- Read and write fields with the dot: `p.x`.
- Structs are copied by value on assignment, argument passing, and return.
- Methods live inside the struct block, refer to the instance as `this`, and are called with the dot: `c.increment()`.
- A method may read or modify `this`, take its own parameters, and return a value or nothing.
- Struct fields may themselves be structs; chain dots to reach nested fields.

9.10 Exercises

Exercise 9.1. D

Define a struct `Circle` with a `radius` `f64` field and a method `area()` `f64` that returns πr^2 (use 3.14159 for π). Print the area of a circle of radius 2.

Exercise 9.2. A

Add a `describe()` method to `Point` that returns a string like "(3, 4)", and print it for a couple of points.

Exercise 9.3. B

Build a struct `Stack` around a fixed `i32[16]` array and a `count`, with `push` and `pop` methods. Push three values and pop them back.

Exercise 9.4. E

Extend the `Account` example with a `transfer(other, amount)` idea: withdraw from one account and deposit into another, only if the withdrawal succeeds.

Exercise 9.5. D

Define a struct `Rectangle` with a `perimeter()` method as well as `area()`, and print both for a 3×5 rectangle.

Exercise 9.6. M

Model a struct `Person` containing a `name` and an `Address` struct (street, city). Create one and print the city via chained field access.

Chapter 10

Enums and Pattern Matching

Many values in a program are not really numbers or text — they are a choice from a small, fixed set of possibilities. A day is one of seven; a traffic light is red, amber, or green; an order is pending, shipped, or delivered. An **enum** (short for *enumeration*) lets you name such a set and use those names directly, instead of remembering that “3 means Thursday.”

10.1 Defining an enum

An enum is declared with the **enum** keyword, a name, and a list of **members** — the allowed values — closed with **end**. The members may be written on one line, separated by commas, or one per line:

```
1 enum Day: Mon, Tue, Wed, Thu, Fri, Sat, Sun end
```

This defines a new type, **Day**, whose only values are the seven named days. A variable of type **Day** can hold exactly one of them — nothing else. That is the whole point: the type makes invalid values *unrepresentable*.

10.2 Using enum values

You refer to a member by qualifying it with the enum’s name and a dot: **Day.Mon**, **Day.Sat**. This makes it clear which enum the value belongs to and avoids name clashes between enums that happen to share a member name.

```
1 enum Day: Mon, Tue, Wed, Thu, Fri, Sat, Sun end
```

```
2
3 func main:
4     today Day = Day.Sat
5     if today == Day.Sat || today == Day.Sun:
6         println "It's the weekend!"
7     else:
8         println "A working day."
9     end
10 end
```

```
It's the weekend!
```

You compare enum values with `==` and `!=`, exactly as you would numbers. The variable `today` can only ever be one of the seven days, so the compiler — and the next person to read your code — knows precisely what values are in play.

10.3 Enums are numbers underneath

Each enum member has an integer value. By default they count up from zero in order: in `Day`, `Mon` is 0, `Tue` is 1, and so on to `Sun` at 6. You can see a member's number by converting it to an integer with `as`:

```
1 enum Day: Mon, Tue, Wed, Thu, Fri, Sat, Sun end
2
3 func main:
4     println Day.Mon as i32      // 0
5     println Day.Sat as i32      // 5
6 end
```

```
0
5
```

10.3.1 Choosing the numbers

When the underlying numbers matter — because they correspond to codes, flags, or external values — you can assign them explicitly. Any member you do not give a number to continues counting up from the previous one:

```
1 enum Color:
2     Red          // 0
3     Green = 5    // 5
```

```

4   Blue           // 6 (continues from 5)
5   end
6
7   func main:
8       println Color.Red as i32, Color.Green as i32, Color.Blue as i32
9   end

```

```
0 5 6
```

Red keeps the default 0, Green is pinned to 5, and Blue follows on at 6. This is handy when an enum mirrors values defined elsewhere, such as HTTP status codes or hardware register values.

Watch out

The conversion goes one way freely: an enum value turns into its integer with `as`. Going the other way — treating an arbitrary integer as an enum — is something to do only with care, because a number might not correspond to any member. Prefer to keep values in their enum form and convert to integers only at the edges of your program, where you must.

10.4 Pattern matching with if/else if

Some languages provide a dedicated `match` or `switch` statement for choosing an action based on which enum value you have. Salam keeps things simple: you “match” on an enum with an ordinary `if/else if` chain, which you already know well from Chapter 4.

```

1   enum Light: Red, Amber, Green end
2
3   func action(light Light) str:
4       if light == Light.Red:
5           ret "Stop"
6       else if light == Light.Amber:
7           ret "Slow down"
8       else:
9           ret "Go"
10      end
11  end
12
13  func main:
14      println action(Light.Red)
15      println action(Light.Amber)

```



```
20 func main:
21     mut s Status = Status.Pending
22     println label(s)
23     s = next(s)
24     println label(s)
25     s = next(s)
26     println label(s)
27 end
```

```
pending
shipped
delivered
```

The enum guarantees that `s` is always a valid status, and the `next` function encodes the legal transitions in one place. This is a small example of a **state machine** — a powerful and common way to model anything that moves through stages, from a game’s screens to a network connection’s lifecycle.

In Depth: why an enum beats a bare integer (optional, for advanced readers)

You could represent a status with a plain `i32` — 0 for pending, 1 for shipped, and so on — and many programs do. But then nothing stops you from assigning 99, or from adding a `Status` to a `Day` by accident, or from forgetting which number meant what. An enum makes those mistakes impossible: the type system knows the only valid values, the names document themselves, and the compiler checks that you use them consistently. The cost is nothing — an enum compiles down to exactly the integer you would have written by hand — so you get the safety for free.

10.6 Chapter summary

- An enum names a fixed set of values; a variable of that type holds exactly one member.
- Refer to members as `EnumName.Member`; compare them with `==` and `!=`.
- Members have integer values, counting from 0 by default; convert with `as i32`. You may assign explicit values, and later members continue counting up.

- Salam has no `match/switch`; “match” on an enum with an `if/else if` chain that covers every member.
- Enums are ideal for modelling states and transitions (state machines), and are safer than bare integers at no run-time cost.

10.7 Exercises

Exercise 10.1. D

efine enum `Suit: Hearts, Diamonds, Clubs, Spades` end and print the integer value of each member.

Exercise 10.2. W

rite a function `is_red(s Suit) bool` that returns `true` for `Hearts` and `Diamonds`, and test it on all four suits.

Exercise 10.3. D

efine enum `Direction: North, East, South, West` end and a function `turn_right(d Direction) Direction` that rotates clockwise (North becomes East, and so on, West wrapping back to North).

Exercise 10.4. M

odel a traffic light that cycles `Green → Amber → Red → Green`. Write a next function and run it through two full cycles.

Exercise 10.5. D

efine enum `HttpStatus` with `Ok = 200`, `NotFound = 404`, and `ServerError = 500`, and print each member’s number.

Chapter 11

Interfaces and Polymorphism

A `Circle` and a `Rect` are different structs, with different fields, yet both have an *area*. Often you want to write code that works with “anything that has an area” without caring exactly which shape it is. An **interface** captures that idea: it describes a set of methods a type must provide, and lets you write code against the interface rather than any one concrete type. This ability to treat many types through one shared description is called **polymorphism**.

11.1 Defining an interface

An interface is declared with the `interface` keyword and a list of method *signatures* — method names with their parameters and return types, but no bodies:

```
1 interface Shape:
2     func area() f64
3     func name() str
4 end
```

This says: “a `Shape` is anything that has an `area()` returning an `f64` and a `name()` returning a `str`.” The interface promises what methods exist; it does not say how they work — that is each type’s job.

11.2 Implementing an interface

Here is the pleasant part: in `Salam` a struct implements an interface simply by *having the right methods*. There is no `implements` clause to write and

nothing to declare — if a struct has an `area()` and a `name()` with matching signatures, it already *is* a `Shape`. This is called **structural conformance**.

```

1 interface Shape:
2     func area() f64
3     func name() str
4 end
5
6 struct Circle:
7     r f64 = 0.0
8     func area() f64: ret 3.14159 * this.r * this.r end
9     func name() str: ret "circle" end
10 end
11
12 struct Rect:
13     w f64 = 0.0
14     h f64 = 0.0
15     func area() f64: ret this.w * this.h end
16     func name() str: ret "rect" end
17 end

```

Both `Circle` and `Rect` provide `area()` and `name()`, so both conform to `Shape` — automatically, without either struct mentioning the interface. Now we can write code that works with any `Shape`.

11.3 Static polymorphism: interface bounds

The first and most efficient way to use an interface is as a **bound** on a generic function. The notation `<T: Shape>` after a function name means “for any type `T` that conforms to `Shape`.” Inside the function you may call any of the interface’s methods on a value of type `T`:

```

1 func describe<T: Shape>(s T):
2     println s.name(), "area =", s.area()
3 end
4
5 func main:
6     describe(Circle { r = 2.0 })
7     describe(Rect { w = 3.0, h = 4.0 })
8 end

```

```

circle area = 12.5664
rect area = 12

```


The one `describe` function serves both shapes. You did not have to write it twice, and you did not have to tell it which type you are passing — the compiler works that out from the argument. Notice there are no angle brackets at the call site: you just write `describe(Circle { r = 2.0 })`, and Salam infers that `T` is `Circle`.

In Depth: monomorphization, or “zero-cost” generics (optional, for advanced readers)

When you call `describe(Circle {...})`, the compiler generates a specialised copy of `describe` in which `T` is exactly `Circle`, and another copy for `Rect`. Each copy calls the methods directly, with no run-time lookup — it is as fast as if you had written two separate functions by hand. This technique, called *monomorphization*, is why interface bounds are described as “zero-cost”: you get the convenience of writing one generic function and the speed of hand-specialised code. The price is a little extra compiled size, one copy per type you actually use.

11.4 Dynamic polymorphism: dyn

Interface bounds are resolved at compile time, which means each call site knows the concrete type. But sometimes you want a *single* collection holding a mixture of shapes — some circles, some rectangles — and to call `area()` on each without knowing which is which until the program runs. For that, Salam offers `dyn`.

A value of type `dyn Shape` is “some shape, decided at run time.” You can store different concrete types in one place and call interface methods on them, and Salam dispatches each call to the right type’s method dynamically:

```
1 func describe(s dyn Shape):
2     println s.name(), "area =", s.area()
3 end
4
5 func main:
6     describe(Circle { r = 1.0 })
7     describe(Rect { w = 2.0, h = 3.0 })
8 end
```

```
circle area = 3.14159
rect area = 6
```

The difference from the bounded version is what happens under the hood: a `dyn Shape` carries, alongside the data, a hidden table of which methods to call, so the right `area()` is chosen at run time. This is exactly what lets a single list hold a heterogeneous mix of shapes.

Note

When to use which. Reach for an interface *bound* (`<T: Shape>`) by default — it is faster and is the right tool when each call works with one known type. Reach for `dyn` when you genuinely need to mix different concrete types in one collection or behind one variable, and decide the method at run time. Bounds are about writing one function for many types; `dyn` is about holding many types in one container.

11.5 Designing with interfaces

Interfaces let you program to a *capability* rather than a concrete type, which keeps your code flexible. A function that sorts “anything comparable,” a logger that writes to “anything writable,” a renderer that draws “any shape” — each is expressed by naming the interface it needs and ignoring everything else about its inputs. When you later add a new type that conforms, the existing code works with it unchanged, because it only ever depended on the methods, never the specific struct.

Tip

Keep interfaces small. An interface with one or two methods (“has an area,” “can be drawn”) is easy for many types to satisfy and easy to reason about. A large interface demanding a dozen methods is a burden to implement and tends to couple your code to one specific design. Prefer several small interfaces over one big one.

11.6 Chapter summary

- An interface lists method signatures (no bodies) that a conforming type must provide.
- Conformance is *structural*: a struct implements an interface just by having the matching methods — no `implements` declaration.
- An interface *bound*, `func f<T: Iface>(x T)`, writes one function usable

with any conforming type; the type is inferred at the call site and resolved at compile time (monomorphization, zero run-time cost).

- `dyn Iface` holds “some conforming type, decided at run time,” enabling heterogeneous collections via dynamic dispatch.
- Use bounds by default; use `dyn` when you must mix concrete types in one place.
- Favour small interfaces that many types can easily satisfy.

11.7 Exercises

Exercise 11.1. D

efine an interface `Greeter` with a method `greet() str`, and two structs (say `Formal` and `Casual`) that conform. Write a bounded function that prints any greeter’s greeting.

Exercise 11.2. A

dd a third shape, `Triangle` (base and height), to the `Shape` example. Confirm the existing `describe` works on it with no changes.

Exercise 11.3. G

ive `Shape` a third method `perimeter() f64` and implement it for `Circle` and `Rect`. What error do you get if you forget one?

Exercise 11.4. W

rite a bounded function `bigger<T: Shape>(a T, b T) str` that returns the name of whichever shape has the larger area. (Both arguments are the same type here — why?)

Exercise 11.5. E

xplain, in a short comment, when you would choose `dyn Shape` over `<T: Shape>`, using a list of mixed shapes as your example.

Chapter 12

Generics

You met `Vector<i32>` back in Chapter 7 and the bound `<T: Shape>` in Chapter 11. Both rely on **generics**: the ability to write code that works for *any* type, with the type filled in later. Generics let you write a function or a data structure once and reuse it for integers, strings, your own structs — anything — without copying and pasting. This chapter brings the idea into full focus.

12.1 The problem generics solve

Suppose you want a function that returns the larger of two values. For integers:

```
1 func max_i32(a i32, b i32) i32:  
2     if a > b: ret a end  
3     ret b  
4 end
```

For floating-point numbers you would write *the same logic again* with `f64` in place of `i32`, and again for any other comparable type. That is wasteful and error-prone — a bug fixed in one copy lingers in the others. Generics let you write the logic a single time.

12.2 Generic functions

A **type parameter** in angle brackets after the function name turns a function generic. The parameter — conventionally `T` — stands for “some type, chosen by the caller,” and you use it like any other type within the function:

```

1 func Max<T>(a T, b T) T:
2     if a > b: ret a end
3     ret b
4 end
5
6 func main:
7     println Max(10, 20)      // T is i32
8     println Max(2.5, 1.5)   // T is f64
9 end

```

```

20
2.5

```

One `Max` now serves every type. The caller writes `Max(10, 20)` with no angle brackets; Salam looks at the arguments and *infers* that `T` is `i32` for the first call and `f64` for the second. The two parameters `a` and `b` share the type `T`, so they must be the *same* type at each call — you cannot write `Max(10, 2.5)`, because `T` cannot be both `i32` and `f64` at once.

12.3 Generic structs

Data structures benefit even more from generics. A “box that holds one value” should not care whether that value is an integer or a string. Declare a type parameter after the struct’s name and use it for fields:

```

1 struct Box<T>:
2     value T
3 end
4
5 func main:
6     mut a Box<i32> = Box { value = 42 }
7     mut b Box<str> = Box { value = "hi" }
8     println a.value
9     println b.value
10 end

```

```

42
hi

```

`Box<i32>` is a box of integers; `Box<str>` is a box of strings. You spell out the concrete type in the variable’s declaration — `Box<i32>` — and from then on

the field `value` has that type. This is precisely how `Vector<T>` works: it is a generic struct holding a growable buffer of `T`.

Note

At the *call site* of a generic function, you usually omit the type — `Max(10, 20)` — because Salam infers it from the arguments. But when you declare a variable of a generic *struct* type, you write the concrete type explicitly: `Box<i32>`, `Vector<str>`. The struct's type parameter cannot always be inferred from an empty `Box {}`, so you state it.

12.4 A generic container: a simple stack

Let us build something real: a generic stack, a last-in-first-out store, on top of a fixed array. Because it is generic, the same code gives you a stack of integers, a stack of strings, or a stack of anything:

```

1 struct Stack<T>:
2     items T[16] = [0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0]
3     count i32 = 0
4     func push(x T):
5         this.items[this.count] = x
6         this.count = this.count + 1
7     end
8     func pop() T:
9         this.count = this.count - 1
10        ret this.items[this.count]
11    end
12    func is_empty() bool:
13        ret this.count == 0
14    end
15 end
16
17 func main:
18     mut s Stack<i32> = Stack {}
19     s.push(3)
20     s.push(7)
21     s.push(1)
22     println s.pop()
23     println s.pop()
24     println s.is_empty()
25 end

```

```
7  
false
```

The methods `push` and `pop` are written entirely in terms of `T`. Declare `Stack<i32>` and you get an integer stack; declare `Stack<str>` and the very same definition gives you a string stack. This is how the standard library’s collections are built, and how you build your own.

Note

The `items` field carries a default (sixteen zeros) so that `Stack {}` is a complete value with every field initialized — the same rule from Chapter 9 that every struct field needs a default if you want to construct the struct with empty braces. For a real, growable container you would build on `Vector<T>` (Chapter 7) instead of a fixed array; this small fixed-size stack keeps the focus on the generics.

12.5 Bounded generics revisited

A bare `<T>` accepts *any* type, but then you can only do to a `T` what every type supports. The moment you call a method on a `T` — `s.area()`, say — the compiler must know that `T` actually *has* that method. That is what an interface **bound** (Chapter 11) provides: `<T: Shape>` means “any `T`, as long as it conforms to `Shape`,” which unlocks the interface’s methods inside the function.

```
1 interface Shape:  
2     func area() f64  
3 end  
4  
5 func total_area<T: Shape>(a T, b T) f64:  
6     ret a.area() + b.area()  
7 end
```

So there is a spectrum: `<T>` for code that treats values opaquely (store, move, return them), and `<T: SomeInterface>` for code that needs to *do* something specific with them. You reach for the bound exactly when you need to call the interface’s methods.

12.6 How generics work: monomorphization

In Depth: one template, many concrete copies (optional, for advanced readers)

Salam implements generics by *monomorphization*, the same mechanism behind interface bounds. The generic definition — `Max<T>` or `Stack<T>` — is a *template*; the compiler never emits it directly. Instead, each time you use it with a concrete type, the compiler stamps out a specialised copy with the type filled in: `Max` becomes a real `Max_i32` and a real `Max_f64`; `Stack<i32>` becomes a concrete C struct with a `i32[16]` field. Those copies are ordinary, fully type-checked code, as fast as anything hand-written — there is no boxing, no run-time type tag, no indirection. The trade-off is compiled size: one copy exists for each distinct type you instantiate. In practice that is a small price for code that is both reusable and fast.

To make this possible, the compiler needs to know an element's size at compile time. The built-in `sizeof(T)` gives it: the standard library's `Vector<T>`, for instance, uses `sizeof(T)` to allocate exactly enough bytes for each element when it grows its buffer. You rarely call `sizeof` yourself, but it is the quiet machinery that lets one generic container hold elements of any size.

12.7 When to make something generic

Generics are powerful, but not everything needs them. A good rule:

- Make a *container* generic almost always — a stack, queue, list, or tree is naturally indifferent to what it stores.
- Make a *function* generic when you find yourself about to copy it for a second type. The first copy is fine; the second is the signal.
- Do *not* reach for generics when a single concrete type is all you will ever need — the extra abstraction earns its keep only when there is real reuse.

12.8 Chapter summary

- Generics write code once for *any* type, using a type parameter `<T>`.

- A generic function infers `T` from its arguments at the call site; parameters sharing `T` must be the same type.
- A generic struct (`Box<T>`, `Stack<T>`, `Vector<T>`) is declared with its concrete type, e.g. `Stack<i32>`.
- A bare `<T>` treats values opaquely; an interface bound `<T: Iface>` lets you call the interface's methods on a `T`.
- Generics compile by *monomorphization* into specialised, fast copies; `sizeof(T)` is the machinery that lets a container size its elements.

12.9 Exercises

Exercise 12.1. W

rite a generic function `Min<T>(a T, b T) T` mirroring `Max`, and test it on integers and floats.

Exercise 12.2. W

rite a generic function `first_of<T>(a T, b T) T` that simply returns its first argument, and explain why it needs no bound.

Exercise 12.3. D

efine a generic struct `Pair<T>` holding two values of type `T` and a method `swap()` that exchanges them. Test it with a pair of integers.

Exercise 12.4. E

xtend the generic `Stack` with a `peek()` method that returns the top element without removing it. Test it with a `Stack<str>`.

Exercise 12.5. E

xplain in a comment why `Max(10, 2.5)` fails to compile, in terms of the type parameter `T`.

Chapter 13

Closures and Lambdas

So far functions have been things we *define* and *call*. In this chapter functions become *values*: things we can store in a variable, pass to another function, and create on the spot without giving them a name. A small, unnamed function is called a **lambda**, and when one captures variables from around it, it becomes a **closure**. These ideas unlock a flexible, expressive style of programming.

13.1 Functions as values

A **lambda** is a function written as an expression, with no name. Its syntax is a parameter list, the arrow `=>`, and a body. For a one-expression body, just write the expression:

```
1 func main:
2   double auto = (x i32) => x * 2
3   println double(21)
4 end
```

42

Here `(x i32) => x * 2` is a function value: it takes an `i32` called `x` and produces `x * 2`. We store it in the variable `double` (letting `auto` infer its type) and then *call* it just like any function: `double(21)`. The lambda has no name of its own — the name `double` belongs to the variable, not the function.

13.1.1 The type of a function value

The type of `double` is `func(i32) i32`: “a function taking an `i32` and returning an `i32`.” In general a function-value type is written `func(argument types) return type`. A function value taking nothing and returning an `i32` has type `func() i32`. You will see these types most often where one function accepts another as a parameter.

13.1.2 Multi-statement bodies

When a lambda needs more than one statement, give it a block body: a colon after the arrow, the statements, and `end` — exactly like a named function, just without the name:

```
1 (x i32) => :
2     mut y i32 = x * x
3     ret y + 1
4 end
```

The single-expression form (`=> x * 2`) is shorthand for the common case; the block form (`=> : ... end`) is there when you need it.

13.2 Higher-order functions

A function that takes another function as a parameter — or returns one — is called a **higher-order function**. Because a function value has a type like `func(i32) i32`, you can declare a parameter of that type and call it inside:

```
1 func apply(f func(i32) i32, x i32) i32:
2     ret f(x)
3 end
4
5 func main:
6     println apply((x i32) => x + 100, 5)
7 end
```

105

`apply` does not know or care what `f` does; it just calls `f(x)`. We hand it a lambda created right at the call site, `(x i32) => x + 100`, and `apply` runs it on 5. This is the essence of higher-order programming: behaviour passed around as data. The same `apply` could square, negate, or double its argument, depending only on the lambda you give it.

13.3 Closures: capturing the surrounding state

Here is where function values become truly powerful. A lambda can *use variables from the scope in which it was created* — not just its own parameters. When it does, it “closes over” those variables, and we call it a **closure**. The captured values travel with the function, even after the scope that created them has returned.

The classic example is a function that *builds* another function:

```
1 func make_adder(n i32) func(i32) i32:
2     ret (x i32) => x + n
3 end
4
5 func main:
6     add10 auto = make_adder(10)
7     add100 auto = make_adder(100)
8     println add10(5)
9     println add100(5)
10 end
```

```
15
105
```

`make_adder` returns a lambda that adds `n` — but `n` is `make_adder`’s parameter, not the lambda’s. The returned closure *captures* the value of `n`. So `make_adder(10)` produces a function that always adds 10, and `make_adder(100)` produces a separate one that always adds 100. Each closure remembers its own captured `n`, long after `make_adder` has finished running.

13.4 Closures that remember and change state

Because a closure carries its captured variables with it, it can also *update* them between calls — giving a function that has memory. A counter is the canonical example:

```
1 func make_counter() func() i32:
2     mut count i32 = 0
3     ret () => :
4         count = count + 1
5         ret count
6     end
7 end
8
```

```
9 func main:
10   next func() i32 = make_counter()
11   println next()
12   println next()
13   println next()
14 end
```

```
1
2
3
```

Each call to `next()` increments and returns the captured `count`. The variable `count` lives on inside the closure even though `make_counter` returned long ago — the closure has kept it alive. Call `make_counter()` a second time and you get a fresh, independent counter starting again at zero.

In Depth: how closures are implemented (optional, for advanced readers)

A closure is more than a pointer to code — it must also carry the variables it captured. Salam implements this by allocating a small *environment* on the heap holding the captured values, and pairing it with the lambda’s code. The function value you pass around is really this code-plus-environment bundle. When you call it, the captured variables are read from (and written to) that environment, which is why `count` survives between calls and why two counters do not interfere. The compiler performs this “lifting” of the lambda and its captures automatically; you simply write the closure and it works.

13.5 Why closures matter

Closures let you parameterise *behaviour*, not just values. A sort routine can take a closure describing how to compare two elements; a retry helper can take a closure describing the work to attempt; an event system can take closures to run when something happens. In each case you hand over a small piece of custom logic, created exactly where it is needed and carrying whatever surrounding context it requires. This keeps general machinery (the sorter, the retriever) cleanly separated from the specific behaviour you plug into it.

Tip

Lambdas are at their best when they are small — a comparison, a transformation, a quick test. If a lambda grows into a substantial block of logic, give it a real name with `func` instead; a named function is easier to read, reuse, and discuss. Reach for a lambda when naming it would add noise, not clarity.

13.6 Chapter summary

- A **lambda** is an unnamed function value: `(params) => expression`, or `(params) => : ... end` for a block body.
- A function value has a type like `func(i32) i32` or `func() i32`.
- A **higher-order function** takes or returns a function value; call a function-typed parameter just like any function.
- A **closure** is a lambda that captures variables from its surrounding scope; the captured values live on with the closure.
- Closures can carry mutable state (e.g. a counter), and each closure instance has its own independent captured variables.
- Use lambdas for small, local behaviour; promote anything substantial to a named function.

13.7 Exercises

Exercise 13.1. C

reate a lambda `square` that returns its argument times itself, store it with `auto`, and print `square(7)`.

Exercise 13.2. W

rite a higher-order function `apply_twice(f func(i32) i32, x i32) i32` that returns `f(f(x))`, and test it with a lambda that adds 3.

Exercise 13.3. W

rite `make_multiplier(factor i32) func(i32) i32` that returns a closure multiplying by `factor`. Build a “times 3” and a “times 10” function from it.

Exercise 13.4. U

Using `make_counter`, create two independent counters and show by interleaving their calls that they do not share state.

Exercise 13.5. W

Write a higher-order function `transform_all` that takes a `Vector<i32>` and a `func(i32) i32`, and returns a new vector with the function applied to every element. Test it with a doubling lambda.

Chapter 14

Error Handling and Deferred Execution

Real programs deal with things that can go wrong: a withdrawal larger than the balance, a number that will not parse, a file that is not there. Equally, real programs acquire resources — open files, allocated memory — that must be released no matter how the work turns out. This chapter covers Salam’s straightforward approach to both: reporting failure through return values, and guaranteeing cleanup with `defer`.

14.1 Salam’s philosophy: errors are values

Salam has no exceptions, no `try/catch`, no invisible control flow that whisks you off to some distant handler. Instead, a function that can fail *says so in what it returns*, and the caller checks. This keeps the flow of your program on the page: every place that can fail is visible, and every failure is handled where it happens. There are a few common patterns, from simplest to most expressive.

14.2 Pattern 1: a boolean success flag

The simplest signal is a `bool`: `true` for success, `false` for failure. You saw this in Chapter 9 with the bank account, whose `withdraw` returned whether it succeeded:

```
1 struct Account:
2     balance i32 = 0
```

```

3  func withdraw(amount i32) bool:
4      if amount > this.balance:
5          ret false           // failure: not enough funds
6      end
7      this.balance = this.balance - amount
8      ret true                // success
9  end
10 end
11
12 func main:
13     mut acc Account = Account { balance = 100 }
14     if acc.withdraw(30):
15         println "withdrew 30; balance is", acc.balance
16     else:
17         println "withdrawal failed"
18     end
19     if acc.withdraw(1000):
20         println "withdrew 1000"
21     else:
22         println "withdrawal failed: insufficient funds"
23     end
24 end

```

```

withdrew 30; balance is 70
withdrawal failed: insufficient funds

```

The caller cannot ignore the outcome by accident: it is right there in the `if`. This pattern is perfect when the only thing you need to know is “did it work?”

14.3 Pattern 2: a sentinel return value

When a function returns a value but might have nothing meaningful to give back, a **sentinel** — a special value that means “no result” — is a common choice. A search that returns the index of an item can return `-1` to mean “not found,” since a real index is never negative:

```

1  func index_of(values i32[5], target i32) i32:
2      for mut i = 0; i < 5; i = i + 1:
3          if values[i] == target:
4              ret i
5          end
6      end
7      ret -1           // sentinel: not found

```

```
8 end
9
10 func main:
11     nums i32[5] = [10, 20, 30, 40, 50]
12     found auto = index_of(nums, 30)
13     if found >= 0:
14         println "found at index", found
15     else:
16         println "not found"
17     end
18     println "search for 99:", index_of(nums, 99)
19 end
```

```
found at index 2
search for 99: -1
```

Watch out

A sentinel only works when there is a value the real results can never take. For an index, -1 is safe. But “zero means failure” is dangerous if zero is a legitimate result, and “empty string means error” fails if the empty string is valid data. When no value can be safely reserved, prefer the Option approach below.

14.4 Pattern 3: Option — a value that may be absent

The standard library’s `Option` type makes “maybe a value, maybe nothing” explicit and safe, without stealing a value from the result’s range. An `Option<T>` either holds a `T` (it is *some* value) or holds nothing (it is *none*). Import it with `import "option"`.

```
1 import "option"
2
3 func safe_divide(a i32, b i32) option.Option<i32>:
4     if b == 0:
5         ret Option { value = 0, present = false } // none
6     end
7     ret option.Some(a / b) // some result
8 end
9
10 func main:
11     r1 auto = safe_divide(10, 2)
```

```

12  r2 auto = safe_divide(10, 0)
13  println "10/2 is some?", option.IsSome(r1)
14  println "10/0 is some?", option.IsSome(r2)
15  println "10/2 =", option.UnwrapOr(r1, -1)
16  println "10/0 =", option.UnwrapOr(r2, -1)    // falls back to -1
17  end

```

```

10/2 is some? true
10/0 is some? false
10/2 = 0
10/0 = -1

```

The `option` module gives you the tools to work with these values safely:

Function	What it does
<code>option.Some(x)</code>	wrap <code>x</code> as a present value
<code>option.IsSome(o)</code>	true if <code>o</code> holds a value
<code>option.IsNone(o)</code>	true if <code>o</code> holds nothing
<code>option.UnwrapOr(o, d)</code>	the held value, or <code>d</code> if none

`UnwrapOr` is especially handy: it gets the value out, supplying a fallback for the “none” case in one step, so you never accidentally use a value that is not there. To build a “none,” you construct the `Option` with `present = false`, as the example shows.

Note

`Option` is the type to reach for whenever “no answer” is a legitimate outcome: a lookup that may miss, a parse that may fail, the first element of a possibly-empty collection. It forces the caller to acknowledge the empty case, turning a whole class of “forgot to check” bugs into something the type system helps you remember.

14.5 Deferred execution with `defer`

Now to the second half of the chapter: making sure cleanup happens. The `defer` keyword schedules a statement to run *when the current function exits* — no matter how it exits. You write the cleanup right next to the thing it cleans up, and Salam guarantees it runs later:

```

1  func main:
2      defer println "cleanup runs last"

```

```
3     println "doing work"
4     println "more work"
5 end
```

```
doing work
more work
cleanup runs last
```

The `defer`'d line was *written* first but *ran* last, at the moment `main` finished. The point is that the cleanup is registered the instant you set up whatever needs cleaning, so you can never forget it later.

14.5.1 Defers run in reverse order

If you register several defers, they run in **last-in, first-out** order — the most recently deferred runs first. This mirrors how resources nest: the last thing you opened is the first thing you should close.

```
1 func main:
2     defer println "cleanup 1"
3     defer println "cleanup 2"
4     println "start"
5     println "end"
6 end
```

```
start
end
cleanup 2
cleanup 1
```

14.5.2 Defer runs even on an early return

The real power of `defer` is that it runs however the function ends — including an early `ret` from somewhere in the middle. This is what makes it reliable: the cleanup cannot be skipped by a code path you forgot about.

```
1 func greet(name str):
2     defer println "done greeting"
3     println "hello"
4     if name == "":
5         println "no name given"
6         ret                                // early return -- defer still
                                           runs
```

```

7     end
8     println name
9 end
10
11 func main:
12     greet("Ali")
13     println "----"
14     greet("")
15 end

```

```

hello
Ali
done greeting
----
hello
no name given
done greeting

```

Both calls print "done greeting" — the normal path through Ali and the early-return path through the empty name. You write the cleanup once, and it fires on every exit.

14.6 Putting it together: acquire, defer, use

The idiomatic pattern is: acquire a resource, immediately **defer** its release, then use it freely without worrying about the exits. Whether the function returns normally, returns early on an error, or falls through, the resource is released exactly once. We will see this with real files and memory in Chapters 17 and 18; the shape is always the same:

```

1 func process():
2     // acquire something here
3     defer println "release the resource"
4
5     println "use the resource"
6     if true:
7         println "early exit on some condition"
8         ret // the defer above still runs
9     end
10    println "this line is skipped"
11 end
12
13 func main:
14    process()

```

15 `end`

```
use the resource
early exit on some condition
release the resource
```

Tip

Pair every acquisition with a `defer` on the very next line. Reading top to bottom, “open file; defer close file” is impossible to get wrong — the close is guaranteed and sits right where you can see it. This single habit eliminates the most common kind of resource leak.

14.7 Chapter summary

- Salam reports errors through *return values*, not exceptions — every failure is visible and handled at the call site.
- Return a `bool` when the caller only needs “did it work?”.
- Return a **sentinel** (like `-1`) when a value’s range has room for a “no result” marker — but only then.
- Use `option.Option<T>` (`Some`, `IsSome`, `IsNone`, `UnwrapOr`) when “no value” must be represented safely.
- `defer` schedules a statement to run when the function exits, in last-in-first-out order, *including* on early returns.
- The reliable pattern is: acquire, immediately `defer` the release, then use.

14.8 Exercises

Exercise 14.1. W

rite `safe_sqrt(x f64) option.Option<f64>` that returns `none` for negative input and `Some` of the square root otherwise (use the `math` module from the next chapter, or `x ** 0.5`).

Exercise 14.2. W

rite a function `find_first_negative(a i32[5]) i32` that returns the index of the first negative element or `-1` if there is none.

Exercise 14.3. A

dd two `defer` statements and a normal statement to a function, and predict the output order before running it.

Exercise 14.4. W

rite a function with an early `ret` guarded by a condition and a `defer` that prints “finished”. Verify “finished” prints on both paths.

Exercise 14.5. R

ewrite the `safe_divide` example so the caller prints a friendly message in the “none” case instead of using `Unwrap0r`.

Chapter 15

Modules and Packages

A program that outgrows a single file needs structure. **Modules** and **packages** are how Salam splits a large program into pieces, controls what each piece exposes to the others, and pulls in the standard library. You have been quietly using this system every time you wrote `import "str"`; now we look at it properly.

15.1 Importing the standard library

The standard library is organised into modules, each a self-contained bundle of related functions: `math` for numbers, `str` for strings, `io` for input and output, and many more. To use one, `import` it by name and call its functions with the module name as a prefix:

```
1 import "math"
2
3 func main:
4     println math.Sqrt(144.0)
5     println math.MaxI(7, 3)
6 end
```

```
12
7
```

The prefix — `math.` — is called **qualification**, and it does two useful things. It tells the reader exactly where `Sqrt` comes from, and it keeps names from colliding: `math.Max` and your own `max` can coexist without confusion.

Note

A handful of operations need no import at all: `print`, `println`, and `len` are built into the language and are always available. Everything else in the standard library lives in a module you import.

15.2 Importing several modules

You can write one `import` per module, or group them in a single parenthesised block — the grouped form is tidy when a file uses several modules:

```
1 import (
2     "math"
3     "str"
4     "io"
5 )
6
7 func main:
8     io.Println(str.Upper("salam"))
9     println math.Gcd(12, 18)
10 end
```

```
SALAM
6
```

15.3 Aliasing an import

Sometimes a module's name is long, or clashes with a name you want to use yourself. You can give an import a shorter **alias** by writing the alias before the module name:

```
1 import (
2     strutil "str"
3     "io"
4 )
5
6 func main:
7     io.Println(strutil.Upper("salam"))
8 end
```

```
SALAM
```

Now the `str` module is reached as `strutil`. Aliases are a matter of convenience and clarity; use them when they make the code read better, and leave them off otherwise.

15.4 How imports are resolved

When you write `import "math"`, Salam looks for the module `math` in the standard library — specifically the file `std/math/math.salam`. A nested module path works the same way: `import "db/sqlite"` resolves to `std/db/sqlite/sqlite.salam`. An import whose name ends in `.salam`, such as `import "helpers.salam"`, is instead treated as a *file relative to the importing file* — which is how you bring in your *own* modules, the subject of the next section.

15.5 Writing your own modules

A module is just a `.salam` file with definitions in it. To split your program across files, put related functions and types in their own file and import it where you need them. Two pieces of machinery make this work: the `package` declaration and the `pub` keyword.

15.5.1 Declaring a package

Every file belongs to a **package**, named at the top with the `package` keyword. Your program's entry point lives in package `main`:

```
1 package main
2
3 func main:
4     println "Hello from package main"
5 end
```

```
Hello from package main
```

A file with no `package` line belongs to `main` by default — which is why the small single-file programs earlier in this book never needed one. As soon as you split code across files, naming the package makes the structure explicit.

15.5.2 Exposing names with `pub`

By default, a top-level function, struct, or type is **private** to its package: other files cannot see it. To make something visible to code that imports

your module, mark it `pub` (short for *public*):

```
1 // file: geometry.salam
2 package geometry
3
4 pub func area_of_circle(r f64) f64:
5     ret 3.14159 * r * r
6 end
7
8 func helper() f64:           // no 'pub' -- private to this file
9     ret 0.0
10 end
```

Another file can then import `geometry.salam` and use its public functions, while `helper` stays hidden:

```
1 // file: main.salam
2 package main
3 import "geometry.salam"
4
5 func main:
6     println geometry.area_of_circle(2.0)
7 end
```

This is **encapsulation**: a module presents a small, deliberate public surface — the things it promises to support — while keeping its internal helpers private and free to change. The same `pub` keyword marks the public parts of the standard library itself; open any `std` module and you will see `pub func` and `pub struct` on exactly the names you are meant to use.

Tip

Make things `pub` sparingly. The smaller a module's public surface, the easier it is to use correctly and the freer you are to rewrite its insides without breaking anyone. Start everything private; promote a name to `pub` only when another file genuinely needs it.

15.6 A tour of the standard library's layout

It helps to know roughly what is available before you need it. The standard library is a collection of modules under `std`, each in its own folder. A selection:

Module	Purpose
<code>io, console</code>	printing and reading input, files
<code>fmt</code>	formatting values into strings
<code>math</code>	numeric functions and constants
<code>str</code>	string operations
<code>rand</code>	random numbers
<code>time</code>	dates, timestamps, sleeping
<code>collections</code>	generic containers (<code>Vector</code> , <code>HashMap</code> , ...)
<code>os, fs</code>	the operating system and file system
<code>json, csv</code>	data formats
<code>regex</code>	regular expressions
<code>option</code>	the <code>Option</code> type
<code>mem</code>	manual memory management
<code>sync, thread</code>	concurrency primitives
<code>testing</code>	assertions for unit tests
<code>net, tcp, http</code>	networking

The next two chapters take a proper tour: Chapter 16 covers the everyday modules (`io`, `fmt`, `math`, `str`, `rand`, `time`), and Chapter 17 covers data structures and the world outside your program (`collections`, `os`, `fs`, `json`, and friends).

In Depth: qualified names and the generated `C` (optional, for advanced readers)

Qualification is not only for readability — it also guarantees that names from different modules never collide in the compiled program. When `math.Sqrt` is compiled, it becomes a uniquely named `C` function (something like `_Salam_math_Sqrt_f64`) that cannot clash with a `Sqrt` from any other module or with your own code. The package system, in other words, gives you human-friendly short names at the source level and collision-proof unique names underneath — the best of both worlds, with no run-time cost.

15.7 Chapter summary

- `import "name"` brings in a standard-library module; call its functions qualified, as `name.func(...)`.
- Group several imports in one `import ( ... )` block; give any import an alias by writing the alias before the name.

- `import "math"` resolves to `std/math/math.salam`; an import ending in `.salam` is a file relative to the importer (your own modules).
- Every file has a **package**; the entry point is in `main` (the default when omitted).
- Top-level names are private to their package unless marked **pub**.
- `print`, `println`, and `len` are built in and need no import.

15.8 Exercises

Exercise 15.1. W

rite a program that imports `math` and `str` in a grouped `import` block and prints the square root of 2 alongside the word "root" in uppercase.

Exercise 15.2. I

import `str` under the alias `s` and use it to reverse a word.

Exercise 15.3. S

ketch (on paper) a two-file program: a `shapes.salam` module with a **pub** function and a private helper, and a `main.salam` that imports it. Mark which names are visible where.

Exercise 15.4. O

pen one of the real `std` modules in the Salam source and list five **pub** functions it offers. Try calling two of them.

Exercise 15.5. E

xplain in a comment why qualification (`math.Max`) lets you keep your own function called `max` without conflict.

Chapter 16

The Standard Library, Part 1

A language is only as productive as the library that comes with it. Salam ships a substantial standard library so that you rarely have to reinvent the basics. This chapter tours the everyday modules — input and output, formatting, mathematics, randomness, and time — with runnable examples of the functions you will use most. The next chapter covers data structures and the world outside your program.

16.1 Input and output: `io`

You already know `println` and `print`, which are built in. The `io` module offers the same operations as named functions (handy when you import it for file work) plus the all-important `Input`, which reads a line of text the user types:

```
1 import "io"
2
3 func main:
4     io.Print("What is your name? ")
5     name str = io.Input()
6     io.Println("Hello, " + name + "!")
7 end
```

`io.Input()` reads one line from the keyboard and returns it as a `str` (without the trailing newline). Combined with the `str` conversions from Chapter 8, this is how you build interactive programs:

```
1 import (
2     "io"
3     "str"
```

```
4 )
5
6 func main:
7     io.Print("Enter a number: ")
8     n auto = str.ToInt(io.Input())
9     io.Println("Its square is " + str.FromInt((n * n) as i64))
10 end
```

Note

Because `io.Input` returns text, you must convert it to a number with `str.ToInt` (or `str.ToFloat`) before doing arithmetic. Reading input as text and converting explicitly is safer than guessing types — you decide exactly how each line should be interpreted.

16.2 Formatting values: `fmt`

The `fmt` module turns values into strings and lays them out neatly. The conversions are the bread and butter:

```
1 import "fmt"
2
3 func main:
4     println fmt.Int(255 as i64)           // "255"
5     println fmt.Float(3.14159)           // "3.14159"
6     println fmt.Bool(true)               // "true"
7 end
```

```
255
3.14159
true
```

For aligning text into columns — tables, menus, reports — `fmt` provides padding and repetition:

```
1 import "fmt"
2
3 func main:
4     println fmt.PadLeft("hi", 5)           // "    hi"
5     println fmt.PadRight("hi", 5) + "|" // "hi    |"
6     println fmt.Repeat("=", 10)           // "==========="
7 end
```



```

    hi
hi   |
=====

```

`PadLeft` and `PadRight` widen a string to a given width by adding spaces on the left or right, so numbers and labels line up in columns. `Repeat` builds a string of repeated copies — perfect for rules and separators.

16.3 Mathematics: `math`

The `math` module covers the numeric functions you would expect, working on `f64` values (with a few integer helpers). Here are the most useful, grouped by purpose.

16.3.1 Rounding and sign

```

1 import "math"
2
3 func main:
4     println math.Floor(3.7)    // 3 (round down)
5     println math.Ceil(3.2)     // 4 (round up)
6     println math.Round(3.5)    // 4 (nearest)
7     println math.Abs(-4.0)     // 4 (absolute value)
8 end

```

```

3
4
4
4

```

16.3.2 Roots, powers, and logarithms

```

1 import "math"
2
3 func main:
4     println math.Sqrt(144.0)   // 12
5     println math.Pow(2.0, 10.0) // 1024
6     println math.Min(2.0, 9.0), math.Max(2.0, 9.0)
7 end

```

```
12
1024
2 9
```

16.3.3 A reference of common math functions

Function	Returns
<code>math.Sqrt(x)</code>	square root of x
<code>math.Pow(b, e)</code>	b raised to the power e
<code>math.Abs(x)</code>	absolute value (float)
<code>math.Floor(x)</code> / <code>math.Ceil(x)</code>	round down / up
<code>math.Round(x)</code>	round to nearest
<code>math.Min(a, b)</code> / <code>math.Max(a, b)</code>	smaller / larger (float)
<code>math.MinI(a, b)</code> / <code>math.MaxI(a, b)</code>	smaller / larger (integer)
<code>math.Gcd(a, b)</code> / <code>math.Lcm(a, b)</code>	greatest common divisor / least common multiple
<code>math.Sin(x)</code> , <code>math.Cos(x)</code> , <code>math.Tan(x)</code>	trigonometric functions
<code>math.Log(x)</code> , <code>math.Log10(x)</code> , <code>math.Exp(x)</code>	logarithms and the exponential

The module also defines the constants `math.PI`, `math.E`, and `math.TAU`:

```
1 import "math"
2
3 func main:
4     r f64 = 2.0
5     area auto = math.PI * r * r
6     println "area of a circle of radius 2:", area
7 end
```

```
area of a circle of radius 2: 12.5664
```

16.3.4 Integer helpers

A handful of functions work directly on integers, so you do not have to cast back and forth. `math.MaxI` and `math.MinI` compare integers, and `math.Gcd` and `math.Lcm` are exact integer operations:

```
1 import "math"
2
3 func main:
```

```

4  println math.MaxI(7, 3)    // 7
5  println math.Gcd(12, 18)  // 6
6  println math.Lcm(4, 6)    // 12
7  end

```

```

7
6
12

```

16.4 Randomness: rand

The `rand` module produces random values — for games, simulations, sampling, and test data. Start by seeding the generator (use `rand.SeedAuto()` for an unpredictable seed from the clock, or a fixed seed for repeatable runs), then draw values:

```

1  import "rand"
2
3  func main:
4      rand.Seed(42 as i64)           // fixed seed:
      repeatable
5      dice auto = rand.IntRange(1 as i64, 6 as i64)
6      println "you rolled:", dice
7      println "coin flip heads?", rand.Bool()
8      println "random fraction:", rand.Float() // in [0, 1)
9  end

```

The most useful functions:

Function	Returns
<code>rand.Seed(s)</code> / <code>rand.SeedAuto()</code>	set a fixed / clock-based seed
<code>rand.IntRange(lo, hi)</code>	a random integer in <code>[lo, hi]</code>
<code>rand.Float()</code>	a random <code>f64</code> in <code>[0, 1)</code>
<code>rand.Bool()</code>	a random <code>true/false</code>
<code>rand.UUID()</code>	a random unique identifier string

Tip

Seed with a *fixed* number while you are developing and testing, so your program behaves the same on every run and bugs are reproducible. Switch to `rand.SeedAuto()` for the real, shipped program, where you

want genuine unpredictability. Forgetting to seed at all gives you the same “random” sequence every time — occasionally useful, usually a surprise.

16.5 Time: `time`

The `time` module reads the clock, formats timestamps, and pauses your program. A timestamp is a count of seconds, which you can format into a readable date or pick apart into fields:

```
1 import "time"
2
3 func main:
4     now auto = time.Now()           // seconds since the epoch
5     println "formatted:", time.Format(now)
6     println "year:", time.Year(now)
7     println "seconds in a day:", time.SecondsPerDay
8 end
```

`time.Now()` returns the current time as an integer count of seconds; `time.Format` renders it as "YYYY-MM-DD HH:MM:SS", and helpers like `time.Year`, `time.Month`, and `time.Day` extract individual fields. The module also defines the constants `time.SecondsPerMinute`, `time.SecondsPerHour`, and `time.SecondsPerDay`.

To pause, use `time.Sleep`, which takes a number of milliseconds:

```
1 import "time"
2
3 func main:
4     println "wait for it..."
5     time.Sleep(500)           // pause half a second
6     println "done"
7 end
```

16.6 Strings: a reminder

The `str` module — `Upper`, `Lower`, `Split`, `Join`, `Find`, `Replace`, `FromInt`, `ToInt`, and the rest — is the other workhorse of everyday programming, and we covered it thoroughly in Chapter 8. Keep it in mind alongside the modules here; together, `io`, `fmt`, `math`, `str`, `rand`, and `time` handle the vast majority of small-program needs.

16.7 A worked example: a guessing game

Here is a complete little program that ties several of these modules together — a number-guessing game (shown with a fixed secret so the output is predictable):

```
1 import (
2     "io"
3     "str"
4     "rand"
5 )
6
7 func main:
8     rand.Seed(7 as i64)
9     secret auto = rand.IntRange(1 as i64, 100 as i64)
10    io.Print("Guess my number (1-100): ")
11    guess auto = str.ToInt(io.Input())
12    if guess == secret:
13        io.Println("Exactly right!")
14    else if guess < secret:
15        io.Println("Too low.")
16    else:
17        io.Println("Too high.")
18    end
19 end
```

It seeds the generator, picks a secret in [1, 100], reads a guess as text, converts it to a number, and compares — drawing on `rand`, `io`, and `str` at once. That is the standard library doing exactly what it is for: letting you assemble a real program from ready-made parts.

16.8 Chapter summary

- `io` provides `Print`, `Println`, and `Input` (read a line as a `str`).
- `fmt` converts values (`Int`, `Float`, `Bool`) and aligns text (`PadLeft`, `PadRight`, `Repeat`).
- `math` offers rounding, roots, powers, trig, logs, integer helpers (`MaxI`, `Gcd`, `Lcm`), and the constants `PI`, `E`, `TAU`.
- `rand` generates random integers, floats, booleans, and ids; seed it for repeatability or with `SeedAuto` for unpredictability.

- `time` reads and formats the clock, extracts date fields, exposes second-count constants, and sleeps with `Sleep`.

16.9 Exercises

Exercise 16.1. W

Write a program that asks for the user's name and age and prints a sentence using both (convert the age with `str.ToInt`).

Exercise 16.2. P

Print a small multiplication table aligned in columns using `fmt.PadLeft`.

Exercise 16.3. U

Use `math` to print the hypotenuse of a right triangle with legs 3 and 4 (`Sqrt` of the sum of squares).

Exercise 16.4. R

Roll two dice with `rand.IntRange` and print their sum; seed with a fixed value so you can predict the result.

Exercise 16.5. P

Print the current date with `time.Format(time.Now())`, then sleep one second and print it again.

Exercise 16.6. C

Combine `rand` and `fmt` to print five random integers between 1 and 99, each padded to width 2, on one line.

Chapter 17

The Standard Library, Part 2

The first standard-library chapter covered computation: numbers, text, time. This one covers *structure and the outside world* — the containers that organise your data, and the modules that read files, parse data formats, and match patterns. With these you can write programs that do real work on real data.

17.1 Collections beyond the vector

You met `Vector<T>` in Chapter 7. The `collections` module — which is available without an explicit import for the core containers — offers several more generic data structures, each suited to a particular access pattern.

17.1.1 HashMap: key-value lookup

A `HashMap<K, V>` associates **keys** of type `K` with **values** of type `V`, and looks a value up by its key almost instantly. It is the right tool whenever you think “given *this*, find *that*” — a username to a score, a word to its count, a product code to a price.

```
1 func main:
2     mut prices HashMap<str, i32> = HashMap {}
3     prices.insert("apple", 3)
4     prices.insert("banana", 5)
5     println "size:", prices.size()
6     println "apple costs:", prices.get("apple")
7     println "have banana?", prices.has("banana")
8     println "have cherry?", prices.has("cherry")
9     prices.remove("apple")
```

```

10     println "size now:", prices.size()
11     prices.free()
12 end

```

```

size: 2
apple costs: 3
have banana? true
have cherry? false
size now: 1

```

The key methods are `insert(k, v)` to add or update an entry, `get(k)` to retrieve a value, `has(k)` to test for a key, `remove(k)` to delete one, `size()` for the count, and `free()` when you are done.

Watch out

Call `get` only for a key you know is present — check first with `has` if you are not sure. As with vectors, a `HashMap` manages its own memory, so remember `free()` when you have finished with it.

17.1.2 Stack and Queue

When you care about the *order* in which items come back out, two classic containers help. A `Stack<T>` is last-in-first-out — the most recently pushed item pops first, like a stack of plates. A `Queue<T>` is first-in-first-out — items leave in the order they arrived, like a line at a counter.

```

1 func main:
2     mut s Stack<i32> = Stack {}
3     s.push(1)
4     s.push(2)
5     println "top:", s.peak()           // 2
6     println "pop:", s.pop()           // 2 (last in, first out)
7     s.free()
8
9     mut q Queue<str> = Queue {}
10    q.enqueue("first")
11    q.enqueue("second")
12    println q.dequeue()                // "first" (first in, first out)
13    println q.dequeue()                // "second"
14    q.free()
15 end

```

```
top: 2
```



```
pop: 2
first
second
```

A `Stack` uses `push/pop/peek`; a `Queue` uses `enqueue/dequeue/peek`. Both report `size()` and `is_empty()`, and both should be `freed`. Reach for a stack when you need to undo or backtrack (the most recent thing first), and a queue when you must process things fairly in arrival order.

17.2 Reading and writing files

The `io` module, which you met for console output, also reads and writes files. The simplest operations handle a whole file at once:

```
1 import "io"
2
3 func main:
4     io.WriteFile("notes.txt", "line one\nline two\n")
5     content auto = io.ReadFile("notes.txt")
6     print content
7 end
```

```
line one
line two
```

`io.WriteFile(path, text)` writes (creating or replacing the file), and `io.ReadFile(path)` returns the whole file as a `str`. For line-by-line processing, `io.Lines(path)` returns a `Vector<str>`, one entry per line, which you can walk with the techniques from Chapter 7.

17.3 The operating system: `os`

The `os` module is your program's window onto the system it runs on: files and directories, environment variables, command-line arguments, and the exit code.

```
1 import "os"
2
3 func main:
4     println "exists?", os.Exists("notes.txt")
5     println "current dir:", os.Cwd()
6     home auto = os.Env("HOME")
```

```
7 println "home:", home
8 end
```

Common `os` operations include `os.Exists(path)` to test for a file, `os.Remove(path)` to delete one, `os.Mkdir(path)` to create a directory, `os.ListDir(path)` to list a directory's contents as a `Vector<str>`, `os.Args()` for the command-line arguments, `os.Env(name)` for an environment variable, and `os.Exit(code)` to end the program with a status code.

```
1 import (
2     "os"
3     "io"
4 )
5
6 func main:
7     // back up a file, then clean up after ourselves
8     io.WriteFile("data.txt", "important\n")
9     defer os.Remove("data.txt")
10    io.WriteFile("data.bak", io.ReadFile("data.txt"))
11    defer os.Remove("data.bak")
12    println "backup made; files exist?", os.Exists("data.bak")
13 end
```

Notice the `defers` from Chapter 14 doing exactly what they were made for: each temporary file is scheduled for removal the instant it is created, so the program tidies up no matter how it ends.

17.4 Working with JSON

JSON is the lingua franca of data exchange, and the `json` module reads it without ceremony. You can validate a document, pull out fields by key, and ask whether a key is present:

```
1 import "json"
2
3 func main:
4     doc str = "{ \"user\": \"ada\", \"age\": 36 }"
5     println "valid?", json.Valid(doc)
6     println "user:", json.Get(doc, "user")
7     println "age:", json.GetInt(doc, "age")
8     println "has email?", json.Has(doc, "email")
9 end
```

```
valid? true
```

```
user: ada
age: 36
has email? false
```

`json.Valid` checks that the text is well-formed JSON; `json.Get` returns a field as a string, while `json.GetInt` and `json.GetFloat` return it as a number; `json.Has` tests for a key. There is also `json.Escape` to make a string safe to embed in JSON, and `json.Minify` to strip whitespace. This is enough to consume the JSON that web services and config files speak.

17.5 Pattern matching with regular expressions

A **regular expression** (regex) is a compact pattern that describes a set of strings — “one or more digits,” “a word followed by a space.” The `regex` module matches text against such patterns. For one-off use, the `...Str` functions take the pattern as a plain string:

```
1 import "regex"
2
3 func main:
4     println "has digits?", regex.MatchStr("[0-9]+", "abc123")
5     println "digits start at:", regex.FindStr("[0-9]+", "abc123def")
6     println regex.ReplaceStr("[0-9]+", "a1b22c333", "#")
7 end
```

```
has digits? true
digits start at: 3
a#b22c333
```

`regex.MatchStr(pattern, text)` reports whether the pattern occurs anywhere in the text; `regex.FindStr` returns the index where the first match begins (or a negative value if there is none); and `regex.ReplaceStr` substitutes a match with replacement text.

In Depth: compile once, match many (optional, for advanced readers)

If you will apply the same pattern many times — say, validating every line of a large file — compiling it once is more efficient than re-parsing the pattern on each call. `regex.Compile(pattern)` returns a reusable `Regex` value, and `regex.Match(re, text)`, `regex.Find(re, text)`, and `regex.Replace(re, text, with)` operate on it. The `...Str` convenience functions are exactly these with an implicit compile each time; prefer the compiled form in a hot loop.

17.6 Other data-format modules

Rounding out the picture, the standard library also includes a `csv` module for reading and writing comma-separated values (the format spreadsheets export), an `encoding` module for common encodings, and the networking modules `net`, `tcp`, and `http` for talking to other machines. The shape is always the same as what you have seen here: import the module, call its qualified functions, and free anything that manages its own memory. Once you are comfortable with the modules in these two chapters, the rest follow the same patterns.

17.7 A worked example: word frequencies

Let us combine a `HashMap`, the `str` module, and a vector into a small but genuinely useful program: counting how often each word appears in a sentence.

```
1 import "str"
2
3 func main:
4     text str = "the cat sat on the mat the cat ran"
5     mut counts HashMap<str, i32> = HashMap {}
6     mut words Vector<str> = str.Split(text, " ")
7     mut i i32 = 0
8     while i < words.len():
9         w auto = words.get(i)[0]
10        if counts.has(w):
11            counts.insert(w, counts.get(w) + 1)
12        else:
13            counts.insert(w, 1)
14        end
15        i += 1
16    end
17    println "distinct words:", counts.size()
18    println "the appears", counts.get("the"), "times"
19    println "cat appears", counts.get("cat"), "times"
20    words.free()
21    counts.free()
22 end
```

```
distinct words: 6
the appears 3 times
cat appears 2 times
```

Split the sentence into words, then for each word either start its count at 1 or add one to its existing count, using `has` to tell the two cases apart. The `HashMap` does the heavy lifting — instant lookup by word — and the result is a tally of the text. This “count occurrences” pattern appears everywhere, from analytics to spell-checkers.

17.8 Chapter summary

- `HashMap<K, V>` maps keys to values: `insert`, `get`, `has`, `remove`, `size`, `free`.
- `Stack<T>` (push/pop/peek, last-in-first-out) and `Queue<T>` (enqueue/dequeue, first-in-first-out) order their elements; both `free`.
- `io.WriteFile`, `io.ReadFile`, and `io.Lines` handle whole files; `os` provides `Exists`, `Remove`, `Mkdir`, `ListDir`, `Args`, `Env`, `Exit`.
- `json` reads JSON: `Valid`, `Get`, `GetInt`, `Has`, `Escape`, `Minify`.
- `regex` matches patterns: `MatchStr`, `FindStr`, `ReplaceStr`, or the compiled `Compile/Match/Find`.
- `csv`, `encoding`, and the networking modules round out the library, all following the same import-and-qualify pattern.

17.9 Exercises

Exercise 17.1. B

Build a `HashMap<str, i32>` of three people’s ages, then look up and print one age and test for a name that is not present.

Exercise 17.2. U

Use a `Stack` to reverse the order of the numbers 1 to 5: push them all, then pop them into a line of output.

Exercise 17.3. W

Write a program that writes three lines to a file with `io.WriteFile`, reads them back with `io.Lines`, and prints how many lines there were.

Exercise 17.4. G

iven the JSON `{ \"title\": \"Salam\", \"pages\": 400 }`, print the title and the page count.

Exercise 17.5. U

se `regex.MatchStr` to test whether a string looks like it contains a year (four digits in a row).

Exercise 17.6. E

xtend the word-frequency example to also print the word “the” count only if it appears more than twice.

Chapter 18

Memory Management

Every value your program works with lives somewhere in the computer's memory. Most of the time Salam handles that for you and you need not think about it. But the growable containers — vectors, hash maps — and a few advanced techniques do involve memory you are responsible for. This chapter explains Salam's memory model plainly, so you know which memory takes care of itself and which you must tend.

18.1 Two kinds of memory

It helps to picture memory in two parts. The **stack** holds your local variables — the `i32`s, the `bool`s, the fixed-size arrays and structs. This memory is managed completely automatically: when a variable comes into scope it is created, and when its scope ends it is reclaimed. You have been relying on this since Chapter 2 without a second thought, and you can continue to.

The **heap** holds things whose size is not known until the program runs, or that must outlive the function that created them — the growable buffer behind a vector, the entries of a hash map. Heap memory must be explicitly released when no longer needed, which is why those containers have a `free` method.

18.2 Values that manage themselves

The good news first: the great majority of what you write needs no memory management at all.

- **Scalars** — `i32`, `f64`, `bool`, `char`, an enum — live on the stack and vanish

automatically.

- **Fixed-size arrays and structs** live inline, on the stack, and are likewise reclaimed automatically when they go out of scope. Recall from Chapter 9 that assigning or passing a struct *copies* it — there is no shared hidden allocation to worry about.
- **Strings** are special: although a `str` points at heap bytes, Salam manages those bytes for you (see the box below). You never `free` a string.

In Depth: the string arena (optional, for advanced readers)

Joining strings with `+`, or calling functions like `str.Upper`, must allocate room for the new text on the heap — yet you never free a string. How? Salam keeps a per-program *arena* for string data: all the bytes that string operations produce are allocated from it, and the whole arena is released at once when the program ends. The trade-off is deliberate: you get the convenience of never managing string memory, at the cost of strings living until the program exits. For the short-lived, string-heavy programs that are most common, this is exactly the right bargain.

18.3 Containers you must free

The exception to “it manages itself” is the growable containers. A `Vector<T>` or a `HashMap<K, V>` owns a heap buffer that it resizes as it grows, and that buffer must be returned to the system when you are finished. So every growable container has a `free` method, and you should call it:

```

1 func main:
2     mut v Vector<i32> = Vector {}
3     v.push(1)
4     v.push(2)
5     v.push(3)
6     println "sum of three pushes:", v.get(0)[0] + v.get(1)[0] + v.get
7         (2)[0]
8     v.free()           // release the buffer
9 end

```

```
sum of three pushes: 6
```

This applies to `Vector`, `HashMap`, `Stack`, `Queue`, and the other growable collections. The fixed-size array `i32[5]`, by contrast, needs no freeing — it lives on the stack.

18.3.1 Pairing creation with free using defer

The cleanest way never to forget a `free` is to schedule it the moment you create the container, with the `defer` you learned in Chapter 14. Written together on adjacent lines, the release is guaranteed and impossible to overlook:

```

1 func sum_first_n(n i32) i32:
2     mut v Vector<i32> = Vector {}
3     defer v.free()           // guaranteed cleanup, however we
4                               // return
5     for mut i = 1; i <= n; i = i + 1:
6         v.push(i)
7     end
8     mut total i32 = 0
9     mut j i32 = 0
10    while j < v.len():
11        total += v.get(j)[0]
12        j += 1
13    end
14    ret total
15 end
16
17 func main:
18     println sum_first_n(5)
19 end

```

15

Because `defer v.free()` runs whenever `sum_first_n` returns — through the normal path or any early `ret` — the vector’s memory is always released. This “create, defer free, use” pattern is the single most important habit for leak-free Salam code.

Watch out

Two mistakes to avoid. **Forgetting to free** a container leaks its memory — harmless in a tiny script that exits immediately, but a real problem in a long-running program that creates containers in a loop. **Using a container after freeing it** is worse: the buffer is gone, and reading it is undefined behaviour. Free a container exactly once, as the last thing you do with it — which is precisely what `defer` at the point of creation guarantees.

18.4 No hidden ownership rules

If you have heard of languages with elaborate “ownership” and “borrowing” systems, you can relax: Salam has none of that ceremony. There are no lifetime annotations to write, no borrow checker to satisfy, no move semantics to learn. The model is the simple one above — stack values manage themselves, growable containers get a `free` — and the discipline is the single habit of pairing each container with a `defer free`.

This simplicity is a deliberate design choice. It keeps the language approachable and the code readable, at the cost of asking you to remember `free` for the handful of containers that need it. In practice that is a small, learnable responsibility, and `defer` reduces it to a reflex.

18.5 Manual memory with `mem`

For most programs the containers are all the heap memory you will ever touch. But when you need raw control — implementing your own data structure, or working at the boundary with C (Chapter 19) — the `mem` module gives you direct allocation, the same primitives the standard library’s containers are built on:

```
1 import "mem"
2
3 func main:
4     p void* = mem.Allocate(64 as u64)    // ask for 64 bytes
5     if p == null:
6         println "allocation failed"
7         ret
8     end
9     println "got 64 bytes"
10    mem.Free(p)                          // hand them back
11    println "released"
12 end
```

```
got 64 bytes
released
```

`mem.Allocate(n)` requests `n` bytes and returns a `void*` (a raw, untyped pointer), or `null` if the request fails — which you should check. `mem.Free(p)` returns the memory. The module also offers `mem.Reallocate` to resize a block, `mem.Set` to fill bytes, and `mem.Copy` to copy them.

Watch out

Manual memory is powerful and unforgiving: a missing `Free` leaks, a double `Free` corrupts, and using freed memory crashes. You will rarely need `mem` directly — the built-in containers cover almost every case. Reach for it only when you are building something the standard library does not provide, and when you do, guard every allocation with a `defer mem.Free` just as you would a container.

In Depth: tracking down leaks (optional, for advanced readers)

Because Salam compiles to C, the usual C tooling works on Salam programs. The compiler can build with AddressSanitizer (the `--asan` flag, or the `salam memcheck` command) to catch use-after-free and out-of-bounds errors at run time, and the `mem` module exposes counters — allocations made, frees done, bytes still live — that let a program audit its own memory use. If you ever suspect a leak, these turn “something is wrong” into a precise location.

18.6 Chapter summary

- Scalars, fixed arrays, and structs live on the stack and are reclaimed automatically; you never manage them.
- Strings are heap-backed but managed by a per-program arena — never `free` a string.
- Growable containers (`Vector`, `HashMap`, `Stack`, `Queue`) own heap buffers and must be freed.
- Pair every container creation with `defer x.free()` to guarantee cleanup on every exit path.
- Salam has no ownership/borrowing system — the model is deliberately simple.
- `mem` (`Allocate`, `Free`, `Reallocate`, `Set`, `Copy`) provides raw manual memory for advanced use; guard it carefully.

18.7 Exercises

Exercise 18.1. W

rite a function that builds a `Vector<str>` of three greetings, prints them, and frees the vector using `defer`.

Exercise 18.2. T

ake a function that creates a `HashMap`, returns early on some condition, and uses `defer` to ensure the map is freed on both paths.

Exercise 18.3. E

xplain in a comment why a fixed array `i32[10]` needs no `free` but a `Vector<i32>` does.

Exercise 18.4. U

se `mem.Allocate` and `mem.Free` to grab and release 128 bytes, checking for a null result, and guard the free with `defer`.

Exercise 18.5. D

escribe, in your own words, what the string arena does and one situation where its “strings live until the program ends” behaviour could matter.

Chapter 19

The Foreign Function Interface

Salam compiles to C, which gives it a superpower: it can call C functions directly, with no glue code and no performance penalty. The decades of C libraries the world runs on — math, compression, databases, graphics, the operating system itself — are all reachable from Salam through the **foreign function interface**, or FFI. This chapter shows how to declare a C function and call it as if it were your own.

19.1 Declaring a C function

To call a C function you first tell Salam about it: its name, its parameters, and its return type. You do this in an `extern "C"` block, which lists the *signatures* of C functions without bodies — the bodies live in C, after all:

```
1 extern "C":
2     func printf(format str, ...) i32
3     func strlen(s str) u64
4 end
5
6 func main:
7     printf("Hello from C's printf!\n")
8     msg str = "external"
9     printf("the length of \"%s\" is %d\n", msg, strlen(msg))
10 end
```

```
Hello from C's printf!
the length of "external" is 8
```

Inside the `extern "C"` block, each `func` line is a *declaration*: it says “a C function with this name and shape exists; let me call it.” There is no body. When you call `printf` or `strlen` from Salam, the compiler emits a direct call to the real C function — exactly as a C program would. The trailing `...` on `printf` marks it **variadic** (it accepts a variable number of arguments), which is how `printf` takes its format arguments.

19.2 How Salam types map to C

For an FFI call to work, Salam’s types must line up with C’s. The mapping is direct and is why the integration is seamless:

Salam	C	Notes
i32	int32_t	a 32-bit signed integer
i64	int64_t	a 64-bit signed integer
u64	uint64_t	a 64-bit unsigned integer
f64	double	a 64-bit float
f32	float	a 32-bit float
bool	one byte	
char	one byte	
str	const char*	a NUL-terminated C string
void*	void*	an untyped pointer
(no return type)	void	a procedure

When you declare an `extern` function, choose the Salam types that match the C function’s real signature according to this table. `strlen` in C takes a `const char*` and returns a `size_t`, so we declared it as `func strlen(s str) u64`.

19.3 Calling the C math library

A practical example: C’s math library (`libm`) has fast, accurate implementations of functions like `sqrt`, `pow`, and `hypot`. Declare the ones you want and call them:

```

1 extern "C":
2   func printf(format str, ...) i32
3   func sqrt(x f64) f64
4   func pow(base f64, exp f64) f64
5   func hypot(x f64, y f64) f64

```

```
6 end
7
8 func main:
9     printf("sqrt(2)    = %.6f\n", sqrt(2.0))
10    printf("2^10      = %.0f\n", pow(2.0, 10.0))
11    printf("hypot(3,4) = %.1f\n", hypot(3.0, 4.0))
12 end
```

```
sqrt(2)    = 1.414214
2^10       = 1024
hypot(3,4) = 5.0
```

Each declared function is called like any Salam function, and the result comes straight back from the C implementation. (Of course, for these particular functions you would normally just use Salam's own `math` module — which is itself partly built on `libm` — but the same technique works for *any* C library, including ones with no Salam wrapper.)

19.4 Linking external libraries

The functions above — `printf`, `sqrt` — come from the C standard library, which is always linked in, so declaring them is enough. To use a function from a *separate* library, you must also tell the compiler to link that library, with a link directive before the `extern` block:

```
1 link "sqlite3"
2
3 extern "C":
4     func sqlite3_libversion() str
5     func printf(format str, ...) i32
6 end
7
8 func main:
9     printf("SQLite version: %s\n", sqlite3_libversion())
10 end
```

`link "sqlite3"` instructs the compiler to link against the SQLite library (the equivalent of passing `-lsqlite3` to a C compiler), making its functions available to your `extern` declarations. You can link several libraries with several `link` lines.

In Depth: platform-conditional linking (optional, for advanced readers)

Library names sometimes differ across operating systems. Salam's pre-processor lets you choose the right one at build time with `@if`:

```
1 @if SALAM_OS_WINDOWS
2 link "winsqlite3"
3 @else
4 link "sqlite3"
5 @end
```

The symbols `SALAM_OS_WINDOWS`, `SALAM_OS_LINUX`, and `SALAM_OS_MACOS` let a single program link the correct library on each platform — the same technique C programmers use with `#ifdef`, expressed in Salam.

19.5 Working with pointers across the boundary

Many C functions deal in pointers — `malloc` returns one, `free` takes one. Salam's `void*` type matches C's, so you can call them directly:

```
1 extern "C":
2   func malloc(size u64) void*
3   func free(ptr void*)
4   func puts(s str) i32
5 end
6
7 func main:
8   mut p void* = malloc(64 as u64)
9   if p == null:
10     puts("malloc failed")
11   else:
12     puts("malloc(64) gave a non-null pointer")
13     free(p)
14   end
15 end
```

```
malloc(64) gave a non-null pointer
```

Notice that the safe habits from Chapter 18 carry straight over the boundary: check the pointer against `null`, and `free` what you allocate. The FFI does not add safety of its own — you are calling C, with C's rules — so the discipline is yours to keep.

19.6 When to use the FFI

The FFI is a bridge to be crossed deliberately, not a place to live. Reach for it when:

- you need a capability that exists as a mature C library — a database driver, a compression codec, an image format — and rewriting it in Salam would be wasteful;
- you must call an operating-system function the standard library does not wrap;
- you are integrating Salam into an existing C codebase.

For everything else, prefer Salam's own standard library and types: they are safer (the compiler checks them), more convenient, and bilingual. The FFI is the escape hatch for when you genuinely need the wider C world — powerful exactly because it is there when you need it and ignorable when you do not.

Watch out

Code reached through the FFI is outside Salam's safety net. The compiler trusts your `extern` declarations: if you declare a function with the wrong types, or pass a bad pointer, the result is a crash or corruption, not a friendly compile-time error. Declare each C function to match its real signature exactly, and treat values that cross the boundary with the same care a C programmer would.

19.7 Chapter summary

- An `extern "C":` block declares C functions (name, parameters, return type, no body) so Salam can call them directly.
- Salam types map one-to-one onto C types (`i32`→`int32_t`, `str`→`const char*`, `void*`→`void*`, ...); declare each function to match.
- A trailing `...` marks a variadic function such as `printf`.
- Standard-library C functions need only declaration; others need a `link "lib"` directive (optionally guarded by `@if` for per-platform names).
- The FFI adds no safety — check pointers, free what you allocate, and declare signatures exactly.

- Use the FFI to reach mature C libraries and OS functions; prefer Salam's own library otherwise.

19.8 Exercises

Exercise 19.1. D

Declare C's `puts` (func `puts(s str) i32`) and use it to print two lines.

Exercise 19.2. D

Declare `sqrt` and `pow` from the `math` library and print the square root of 2 and 2 to the 16th power.

Exercise 19.3. D

Declare `getenv(name str) str` and print the value of an environment variable of your choice.

Exercise 19.4. D

Declare `malloc` and `free`, allocate 256 bytes, check for `null`, and free them — guarding the free with `defer`.

Exercise 19.5. E

Explain in a comment why an incorrect `extern` declaration cannot be caught by the Salam compiler, and what discipline protects you instead.

Chapter 20

Concurrency

So far our programs have done one thing at a time, in order. **Concurrency** is doing several things at once — splitting work across the multiple cores every modern computer has, so a big job finishes sooner. Salam expresses concurrency with *threads*, and protects shared data with *synchronization*. This chapter introduces both, along with the care they demand.

20.1 Threads: running code in parallel

A **thread** is an independent strand of execution. Your program already has one — the one running **main**. With **spawn** you can start another, which runs a function of your choosing *alongside* the rest of the program:

```
1 func greet():  
2     println "hello from a thread"  
3 end  
4  
5 func main:  
6     handle i64 = spawn(greet)  
7     println "hello from main"  
8     join(handle)  
9     println "the thread has finished"  
10 end
```

spawn(greet) starts **greet** running on a new thread and immediately returns a **handle** — an **i64** that identifies the thread — while **main** carries on. Because the two run concurrently, the order of the first two lines of output is not guaranteed. **join(handle)** then *waits* for the thread to finish before continuing, so the final line always prints last.

Note

`spawn` takes a function with no parameters. To give a thread something to work on, have it read shared variables (carefully — see below) or use a closure that captures the data it needs. `join` is how you know a thread is done; a program should `join` every thread it spawns before relying on that thread's results.

20.2 The danger of shared data

Threads become useful — and dangerous — the moment they touch the *same* data. Suppose two threads each add to a shared counter a thousand times. You would expect the total to climb by two thousand. But incrementing a counter is really three steps under the hood — read the value, add one, write it back — and if two threads interleave those steps, they can read the same old value and each write back the same new one, losing an increment. This is a **race condition**: the result depends on the unpredictable timing of the threads, and it is one of the hardest kinds of bug to find, because it appears only sometimes.

20.3 Mutexes: one at a time

The fix is to ensure that only one thread touches the shared data at a time. A **mutex** (from “mutual exclusion”) is a lock that exactly one thread can hold. A thread *locks* it before touching the shared data and *unlocks* it afterwards; any other thread that tries to lock it must wait until it is free. The `sync` module provides them:

```
1 import "sync"
2
3 mut counter i32 = 0
4 mut lock sync.Mutex = sync.NewMutex()
5
6 func work():
7     mut i i32 = 0
8     while i < 1000:
9         sync.Lock(lock)
10        counter = counter + 1    // protected: only one thread here
11        at a time
12        sync.Unlock(lock)
13        i += 1
14    end
```

```
14 end
15
16 func main:
17     t1 i64 = spawn(work)
18     t2 i64 = spawn(work)
19     join(t1)
20     join(t2)
21     println "counter:", counter
22     sync.Destroy(lock)
23 end
```

```
counter: 2000
```

Now the answer is reliably 2000. Between `sync.Lock(lock)` and `sync.Unlock(lock)` — the *critical section* — only one thread ever runs, so the read-add-write happens without interference. Create a mutex with `sync.NewMutex()`, and call `sync.Destroy` on it when all the threads are done.

Watch out

A mutex protects data only if *every* access to that data goes through it. One thread that touches the shared counter without locking re-introduces the race. And always pair a `Lock` with an `Unlock` on every path — a lock that is never released leaves other threads waiting forever (a *deadlock*). Keep critical sections short and make the lock/unlock pairing obvious.

20.4 Waiting for many threads

When you spawn several threads and need to wait for all of them, you can collect their handles and `join` each. For larger fan-outs, the `sync` module also offers a **wait group** — a counter you raise as you spawn work and that threads decrement as they finish, letting the main thread block until the count reaches zero:

```
1 import "sync"
2
3 mut wg sync.WaitGroup = sync.NewWaitGroup()
4
5 func task():
6     println "working"
7     sync.Done(wg)           // signal completion
8 end
```

```
9
10 func main:
11     sync.Add(wg, 3 as i64)
12     spawn(task)
13     spawn(task)
14     spawn(task)
15     sync.Wait(wg)           // block until all three call Done
16     println "all tasks complete"
17     sync.DestroyWaitGroup(wg)
18 end
```

`sync.Add(wg, n)` announces `n` tasks; each task calls `sync.Done` when finished; `sync.Wait` blocks until every `Done` has arrived. A wait group is convenient when the number of workers is known but you do not want to track each handle by hand.

20.5 When concurrency pays — and when it costs

Concurrency is not free. Threads have a setup cost, locks add overhead, and — most of all — concurrent code is harder to reason about and harder to debug than sequential code. It pays off when there is genuine parallel work to do: a big computation you can split into independent chunks, or many slow operations (such as network requests) that can wait at the same time instead of one after another.

Tip

Reach for concurrency only when a measured need justifies it, and keep shared, mutable state to an absolute minimum — the less data threads share, the fewer locks you need and the fewer races can occur. The simplest concurrent designs give each thread its own data to work on and combine the results at the end, touching shared state only briefly and under a lock. Start sequential; add threads when the workload truly calls for them.

In Depth: how threads map to the system (optional, for advanced readers)

Salam's `spawn` creates a real operating-system thread (a `pthread` on Unix-like systems, a Windows thread on Windows), so threads run on separate cores and your program can genuinely use a multi-core machine. The `sync.Mutex` is a thin wrapper over the platform's native mutex. Because

these are true OS threads, the costs and the rules are the same as in C — which is why the discipline around shared data matters, and why the standard library gives you the classic tools (mutexes, wait groups) to manage it.

20.6 Chapter summary

- `spawn(f)` runs the parameterless function `f` on a new thread and returns an `i64` handle; `join(handle)` waits for it to finish.
- Sharing mutable data between threads risks a *race condition*, where the result depends on timing.
- A `sync.Mutex` (`NewMutex`, `Lock`, `Unlock`, `Destroy`) ensures only one thread is in a critical section at a time.
- Every access to shared data must go through the lock, and every `Lock` must be matched by an `Unlock`.
- A `sync.WaitGroup` (`Add`, `Done`, `Wait`) waits for a known number of threads to complete.
- Use concurrency only when real parallel work justifies it, and minimise shared mutable state.

20.7 Exercises

Exercise 20.1. S

spawn a thread that prints the numbers 1 to 5 while `main` prints the letters A to E. Join the thread, and observe that the interleaving varies between runs.

Exercise 20.2. M

Modify the shared-counter example to use *four* threads of 1000 increments each and confirm the total is 4000.

Exercise 20.3. D

deliberately remove the `Lock/Unlock` from the counter example, run it several times, and describe what you observe about the final total.

Exercise 20.4. U

se a `WaitGroup` to run five tasks that each print their own number, and wait for all of them before printing “done”.

Exercise 20.5. E

xplain in a comment why keeping the critical section (the code between `Lock` and `Unlock`) short is good practice.

Chapter 21

Testing and Debugging

Code that is never tested is code you only *hope* works. Tests turn hope into evidence: small programs that check your code does what you claim, and keep checking as you change it. And when something is wrong, debugging is how you find out why. This chapter covers both — writing tests with the `testing` module, and the tools and habits that make finding bugs faster.

21.1 Why test?

A test is a tiny experiment: “if I call this function with these inputs, I should get this result.” Writing tests has three payoffs. They catch mistakes immediately, while the code is fresh in your mind. They guard against *regressions* — a change that fixes one thing while quietly breaking another — because you can re-run every test after every change. And they document how your code is meant to be used, in the most honest form possible: working examples.

21.2 Assertions with the `testing` module

The `testing` module provides **assertions** — statements that check a condition and record a pass or a failure. You write a normal program that calls a function and asserts the expected result:

```
1 import "testing"
2
3 func add(a i32, b i32) i32:
4     ret a + b
5 end
```

```

6
7 func main:
8     testing.AssertEqInt(add(2, 3), 5)
9     testing.AssertEqInt(add(-1, 1), 0)
10    testing.AssertTrue(add(2, 2) == 4)
11    testing.Summary()
12 end

```

```
tests: 3 passed, 0 failed
```

Each `Assert` checks its condition and tallies the result; `testing.Summary()` prints the totals at the end. When everything passes, you get a clean report like the one above — quiet confidence that your code does what you said.

21.2.1 The assertion functions

Assertion	Passes when
<code>testing.AssertTrue(c)</code>	<code>c</code> is true
<code>testing.AssertFalse(c)</code>	<code>c</code> is false
<code>testing.AssertEqInt(got, want)</code>	two integers are equal
<code>testing.AssertEqStr(got, want)</code>	two strings are equal
<code>testing.AssertEqBool(got, want)</code>	two booleans are equal
<code>testing.Assert(c)</code>	<code>c</code> is true (general)

The `...Eq...` forms are worth preferring over a plain `AssertTrue(a == b)` because, when they fail, they tell you *both* values — what you got and what you wanted — which is exactly what you need to diagnose the problem.

21.2.2 Reading a failure

A failing assertion does not stop the program; it records the failure, prints a message, and lets the remaining checks run, so one run tells you about *all* the problems. Watch what happens when an expected value is wrong:

```

1 import "testing"
2
3 func add(a i32, b i32) i32:
4     ret a + b
5 end
6
7 func main:
8     testing.AssertEqInt(add(2, 3), 5)

```

```
9     testing.AssertEqInt(add(2, 2), 5)    // wrong on purpose
10     testing.Summary()
11 end
```

```
FAIL: ints differ: got 4 want 5
tests: 1 passed, 1 failed
```

The message names exactly what went wrong — `got 4 want 5` — and the summary counts one pass and one failure. In a real test file you would fix the code (or the expectation) until the summary reads all passes.

21.3 Organising tests

Group related checks into named functions, then call them from `main`. This keeps each function focused on one piece of behaviour and makes the output easier to follow:

```
1 import "testing"
2
3 func is_even(n i32) bool:
4     ret n % 2 == 0
5 end
6
7 func test_is_even():
8     testing.AssertTrue(is_even(4))
9     testing.AssertFalse(is_even(7))
10    testing.AssertTrue(is_even(0))
11 end
12
13 func main:
14     test_is_even()
15     testing.Summary()
16 end
```

```
tests: 3 passed, 0 failed
```

As your program grows, keep test functions next to the code they exercise, give them clear `test_` names, and call them all from a single `main` that ends in `testing.Summary()`. The exit code reflects the number of failures, so a test program also slots neatly into an automated build that should stop when a test fails.

Tip

Write a test the moment you find a bug, *before* you fix it: a test that reproduces the bug. Then fix the code until the test passes. Now you have not only solved the problem but guaranteed it can never silently come back — the test will catch it if it does. This habit, fixing bugs “test first,” steadily builds a net that makes your whole program safer to change.

21.4 Debugging: finding out why

When a program misbehaves, the goal is to replace guessing with *seeing*. The most powerful debugging tool is also the humblest.

21.4.1 Print debugging

A well-placed `println` that shows a variable’s value at a key moment will solve the large majority of bugs. When a result is wrong, print the inputs and intermediate values along the path that produces it, and compare what you see with what you expected. The discrepancy is almost always where the bug lives:

```
1 func average(a i32[3]) f64:
2   mut sum i32 = 0
3   for mut i = 0; i < 3; i = i + 1:
4     sum += a[i]
5     println "after adding", a[i], "sum is", sum    // a temporary
6     probe
7   end
8   ret (sum as f64) / 3.0
9 end
10 func main:
11   data i32[3] = [10, 20, 30]
12   println "average:", average(data)
13 end
```

```
after adding 10 sum is 10
after adding 20 sum is 30
after adding 30 sum is 60
average: 20
```

The probe lines confirm the running total is building correctly. Once you have found and fixed the problem, remove the temporary prints — they were scaffolding, not part of the finished program.

21.4.2 Let the compiler help

Many bugs never reach run time at all: Salam’s static checks catch them first. A type mismatch, an immutable variable you tried to reassign, an unused parameter, an undefined name — each is reported at compile time with a clear message and the line it occurred on. Read those messages carefully; they are the cheapest bug reports you will ever get, arriving before the program even runs. Much of what would be a debugging session in a dynamic language is, in Salam, simply a compiler error you fix on the spot.

21.4.3 Heavier tools

For the stubborn cases, the compiler offers more. Building with debug information (`salam debug`) lets you step through your program in a debugger, inspecting variables as it runs. And because Salam compiles to C, the memory checker (`salam memcheck`, which builds with AddressSanitizer) pinpoints use-after-free and out-of-bounds errors — invaluable when you have been working with manual memory or the FFI. You will not need these often, but they are there when a print statement is not enough.

In Depth: a debugging method (optional, for advanced readers)

When a bug resists, work like a scientist rather than thrashing. **Reproduce** it reliably — find the smallest input that triggers it. **Localise** it by bisection: confirm the data is right at the start and wrong at the end, then check the middle, narrowing the region until you find the line where right becomes wrong. **Form a hypothesis** about the cause, and **test** it with a targeted print or a change. The smallest reproducing case plus bisection will corner almost any bug, far faster than staring at the code hoping for inspiration.

21.5 Chapter summary

- Tests turn “I hope it works” into evidence and guard against regressions.
- The `testing` module asserts conditions (`AssertTrue`, `AssertEqInt`, `AssertEqStr`, ...) and reports totals with `Summary`; failures print both values

and do not halt the run.

- Group checks into `test_`-named functions called from `main`.
- Write a failing test that reproduces a bug before fixing it.
- `println` probes solve most bugs; remove them once done.
- Salam's compile-time checks catch many bugs before run time — read the messages.
- `salam debug` and `salam memcheck` handle the hard cases.

21.6 Exercises

Exercise 21.1. W

rite a function `factorial(n i32) i32` and a `test_factorial` that asserts `factorial(0) == 1`, `factorial(5) == 120`, and one more case of your choosing.

Exercise 21.2. A

dd a deliberately wrong assertion to a passing test file and study the failure message and the summary line.

Exercise 21.3. T

ake a buggy function (for example, a loop with an off-by-one error) and use `println` probes to locate the exact line where the values go wrong.

Exercise 21.4. W

rite tests for a `clamp(x, lo, hi)` function covering a value below the range, inside it, and above it.

Exercise 21.5. D

escribe, in a comment, the difference between a bug Salam catches at compile time and one you would only find by running tests, with an example of each.

Chapter 22

Bilingual Programming in Salam

We have mentioned throughout this book that Salam can be written in Persian as well as English. Now we give that feature the chapter it deserves. Salam’s bilingualism is not a translation layer bolted on top — it is built into the language: the very same compiler reads English and Persian keywords, and produces identical programs from either. This chapter shows how, and why it matters.

22.1 One language, two vocabularies

The key idea is simple. A programming language is made of *keywords* (the fixed words like `func`, `if`, `while`) and *your* names (the variables and functions you invent). Salam keeps the grammar — the structure, the rules, the meaning — exactly the same in both languages, and changes only the spelling of the keywords. `func` becomes تابع; `if` becomes اگر; everything else about the program is unchanged.

Here is the same program twice. First in English:

```
1 func add(a i32, b i32) i32:
2     ret a + b
3 end
4
5 func main:
6     mut total i32 = 0
7     for mut i = 1; i <= 5; i = i + 1:
8         total = total + i
9     end
```

```

10 println "sum 1..5 =", total
11 end

```

And now, exactly the same program, in Persian:

```

1 تابع جمع (الف صحیح ۳۲, ب صحیح ۳۲) صحیح ۳۲:
2 بازگشت الف + ب
3 پایان
4
5 تابع اصلی:
6 متغیر مجموع صحیح ۳۲ = 0
7 برای متغیر i = 1 تا 5; i = i + 1:
8 مجموع = مجموع + i
9 پایان
10 چاپ "مجموع ۱ تا ۵ =", مجموع
11 پایان

```

```

مجموع
۱ تا ۵ = 15

```

Look closely and you will see the structures are identical, line for line. Only the keywords and the type names changed. A Persian-speaking student reads the second version as naturally as you read the first — and they are learning exactly the same concepts, because they *are* the same concepts.

22.2 The keyword equivalents

Every keyword you have learned has a Persian twin. Here are the most common; the complete list is in Appendix A.

English	Persian	English	Persian
func	تابع	ret	بازگشت
if	اگر	else	وگرنه
while	تاوقتی	for	برای
repeat	تکرار	to	تا
mut	متغیر	const	ثابت
struct	ساختار	enum	شمارش
interface	واسط	end	پایان
import	وارد کردن	pub	عمومی
true	درست	false	نادرست
break	بشکن	continue	ادامه
println	چاپ	print	بنویس
main	اصلی	this	این

The type names have Persian forms too, which you first saw in Chapter 2:

English	Persian	English	Persian
i32	صحیح ۳۲	i64	صحیح ۶۴
f64	اعشاری ۶۴	bool	منطقی
str	رشته	char	کاراکتر
auto	خودکار		

Note

The entry point itself is part of the vocabulary: it is `main` in English and `اصلی` in Persian. When you run a Persian program, the compiler looks for `اصلی` as the starting point, just as it looks for `main` in an English one.

22.3 Compiling a Persian program

A Salam source file is written in *one* language, and you tell the compiler which with the `--lang` flag. English is the default; for a Persian file, pass `--lang=fa`:

```
salam run hello.salam --lang=fa
salam build hello.salam --lang=fa --output=hello
```

Everything else about the workflow — `run`, `build`, `exec`, `fmt` — is identical. The flag simply tells the compiler which keyword vocabulary to expect.

Watch out

A single file uses a single language: you cannot mix English and Persian *keywords* in the same file. A file is either an English program or a Persian one, compiled with the matching `--lang`. This is deliberate — it keeps each file coherent and unambiguous. What you *can* freely mix is covered next.

22.4 What you can mix

The keyword rule is about keywords only. The *names you choose* — variables, functions, struct fields — may be in whatever language you like, in either kind of file, because Salam identifiers may contain Unicode letters. So a Persian program can use English identifiers (as the loop variable `i` did above), and an English program can use Persian identifiers:

```

1 func main:
2     naam str = "Sara"           // a Latin-letter Persian word as a name
3     println "Hello,", naam
4 end

```

Hello, Sara

Likewise, *strings* and *comments* are free text and may be in any language regardless of the file's keyword language — which is why the Persian program above could print the Persian message "مجموع ۱ تا ۵ =" and an English program can hold Persian comments. Keywords pick a side; your content does not have to.

Note

Notice the digits. Inside the type name صحیح ۳۲ the digits are Persian (۳۲), but arithmetic values such as 15 or loop bounds such as 5 are written with the ordinary digits 0–9. The Persian digits are part of the *spelling* of certain keywords; computation uses the standard digits.

22.5 Why bilingualism matters

This feature is more than a novelty. For a learner whose first language is Persian, writing اگر instead of `if` removes a layer of translation between thought and code — the program reads the way they think. A classroom can teach loops and functions without first teaching English. And because the

structure is identical, the day that learner needs to work in English — to use an English library, join an international team, read English documentation — the transition is trivial: they already know every concept, and only the spellings change.

Tip

If you are learning, start in whichever language is more comfortable, and once a concept is solid, try rewriting a program in the other language. Because only the keywords differ, this is quick — and it cements both the idea *and* its English vocabulary at once, which is exactly the bridge Salam was built to be.

In Depth: how the compiler reads two languages (optional, for advanced readers)

Internally, Salam keeps a *language pack*: a table mapping each keyword to its spellings in each supported language. When the lexer (the first stage of the compiler, from Chapter 1) reads your source, it consults the pack for the language you selected and recognises تابع or func as the very same internal token. From that point on — parsing, type-checking, code generation — the compiler has no idea which language the source was written in, because the difference has already been erased. That is why the two versions produce byte-for-byte identical programs: after the first stage, there is only one language left, the language of meaning.

22.6 Chapter summary

- Salam’s grammar is identical in English and Persian; only the spelling of keywords and type names differs.
- Every keyword has a Persian twin (func/تابع, if/اگر, ...); the entry point is main / اصلی. The full table is in Appendix A.
- Compile a Persian file with `--lang=fa`; English is the default. The rest of the workflow is unchanged.
- A file uses one keyword language, but identifiers, strings, and comments may be in any language.
- Persian digits appear inside certain keyword spellings; computation uses ordinary digits.

- Bilingualism lets learners code in their own language and cross to English effortlessly, because the concepts are identical.

22.7 Exercises

Exercise 22.1. T

Take any English program from an earlier chapter and rewrite it in Persian, then run it with `--lang=fa`. Confirm the output matches.

Exercise 22.2. W

Write a Persian program with a function (تابع) that returns (بازگشت) the larger of two numbers, using اگر.

Exercise 22.3. I

In an English program, use a Persian word (in Latin letters, like `naam`) as a variable name and a Persian sentence as a printed string.

Exercise 22.4. W

Write the same “count to ten” loop twice — once in English, once in Persian — and place them side by side to see how only the keywords change.

Exercise 22.5. E

Explain in a comment why two versions of a program, one English and one Persian, compile to identical results.

Chapter 23

Advanced Topics and Best Practices

You now know the whole language: types, control flow, functions, data structures, generics, interfaces, closures, the standard library, concurrency, and the FFI. This final chapter steps back from features to *judgement* — how to use what you know well. It gathers advice on performance, structure, style, and the pitfalls worth knowing, so that the programs you write next are not just correct but good.

23.1 Performance: fast by default, faster with care

Salam programs are fast because they compile to C and run as native code, with static types and no hidden boxing. For most programs that is the end of the performance story — you get speed for free. When you do need to squeeze more, a few principles matter far more than micro-tricks:

- **Choose the right data structure.** An algorithm that looks up by key thousands of times wants a `HashMap`, not a linear scan through a `Vector`. The biggest speed wins come from structure, not from shaving instructions.
- **Avoid needless allocation.** Building a string by concatenating in a tight loop allocates on every step. When you can, compute sizes up front or reuse a buffer. Reserve a vector's capacity if you know roughly how many elements it will hold.
- **Measure before optimising.** Intuition about what is slow is usually

wrong. Time the program, find the part that actually dominates, and improve that — not the part that merely *looks* expensive.

- **Prefer the compiled path.** `salam exec` (the interpreter) is perfect for learning and quick runs, but a `salam build` executable is far faster for real work.

In Depth: optimization levels and the LLVM backend (optional, for advanced readers)

Because the default backend emits C, the C compiler’s own optimizer does a great deal of work on your behalf — inlining small functions, unrolling loops, eliminating dead code. Salam also has an experimental LLVM backend (`salam llvm`) with explicit optimization levels (`-O0` through `-O3`, plus `-Os/-Oz` for size), which can target native code or WebAssembly. For everyday work the defaults are excellent; reach for the optimization flags only when a measurement tells you to.

23.2 Structuring a program

Small programs fit in one file; real ones do not. As a program grows, structure is what keeps it understandable:

- **Split by responsibility.** Put related types and functions together in a module (a file with a `package`), and keep each module focused on one area — parsing, storage, the user interface. A reader should be able to guess what is in a file from its name.
- **Keep public surfaces small.** Mark only what other modules truly need with `pub`; keep helpers private. The less a module exposes, the freer you are to change its insides (Chapter 15).
- **Let data own its rules.** Put the operations that protect a struct’s invariants in its methods, as the `Account` did in Chapter 9, so the rules live with the data they guard.
- **Depend on interfaces, not concrete types,** where you want flexibility (Chapter 11) — it lets you swap implementations without rewriting the code that uses them.

23.3 Coding style

Style is not decoration; it is how you make code readable to the next person — who is often you, months later. Salam even ships a formatter, `salam fmt`, that rewrites your code into the canonical layout automatically, so you never argue about spacing. Beyond formatting, a few habits carry most of the weight:

- **Name for meaning.** `user_count` and `is_valid` tell the reader what a value *is* and what a function *answers*. Single-letter names earn their place only as short-lived loop indices.
- **Prefer immutability.** Reach for `mut` only when a value must change (Chapter 2). Every binding that cannot change is one fewer thing to track.
- **Keep functions small and single-purpose.** If you cannot name a function for the one thing it does, it is doing too many things (Chapter 6).
- **Comment the *why*.** The code already shows *what* it does; a comment is most valuable when it records the reasoning the code cannot.
- **Handle the unhappy path.** Check the `bool` a function returns, the `Option` that might be empty, the pointer that might be `null`. The cases you ignore are the bugs you ship.

Tip

Run `salam fmt` before you save or share code, and let it settle every question of layout. Consistent formatting makes differences between versions show real changes, not reshuffled whitespace, and frees your attention for what the code actually does.

23.4 Common pitfalls, revisited

A consolidated list of the traps this book has flagged — worth a glance whenever something behaves unexpectedly:

- **Integer division truncates.** `7 / 2` is 3; use a float operand for a fractional result (Chapter 3).
- **Literals are `i32`.** Storing one in a smaller or unsigned type needs an explicit `as` cast (Chapter 2).

- **Off-by-one with indices.** Arrays run 0 to n-1, while repeat A to B *includes* B (Chapters 7 and 5).
- **Forgetting free.** Growable containers and manual allocations leak unless freed; pair each with a **defer** at creation (Chapter 18).
- **Unprotected shared state.** Two threads touching the same data without a lock race (Chapter 20).
- **Ignoring a result.** A `bool` or `Option` that signals failure is only useful if you check it (Chapter 14).
- **An = where you meant ==.** Assignment versus comparison (Chapter 3).

23.5 Where to go next

The best way to consolidate everything in this book is to build something real — something a little beyond what you are sure you can do. A few projects that exercise the whole language:

- A **command-line tool** that reads a file, processes it, and writes a result — exercising `io`, `str`, and the collections.
- A small **interpreter or calculator** that parses expressions — exercising `enums`, `structs`, `recursion`, and error handling.
- A **simulation or game** (a grid world, a card game) — exercising `structs`, `methods`, `arrays`, and `rand`.
- A **data tool** that reads JSON or CSV, tallies something with a `HashMap`, and reports — exercising the standard library end to end.

Read the standard library's own source, too: it is written in Salam, and seeing how `Vector` and `HashMap` are built is one of the best ways to deepen your understanding. And when you hit something the language does not yet do, remember the FFI — the whole world of C is one `extern` block away.

23.6 A closing word

You began this book, perhaps, never having written a line of code; you end it knowing a complete, modern programming language — one you can write in your own words. Programming is a craft, and like every craft it rewards

practice more than reading. So close the book and open an editor. Write a program that does something you care about. Make it work, then make it clear, then make it fast — in that order. Argue with the compiler until you both agree. That is the whole of the craft, and you now have every tool you need to practise it.

Salam, and happy coding.

23.7 Chapter summary

- Salam is fast by default; for more, choose good data structures, avoid needless allocation, and *measure* before optimising.
- Structure a program by responsibility, keep public surfaces small, and let data own its rules.
- Good style — meaningful names, immutability, small functions, “why” comments, handled error paths — is how code stays readable; `salam fmt` handles layout.
- Keep the list of common pitfalls in mind when something surprises you.
- Consolidate by building real projects and reading the standard library’s source.

23.8 Exercises

Exercise 23.1. T

Take a program you wrote for an earlier chapter and improve it: run `salam fmt` on it, rename anything unclear, and add a comment explaining one non-obvious decision.

Exercise 23.2. F

Find a function in one of your programs longer than a screenful and split it into smaller, well-named helpers.

Exercise 23.3. P

Pick one project from “Where to go next” and sketch its modules: what files, what `pub` functions, what data structures.

Exercise 23.4. R

Review one of your programs against the “common pitfalls” list and fix any you find.

Exercise 23.5. O

Open a standard-library module (such as `collections`) and read one of its types end to end. Write a paragraph on how it works.

Appendix A

Keyword Reference (English / Persian)

This appendix lists every Salam keyword in both languages, drawn directly from the compiler's language pack. A program is written in one language and compiled with the matching `--lang` flag (English is the default, `--lang=fa` for Persian); only the spellings below differ between the two — the meaning and grammar are identical.

Keywords

English	Persian	Meaning
<code>func</code>	تابع	define a function
<code>ret</code>	بازگشت	return from a function
<code>if</code>	اگر	conditional
<code>else</code>	وگرنه	alternative branch
<code>while</code>	تاوقتی	while loop
<code>for</code>	برای	C-style for loop
<code>repeat</code>	تکرار	repeat / range loop
<code>to</code>	تا	range upper bound (with <code>repeat</code>)
<code>every</code>	هر	loop step (with <code>repeat ... to</code>)
<code>break</code>	بشکن	exit the innermost loop
<code>continue</code>	ادامه	skip to the next iteration
<code>mut</code>	متغیر	mutable binding
<code>const</code>	ثابت	compile-time constant
<code>type</code>	نوع	type alias

English	Persian	Meaning
struct	ساختار	define a struct
enum	شمارش	define an enum
interface	واسط	define an interface
impl	پیاده‌سازی	implementation block
end	پایان	close a block
import	وارد کردن	import a module
package	بسته	declare a package
pub	عمومی	make a name public
as	بعنوان	type conversion
this	این	the current instance (in a method)
true	درست	boolean true
false	نادرست	boolean false
null	پوچ	null pointer
defer	تعلق	schedule cleanup at scope exit
extern	خارجی	declare external (C) functions
operator	عملگر	operator overload
component	مولفه	layout component
layout	صفحه	layout block (web DSL)
print	بنویس	print without a newline
println	چاپ	print with a newline
println	خطابنویس	print to standard error
println	خطاچاپ	print to standard error with a newline
input	بخوان	read a line of input
main	اصلی	the program's entry point

Note

print and println have the short aliases `_` and `__` in English. The entry point `main` / اصلی is part of the language: the compiler looks for it as the place to start.

Type names

English	Persian	Meaning
i8, i16, i32, i64	صحیح ۸, صحیح ۱۶, صحیح ۳۲, صحیح ۶۴	signed integers
u8, u16, u32, u64	طبیعی ۸, ...	unsigned integers
f32, f64	اعشاری ۳۲, اعشاری ۶۴	floating-point
bool	منطقی	boolean
char	کاراکتر	single byte / character
str	رشته	string
auto	خودکار	inferred type

The aliases `int` (= `i32`), `uint` (= `u32`), and `float` (= `f32`) are also accepted. Note that the digits inside the Persian type names (for example ۳۲ in صحیح ۳۲) are Persian digits, part of the keyword's spelling; numeric *values* in a program use the ordinary digits 0–9.

Appendix B

Grammar (Simplified)

This is a simplified, informal grammar of the Salam general language, in the style of EBNF, intended as a reader's reference rather than a precise specification. Square brackets [] mark an optional part, a star * means “zero or more,” a plus + means “one or more,” and a vertical bar | means “or.” It reflects the colon-and-end block style used throughout this book.

Program structure

```
program      = [ package ] import* definition*

package      = "package" IDENT

import       = "import" STRING
              | "import" "(" ( [ IDENT ] STRING )* ")"

definition   = function | struct | enum | interface
              | typealias | const | extern
```

Functions

```
function     = "func" IDENT [ generics ] "(" params ")" [ type ] ":"
              statement*
              "end"

params       = [ param ( "," param )* ]

param        = IDENT type
```

```

generics    = "<" typeparam ( "," typeparam )* ">"
typeparam   = IDENT [ ":" IDENT ]           // optional interface
            bound

```

Types

```

type        = basetype dimension* [ "*" ]
basetype    = "i8" | "i16" | "i32" | "i64"
              | "u8" | "u16" | "u32" | "u64"
              | "f32" | "f64"
              | "bool" | "char" | "str" | "void"
              | "int" | "uint" | "float"           // aliases
              | IDENT [ "<" type ( "," type )* ">" ] // struct/enum/
generic
              | "dyn" IDENT                       // dynamic interface
dimension    = "[" [ INT ] "]"                   // array

```

Declarations

```

vardecl     = [ "mut" ] IDENT type [ "=" expression ]
              | [ "mut" ] IDENT "auto" "=" expression
const       = "const" IDENT [ type ] "=" expression
typealias   = "type" IDENT "=" type

```

Statements

```

statement   = vardecl | const
              | assignment
              | expression           // e.g. a function or method
              call
              | if | while | for | repeat
              | "break" | "continue"
              | "ret" [ expression ]
              | "defer" statement
              | "println" args | "print" args

assignment = lvalue ( "=" | "+=" | "-=" | "*=" | "/=" | "%=" )
            expression

```



```
lvalue      = IDENT ( "." IDENT | "[" expression "]" ) *
args        = [ expression ( "," expression ) * ]
```

Control flow

```
if          = "if" expression ":" statement*
              ( "else" "if" expression ":" statement* ) *
              [ "else" ":" statement* ]
              "end"

while       = "while" expression ":" statement* "end"

for         = "for" vardecl ";" expression ";" assignment ":"
              statement*
              "end"

repeat      = "repeat" expression ":" statement* "end"
              | "repeat" expression "to" expression [ "every"
expression ] ":"
              statement*
              "end"
```

Structs, enums, and interfaces

```
struct      = "struct" IDENT [ generics ] ":"
              field*
              method*
              "end"

field       = IDENT type [ "=" expression ]

method      = function                                // defined inside the
              struct body

enum        = "enum" IDENT ":" member ( "," member ) * [ "," ] "end"
              | "enum" IDENT ":" ( member ) * "end"

member      = IDENT [ "=" INT ]

interface   = "interface" IDENT ":"
              signature*
              "end"

signature   = "func" IDENT "(" params ")" [ type ]    // no body
```

Expressions (by decreasing precedence)

```

primary      = INT | FLOAT | STRING | CHAR | "true" | "false" | "null"
              | IDENT
              | IDENT "(" args ")"                // call
              | IDENT "{" ( IDENT "=" expression ("," ...)* )? "}" //
struct literal
              | "[" ( expression ("," expression)* )? "]" //
array literal
              | "(" params ")" "=>" ( expression | ":" statement* "end"
              ) // lambda
              | "(" expression ")"

postfix      = primary ( "." IDENT [ "(" args ")" ] // field /
method
              | "[" expression "]" )* // index

unary       = [ "!" | "-" ] postfix
power       = unary [ "**" power ]
cast        = power [ "as" type ]
product     = cast ( ( "*" | "/" | "%" ) cast )*
sum         = product ( ( "+" | "-" ) product )*
compare     = sum ( ( "<" | ">" | "<=" | ">=" | "==" | "!=" ) sum )*
logic       = compare ( ( "&&" | "||" ) compare )*
expression  = logic

```

Foreign functions

```

extern      = [ link ] "extern" "\"C\"" ":"
              ( "func" IDENT "(" externparams ")" [ type ] )*
              "end"
externparams= [ param ( "," param )* [ "," "..." ] ] // ... =
              variadic
link        = "link" STRING

```

Lexical notes

- IDENT — a name: a letter or underscore, then letters, digits, or underscores. Unicode (including Persian) letters are allowed.
- INT — a whole number (type `i32` by default).

- **FLOAT** — a number with a decimal point or exponent (type **f64**).
- **STRING** — text in double quotes, or triple quotes (`"""..."""`); supports escapes `\n`, `\t`, `\\`, `\"`.
- **CHAR** — a single character in single quotes.
- Comments are `//` to end of line, or `/* ... */` across lines.
- Blocks open with `:` and close with `end`; a single-statement branch or body may follow the colon on the same line.

Appendix C

Cheat Sheet

A compact reference to Salam's syntax and the most-used standard-library calls. Every snippet is valid Salam; refer back to the named chapter for the full story.

Hello, world (Chapter 1)

```
1 func main:
2     println "Hello, Salam!"
3 end
```

Run it: `salam exec hello.salam` Build it: `salam build hello.salam --output=hello`

Variables and types (Chapters 2–3)

```
1 name str = "Sara"           // immutable
2 mut count i32 = 0           // mutable
3 const MAX = 100             // compile-time constant
4 x auto = 3.14               // inferred (f64)
5 y i32 = 250                 // i32 by default
6 b u8 = 250 as i32 as u8     // explicit conversion with 'as'
```

Types: `i8 i16 i32 i64, u8 u16 u32 u64, f32 f64, bool, char, str`.

Operators: `+ - * / %, ** (power, float), == != < > <= >=, && || !, += -= *= /= %=`.

Integer `/` truncates; no bitwise operators.

Control flow (Chapter 4)

```
1 if x > 0:
2     println "positive"
3 else if x == 0:
4     println "zero"
5 else:
6     println "negative"
7 end
```

Loops (Chapter 5)

```
1 while cond:          ... end
2 for mut i = 0; i < n; i = i + 1:  ... end
3 repeat 3:            ... end          // 3 times
4 repeat 1 to 5:        ... end          // 1..5 inclusive
5 repeat 0 to 20 every 2:  ... end      // step by 2
6 // break -- exit loop; continue -- next iteration
```

Functions (Chapter 6)

```
1 func add(a i32, b i32) i32:
2     ret a + b
3 end
4
5 func greet(name str):          // no return type = void
6     println "Hi,", name
7 end
```

Pass by value; no default or reference parameters; overloading by parameter types; definition order does not matter.

Arrays and vectors (Chapter 7)

```
1 a i32[3] = [1, 2, 3]           // fixed array, indexed 0..2
2 g i32[2][3] = [[1,2,3],[4,5,6]] // 2-D
3
4 mut v Vector<i32> = Vector {}
5 v.push(10)                   // append
```

```

6 v.get(0)[0]      // read element 0
7 v.set(0, 99)     // write
8 v.len()          // count
9 v.pop()          // remove last
10 v.free()        // release memory

```

Strings (Chapter 8)

```

1 import "str"
2 "a" + "b"        // concatenation
3 len("hello")     // 5
4 str.Upper(s) str.Lower(s) str.Trim(s) str.Reverse(s)
5 str.Find(s, sub) str.Contains(s, sub) str.Replace(s, a, b)
6 str.Substr(s, start, n)
7 str.Split(s, sep) str.Join(vec, sep)
8 str.FromInt(n as i64) str.ToInt(s)

```

Structs and methods (Chapter 9)

```

1 struct Point:
2     x i32 = 0
3     y i32 = 0
4     func move_by(dx i32, dy i32):
5         this.x = this.x + dx
6         this.y = this.y + dy
7     end
8 end
9
10 mut p Point = Point { x = 1, y = 2 }
11 p.move_by(3, 4)
12 println p.x, p.y

```

Enums (Chapter 10)

```

1 enum Day: Mon, Tue, Wed, Thu, Fri, Sat, Sun end
2 d Day = Day.Sat
3 println d as i32 // 5
4 // "match" with if / else if (no match statement)

```

Interfaces and generics (Chapters 11–12)

```
1 interface Shape:
2     func area() f64
3 end
4
5 func describe<T: Shape>(s T):    // static bound
6     println s.area()
7 end
8 func draw(s dyn Shape): ... end  // dynamic dispatch
9
10 func Max<T>(a T, b T) T:        // generic function
11     if a > b: ret a end
12     ret b
13 end
14 struct Box<T>: value T end      // generic struct
```

Closures (Chapter 13)

```
1 double auto = (x i32) => x * 2
2 apply((x i32) => x + 1, 5)      // lambda as argument
3 func make_adder(n i32) func(i32) i32:
4     ret (x i32) => x + n        // closure capturing n
5 end
```

Errors and defer (Chapter 14)

```
1 func withdraw(n i32) bool: ... end // bool success flag
2
3 import "option"
4 option.Some(x) option.IsSome(o) option.UnwrapOr(o, fallback)
5 Option { value = 0, present = false } // a "none"
6
7 defer v.free()                  // runs at scope exit (LIFO)
```

Modules (Chapter 15)

```
1 package main
2 import (
```



```

3     "math"
4     strutil "str"      // alias
5     "io"
6 )
7 pub func public_fn(): ... end           // exported; default is private

```

Standard library (Chapters 16–17)

```

1 io.Print(s) io.Println(s) io.Input() io.ReadFile(p) io.WriteFile(
   p, t)
2 fmt.Int(n as i64) fmt.Float(x) fmt.PadLeft(s, w)
3 math.Sqrt math.Pow math.Floor math.MaxI math.Gcd math.PI
4 rand.Seed(s as i64) rand.IntRange(lo, hi) rand.Float() rand.Bool()
5 time.Now() time.Format(t) time.Sleep(ms)
6 HashMap: insert(k,v) get(k) has(k) remove(k) size() free()
7 Stack: push/pop/peek; Queue: enqueue/dequeue
8 json.Valid json.Get json.GetInt json.Has
9 regex.MatchStr(pat, s) regex.FindStr(pat, s) regex.ReplaceStr(pat,
   s, w)
10 os.Exists os.Remove os.Mkdir os.ListDir os.Args os.Env os.Exit

```

Memory and FFI (Chapters 18–19)

```

1 import "mem"
2 mem.Allocate(n as u64) mem.Free(p) mem.Reallocate mem.Copy
3
4 extern "C":
5     func printf(format str, ...) i32
6     func sqrt(x f64) f64
7 end
8 link "sqlite3"           // link an external library

```

Concurrency (Chapter 20)

```

1 import "sync"
2 mut lock sync.Mutex = sync.NewMutex()
3 t i64 = spawn(work)           // start a thread
4 join(t)                       // wait for it
5 sync.Lock(lock) ... sync.Unlock(lock) // critical section
6 sync.Destroy(lock)

```

Testing (Chapter 21)

```
1 import "testing"
2 testing.AssertEqInt(got, want)
3 testing.AssertEqStr(got, want)
4 testing.AssertTrue(cond)
5 testing.Summary()           // prints "tests: N passed, M
                             failed"
```

The salam commands

Command	Does
salam exec f.salam	run with the interpreter (no C toolchain)
salam run f.salam	build and run, keep nothing
salam build f.salam --output=f	build a native executable
salam fmt f.salam	reformat source in place
salam new name	scaffold a new project
salam debug f.salam	build with debug info, launch a debugger
salam memcheck f.salam	build with AddressSanitizer and run
salam ... --lang=fa	treat the source as Persian

Appendix D

Solutions to Selected Exercises

Here are worked solutions to a selection of exercises from across the book. They are not the only correct answers — programming rewards many paths to the same result — but each is complete, idiomatic, and runs as shown. Try the exercise yourself before reading the solution; the struggle is where the learning happens.

Chapter 1 — Getting Started

Exercise 1.2 (print a labelled calculation):

```
1 func main:
2     println "12 * 8 =", 12 * 8
3 end
```

```
12 * 8 = 96
```

Chapter 2 — Variables and Data Types

Exercise 2.3 (average as a float):

```
1 func main:
2     sum i32 = 10 + 15 + 20
3     avg f64 = (sum as f64) / 3.0
4     println "average =", avg
5 end
```

```
average = 15
```

The casts to `f64` are what make this a real division; without them `45 / 3` would (here, coincidentally) also give 15, but for non-divisible totals integer division would be wrong.

Chapter 3 — Operators

Exercise 3.3 (between 1 and 12 inclusive):

```
1 func main:
2   n i32 = 7
3   in_range auto = n >= 1 && n <= 12
4   println n, "in 1..12?", in_range
5 end
```

```
7 in 1..12? true
```

Chapter 4 — Control Flow

Exercise 4.3 (the sign of a number):

```
1 func sign(n i32) str:
2   if n < 0: ret "negative"
3   else if n == 0: ret "zero"
4   else: ret "positive"
5   end
6 end
7
8 func main:
9   println sign(-5), sign(0), sign(42)
10 end
```

```
negative zero positive
```

Chapter 5 — Loops

Exercise 5.2 (sum 1 to 100):

```
1 func main:
2   mut total i32 = 0
3   for mut i = 1; i <= 100; i = i + 1:
4     total += i
5   end
6   println total
7 end
```

5050

Chapter 6 — Functions

Exercise 6.4 (recursive sum):

```
1 func sum_to(n i32) i32:
2   if n <= 0: ret 0 end // base case
3   ret n + sum_to(n - 1) // recursive step
4 end
5
6 func main:
7   println sum_to(10)
8 end
```

55

Chapter 7 — Arrays and Vectors

Exercise 7.3 (a vector of squares):

```
1 func main:
2   mut squares Vector<i32> = Vector {}
3   for mut i = 1; i <= 10; i = i + 1:
4     squares.push(i * i)
5   end
6   println "count:", squares.len()
7   mut j i32 = 0
8   while j < squares.len():
9     print squares.get(j)[0]
10    print " "
11    j += 1
12 end
```

```
13     println ""
14     squares.free()
15 end
```

```
count: 10
1 4 9 16 25 36 49 64 81 100
```

Chapter 8 — Strings

Exercise 8.4 (palindrome test):

```
1 import "str"
2
3 func is_palindrome(s str) bool:
4     ret str.Equals(s, str.Reverse(s))
5 end
6
7 func main:
8     println is_palindrome("level")
9     println is_palindrome("salam")
10 end
```

```
true
false
```

Chapter 9 — Structs and Methods

Exercise 9.1 (a circle's area):

```
1 struct Circle:
2     radius f64 = 0.0
3     func area() f64:
4         ret 3.14159 * this.radius * this.radius
5     end
6 end
7
8 func main:
9     c Circle = Circle { radius = 2.0 }
10    println "area:", c.area()
11 end
```

```
area: 12.5664
```

Chapter 10 — Enums

Exercise 10.3 (turn right):

```
1 enum Direction: North, East, South, West end
2
3 func turn_right(d Direction) Direction:
4     if d == Direction.North: ret Direction.East
5     else if d == Direction.East: ret Direction.South
6     else if d == Direction.South: ret Direction.West
7     else: ret Direction.North
8     end
9 end
10
11 func main:
12     mut d Direction = Direction.North
13     d = turn_right(d)
14     println "now facing index:", d as i32    // East = 1
15 end
```

```
now facing index: 1
```

Chapter 11 — Interfaces

Exercise 11.1 (a greeter interface):

```
1 interface Greeter:
2     func greet() str
3 end
4
5 struct Formal:
6     func greet() str: ret "Good evening." end
7 end
8
9 struct Casual:
10     func greet() str: ret "Hey!" end
11 end
12
13 func announce<T: Greeter>(g T):
```

```
14     println g.greet()
15 end
16
17 func main:
18     announce(Formal {})
19     announce(Casual {})
20 end
```

```
Good evening.
Hey!
```

Chapter 12 — Generics

Exercise 12.1 (a generic Min):

```
1 func Min<T>(a T, b T) T:
2     if a < b: ret a end
3     ret b
4 end
5
6 func main:
7     println Min(10, 20)
8     println Min(2.5, 1.5)
9 end
```

```
10
1.5
```

Chapter 13 — Closures

Exercise 13.3 (a multiplier factory):

```
1 func make_multiplier(factor i32) func(i32) i32:
2     ret (x i32) => x * factor
3 end
4
5 func main:
6     times3 auto = make_multiplier(3)
7     times10 auto = make_multiplier(10)
8     println times3(5)
9     println times10(5)
10 end
```



```
15
50
```

Chapter 14 — Errors and Defer

Exercise 14.2 (find the first negative):

```
1 func find_first_negative(a i32[5]) i32:
2     for mut i = 0; i < 5; i = i + 1:
3         if a[i] < 0:
4             ret i
5         end
6     end
7     ret -1
8 end
9
10 func main:
11     data i32[5] = [3, 8, -2, 5, -7]
12     println "first negative at:", find_first_negative(data)
13 end
```

```
first negative at: 2
```

Chapter 16 — Standard Library, Part 1

Exercise 16.3 (the hypotenuse):

```
1 import "math"
2
3 func main:
4     a f64 = 3.0
5     b f64 = 4.0
6     println "hypotenuse:", math.Sqrt(a * a + b * b)
7 end
```

```
hypotenuse: 5
```

Chapter 17 — Standard Library, Part 2

Exercise 17.1 (a map of ages):

```
1 func main:
2   mut ages HashMap<str, i32> = HashMap {}
3   ages.insert("Sara", 30)
4   ages.insert("Ali", 25)
5   ages.insert("Mina", 41)
6   println "Ali is", ages.get("Ali")
7   println "have Reza?", ages.has("Reza")
8   ages.free()
9 end
```

```
Ali is 25
have Reza? false
```

Chapter 21 — Testing

Exercise 21.1 (testing factorial):

```
1 import "testing"
2
3 func factorial(n i32) i32:
4   if n <= 1: ret 1 end
5   ret n * factorial(n - 1)
6 end
7
8 func test_factorial():
9   testing.AssertEqInt(factorial(0), 1)
10  testing.AssertEqInt(factorial(5), 120)
11  testing.AssertEqInt(factorial(3), 6)
12 end
13
14 func main:
15   test_factorial()
16   testing.Summary()
17 end
```

```
tests: 3 passed, 0 failed
```

Beyond these

The remaining exercises follow the same patterns shown here and in the chapters. If you get stuck, re-read the relevant section, run a smaller version

of the problem, and add `println` probes to see what your code is actually doing — the very habits from Chapter 21. Every program in this book was written by someone who started exactly where you are now.